University of Chittagong

Department of Computer Science and Engineering

CSE 615: Machine Learning

Lab Report

Thirteen Machine Learning Algorithms
Built from Scratch in Python

k-NN · Linear Regression · Gradient Descent · MLE and MAP · Naive Bayes
Logistic Regression · Perceptron · MLP · RBF Network · SVM · SVR
Decision Tree (ID3) · Genetic Algorithm

SUBMITTED BY

Department of CSE
University of Chittagong

SUBMITTED TO

[Teacher's Name]
[Designation]
Department of CSE
University of Chittagong

Date of submission: October 26, 2026

Contents

Introduction **1**

1 k-Nearest Neighbours **3**

1.1 Dataset 3
1.2 Code and experiments 3
1.3 Results 3
1.4 Source code 4

2 Linear and Polynomial Regression **10**

2.1 Dataset 10
2.2 Code and experiments 10
2.3 Results 11
2.4 Source code 12

3 Gradient Descent **17**

3.1 Dataset 17
3.2 Code and experiments 17
3.3 Results 17
3.4 Source code 18

4 Maximum Likelihood and Maximum A Posteriori Estimation **23**

4.1 Dataset 23
4.2 Code and experiments 23
4.3 Results 23
4.4 Source code 24

5 Naive Bayes Classifier **29**

5.1 Dataset 29
5.2 Code and experiments 29
5.3 Results 29
5.4 Source code 30

6 Logistic Regression **35**

6.1 Dataset 35
6.2 Code and experiments 35
6.3 Results 35
6.4 Source code 36

7 Perceptron **41**

7.1 Dataset 41
7.2 Code and experiments 41
7.3 Results 41
7.4 Source code 42

8 Multi-Layer Perceptron **47**

8.1 Dataset 47
8.2 Code and experiments 47
8.3 Results 47
8.4 Source code 48

9 Radial Basis Function Neural Network **53**

9.1 Dataset 53
9.2 Code and experiments 53
9.3 Results 53
9.4 Source code 54

10 Support Vector Machine 59
10.1 Dataset 59
10.2 Code and experiments 59
10.3 Results 59
10.4 Source code 60
11 Support Vector Regression 66
11.1 Dataset 66
11.2 Code and experiments 66
11.3 Results 66
11.4 Source code 67
12 Decision Tree (ID3) 72
12.1 Dataset 72
12.2 Code and experiments 72
12.3 Results 72
12.4 Source code 73
13 Genetic Algorithm 78
13.1 Dataset 78
13.2 Code and experiments 78
13.3 Results 78
13.4 Source code 79
Summary and References 84

Introduction

This report covers the algorithms we studied in CSE 615 (Machine Learning). For each one I wrote my own implementation in plain Python, picked a real dataset, ran the algorithm on it and measured how well it did. Every lab follows the same plan:

1. a description of the dataset and how I prepared it,
2. how the code is organised and what experiments it runs,
3. the results (metrics and tables), and
4. the full source code.

Ground rules I followed

- **No machine learning libraries.** The algorithms are written with NumPy and Pandas only. scikit-learn, PyTorch, TensorFlow and similar libraries are not used anywhere, not even for splitting the data or computing metrics. Matplotlib is used only to draw the figures.
- **Plain scripts, no notebooks.** Each lab is one `.py` file that runs from top to bottom with `python <script>.py`.
- **One folder per algorithm.** Every folder is self-contained: it has its own `data/` folder, its script, a `figures/` folder that the script fills, and a `results.txt` file with every number the script prints. Small helpers such as the train/test split are repeated in each script on purpose, so any folder can be copied and run on its own.
- **A different dataset for every algorithm.** Thirteen labs, thirteen datasets.
- **Repeatable results.** All random choices use the seed 42, so running a script again gives the same numbers as this report (only the timing numbers change from machine to machine).

The thirteen labs

Table 1. Algorithms, datasets and tasks covered in this report.

Lab	Algorithm	Dataset (size)	Task
1	k-Nearest Neighbours	Wine (178 × 13)	3-class classification
2	Linear / Polynomial Regression	California Housing (20,433 × 4)	Regression
3	Gradient Descent	Concrete Compressive Strength (1,030 × 8)	Regression
4	MLE and MAP estimation	Diabetes Progression (442 × 10)	Regression
5	Naive Bayes	SMS Spam Collection (5,572 messages)	Binary text classification
6	Logistic Regression	Breast Cancer Wisconsin (569 × 30)	Binary classification
7	Perceptron	Iris, two species (100 × 4)	Binary classification
8	Multi-Layer Perceptron	Iris (150 × 4)	3-class classification
9	RBF Neural Network	Wheat Seeds (210 × 7)	3-class classification
10	Support Vector Machine	Sonar, Mines vs. Rocks (208 × 60)	Binary classification
11	Support Vector Regression	Auto MPG (392 × 8)	Regression
12	Decision Tree (ID3)	Mushroom (8,124 × 22)	Binary classification
13	Genetic Algorithm	Breast Cancer Wisconsin (569 × 30)	Feature selection

All datasets are public. Most come from the UCI Machine Learning Repository. I converted every source file to a CSV with proper column names and kept it next to the script that uses it. Lab 2 uses California Housing, Labs 7 and 8 use Iris, Lab 12 uses Mushroom, and Lab 13 uses the Breast Cancer Wisconsin Diagnostic data.

How to run the code

You need Python 3.10 or newer with `numpy`, `pandas` and `matplotlib`.

```
cd "01_KNN"
python knn.py
```

The script prints its results, writes them to `results.txt`, and saves its plots in `figures/`. Most labs finish in a few seconds. The SVR lab takes about a minute, because it tries many parameter settings.

How I measured performance

The course topics on overfitting, cross-validation and the confusion matrix are not algorithms on their own, so instead of giving them separate labs I used them inside every lab.

Train, validation and test data. Every lab keeps a test set aside (20–30% of the data) and does not touch it until the very end. When a lab has to choose something (such as k in k-NN or C in an SVM), it uses *k-fold cross-validation* on the training part only: the training data is cut into 5 folds, the model is trained on 4 and scored on the 5th, and this is repeated so every fold is used once for scoring. For classification I use *stratified* splits, which keep the class proportions the same in every part. The MLP lab holds out validation and test portions from the Iris data instead.

Classification metrics. For two classes, the confusion matrix counts true positives (TP), false positives (FP), true negatives (TN) and false negatives (FN). From these:

Metric	Formula	What it answers
Accuracy	$(TP + TN)/N$, with N = all examples	How often is the model right?
Precision	$TP/(TP + FP)$	When it says “positive”, how often is that true?
Recall (sensitivity)	$TP/(TP + FN)$	How many of the real positives did it find?
Specificity	$TN/(TN + FP)$	How many of the real negatives did it leave alone?
F1 score	$2PR/(P + R)$, with P = precision, R = recall	One number that balances both
ROC AUC	area under the TPR–FPR curve	How well the scores rank positives above negatives

For more than two classes I compute precision, recall and F1 for each class (that class against the rest) and report their plain average, the *macro* average, so small classes count as much as big ones.

Regression metrics. With true values y_i and predictions $\hat{y}_i$:

$$\text{MAE} = \frac{1}{n}\sum_i |y_i - \hat{y}_i|, \qquad \text{RMSE} = \sqrt{\frac{1}{n}\sum_i (y_i - \hat{y}_i)^2}, \qquad R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}.$$

MAE and RMSE are in the units of the target (dollars, MPa, miles per gallon); R^2 is the share of the variation in y that the model explains, where 1 is perfect and 0 is no better than always guessing the mean.

LAB 1

k-Nearest Neighbours

Folder	`01_KNN`
Script	`knn.py`
Dataset	UCI Wine: 178 wines, 13 chemical measurements, 3 cultivars
Task	Multi-class classification
Main result	Test accuracy 92.5%, macro F1 0.924 (k = 17, Manhattan distance)

1.1 Dataset

The **Wine** dataset (UCI) contains the results of a chemical analysis of 178 wines grown in the same region of Italy but from three different cultivars. Each wine is described by 13 continuous measurements: alcohol, malic acid, ash, alcalinity of ash, magnesium, total phenols, flavanoids, non-flavanoid phenols, proanthocyanins, colour intensity, hue, OD280/OD315 of diluted wines and proline. The classes are a little unbalanced: 59, 71 and 48 wines.

The features live on very different scales (magnesium is around 70–162, proline 278–1680, while hue and non-flavanoid phenols are below 2), which makes this dataset a good test of why k-NN needs scaling. I split it with a stratified 70/30 split into 125 training and 53 test wines.

1.2 Code and experiments

The script `knn.py` is organised as follows.

- `stratified_split` and `stratified_folds` split indices while keeping the class mix.
- `Standardizer` learns the mean and standard deviation on the training rows and applies them to any other rows.
- `pairwise_distance` computes all query-to-training distances at once. For Euclidean distance it uses the identity $\|\mathbf{a}-\mathbf{b}\|^2 = \|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - 2\,\mathbf{a}^\top\mathbf{b}$, which turns the whole job into one matrix product.
- `KNNClassifier` has the usual `fit`/`predict` pair. `fit` only stores the data. `predict` sorts each row of the distance matrix, takes the first k indices and runs the vote, with the nearest-neighbour tie-break described above.
- `confusion_matrix` and `scores_from_confusion` compute accuracy and per-class precision, recall and F1.

The experiment has four parts: (1) 5-fold cross-validation on the training set for $k = 1, \dots, 25$ with three settings: Euclidean with scaling, Manhattan with scaling, and Euclidean without scaling; (2) the best (k, metric) pair is trained on the full training set and scored once on the test set; (3) the same model is tried without scaling and with distance-weighted votes; (4) to see what the decision boundary looks like, I project the data onto its first two principal components and colour the plane by the k-NN prediction.

1.3 Results

The cross-validation curves say the most important thing straight away: **scaling is not optional**. Without it the CV accuracy stays between 71% and 77% whatever k is; with it, accuracy jumps to about 95–99%.

With scaled features both distances work well; Manhattan is slightly more stable for small k. The best CV score was 99.2% for Manhattan distance with $k = 17$, so that is the final model.

Table 1.1. Test-set results of the final model (Manhattan distance, $k = 17$, 53 test wines).

Class	Precision	Recall	F1	Support
Cultivar 1	0.947	1.000	0.973	18
Cultivar 2	1.000	0.810	0.895	21
Cultivar 3	0.824	1.000	0.903	14
Macro average	0.924	0.937	0.924	53

Accuracy: **0.925** (49 of 53 correct)
Same k without scaling: 0.679 Distance-weighted voting: 0.925

The test accuracy (92.5%) is lower than the CV estimate (99.2%). With only 53 test wines, every single mistake costs almost 2 percentage points, so 4 mistakes instead of 0 or 1 is enough to explain the gap; it is a reminder that small test sets give noisy numbers. Distance weighting did not change any prediction here. Dropping the scaling, on the other hand, cost 25 percentage points on the test set as well.

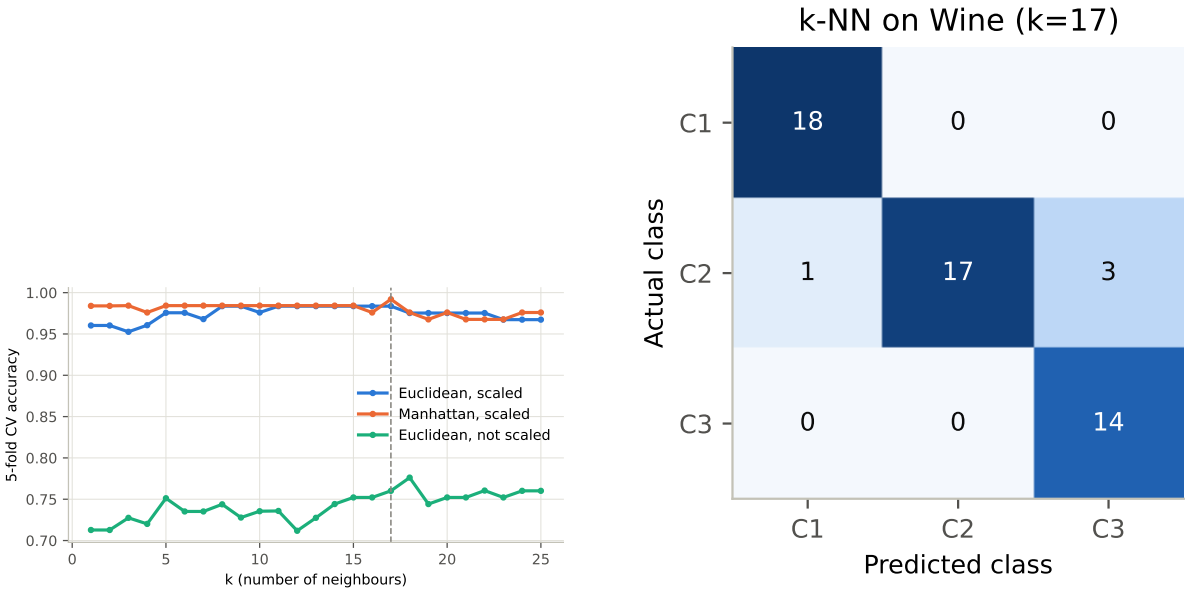


Figure 1.1. k-NN cross-validation curve and test confusion matrix.

1.4 Source code

Listing 1.1. `01_KNN/knn.py`

```
"""
Lab 01 - k-Nearest Neighbours (k-NN), written from scratch
Dataset : UCI Wine (178 wines, 13 chemical measurements, 3 cultivars)
Run     : python knn.py      (only NumPy, Pandas and Matplotlib are used)
"""
import os
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
```

```python
import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap, ListedColormap

HERE = os.path.dirname(os.path.abspath(__file__))
FIG = os.path.join(HERE, "figures")
os.makedirs(FIG, exist_ok=True)
SEED = 42

# ---------------------------------------------------------------- style
plt.rcParams.update({
    "font.size": 9, "axes.spines.top": False, "axes.spines.right": False,
    "axes.edgecolor": "#c3c2b7", "xtick.color": "#52514e", "ytick.color": "#52514e",
    "axes.grid": True, "grid.color": "#e1e0d9", "grid.linewidth": 0.6,
    "lines.linewidth": 2, "savefig.bbox": "tight",
})
BLUE, ORANGE, AQUA = "#2a78d6", "#eb6834", "#1baf7a"
SEQ = LinearSegmentedColormap.from_list("seq", ["#f4f8fd", "#86b6ef", "#2a78d6", "#0d366b"])

LINES = []
def log(text=""):
    """Print a line and keep it so we can save everything to results.txt."""
    print(text)
    LINES.append(str(text))


# ---------------------------------------------------------------- helpers
def stratified_split(y, test_frac, rng):
    """Return train/test indices, keeping the class balance the same in both."""
    test = []
    for c in np.unique(y):
        idx = rng.permutation(np.where(y == c)[0])
        test.extend(idx[:round(test_frac * len(idx))])
    mask = np.zeros(len(y), bool)
    mask[test] = True
    return np.where(~mask)[0], np.where(mask)[0]


def stratified_folds(y, n_folds, rng):
    """Split indices into n_folds groups, each with roughly the same class mix."""
    folds = [[] for _ in range(n_folds)]
    for c in np.unique(y):
        idx = rng.permutation(np.where(y == c)[0])
        for i, chunk in enumerate(np.array_split(idx, n_folds)):
            folds[i].extend(chunk)
    return [np.array(f) for f in folds]


class Standardizer:
    """z = (x - mean) / std, with mean and std learned from the training set only."""
    def fit(self, X):
        self.mu, self.sd = X.mean(0), X.std(0)
        self.sd[self.sd == 0] = 1.0
        return self

    def transform(self, X):
        return (X - self.mu) / self.sd
```

```python


# ---------------------------------------------------------------- k-NN itself
def pairwise_distance(A, B, metric="euclidean"):
    """Distance from every row of A to every row of B."""
    if metric == "euclidean":
        sq = (A ** 2).sum(1)[:, None] + (B ** 2).sum(1)[None, :] - 2 * A @ B.T
        return np.sqrt(np.maximum(sq, 0))
    if metric == "manhattan":
        return np.abs(A[:, None, :] - B[None, :, :]).sum(-1)
    raise ValueError(metric)


class KNNClassifier:
    def __init__(self, k=5, metric="euclidean", weighted=False):
        self.k, self.metric, self.weighted = k, metric, weighted

    def fit(self, X, y):
        # k-NN is a "lazy" learner: training is just remembering the data
        self.X, self.y = X, y
        return self

    def predict(self, Xq):
        D = pairwise_distance(Xq, self.X, self.metric)
        nearest = np.argsort(D, axis=1, kind="stable")[:, :self.k]
        preds = []
        for row, idx in enumerate(nearest):
            votes = {}
            for j in idx:
                w = 1.0 / (D[row, j] + 1e-9) if self.weighted else 1.0
                votes[self.y[j]] = votes.get(self.y[j], 0.0) + w
            best = max(votes.values())
            # tie-break: among tied classes, pick the one whose member is closest
            for j in idx:
                if votes[self.y[j]] == best:
                    preds.append(self.y[j])
                    break
        return np.array(preds)


# ---------------------------------------------------------------- metrics
def confusion_matrix(y_true, y_pred, labels):
    pos = {c: i for i, c in enumerate(labels)}
    M = np.zeros((len(labels), len(labels)), int)
    for t, p in zip(y_true, y_pred):
        M[pos[t], pos[p]] += 1
    return M


def scores_from_confusion(M):
    tp = np.diag(M).astype(float)
    precision = np.divide(tp, M.sum(0), out=np.zeros_like(tp), where=M.sum(0) > 0)
    recall = np.divide(tp, M.sum(1), out=np.zeros_like(tp), where=M.sum(1) > 0)
    f1 = np.divide(2 * precision * recall, precision + recall,
                   out=np.zeros_like(tp), where=(precision + recall) > 0)
    return tp.sum() / M.sum(), precision, recall, f1
```

```python


def cv_accuracy(X, y, k, metric, scale, n_folds=5):
    """Mean accuracy of k-NN over stratified k-fold cross-validation."""
    folds = stratified_folds(y, n_folds, np.random.default_rng(SEED))
    accs = []
    for f in range(n_folds):
        val = folds[f]
        tr = np.concatenate([folds[g] for g in range(n_folds) if g != f])
        Xtr, Xval = X[tr], X[val]
        if scale:
            s = Standardizer().fit(Xtr)
            Xtr, Xval = s.transform(Xtr), s.transform(Xval)
        pred = KNNClassifier(k, metric).fit(Xtr, y[tr]).predict(Xval)
        accs.append((pred == y[val]).mean())
    return float(np.mean(accs))


def plot_confusion(M, labels, path, title):
    fig, ax = plt.subplots(figsize=(3.3, 2.9))
    ax.imshow(M, cmap=SEQ)
    ax.grid(False)
    for i in range(M.shape[0]):
        for j in range(M.shape[1]):
            ax.text(j, i, M[i, j], ha="center", va="center",
                    color="white" if M[i, j] > M.max() / 2 else "#0b0b0b")
    ax.set_xticks(range(len(labels)), labels)
    ax.set_yticks(range(len(labels)), labels)
    ax.set_xlabel("Predicted class")
    ax.set_ylabel("Actual class")
    ax.set_title(title)
    fig.savefig(path)
    plt.close(fig)


# ---------------------------------------------------------------------- experiment
def main():
    rng = np.random.default_rng(SEED)
    df = pd.read_csv(os.path.join(HERE, "data", "wine.csv"))
    X = df.drop(columns="class").to_numpy(float)
    y = df["class"].to_numpy()
    labels = np.unique(y)
    log(f"Wine data: {X.shape[0]} samples, {X.shape[1]} features, "
        f"class counts {pd.Series(y).value_counts().sort_index().to_dict()}")

    tr, te = stratified_split(y, 0.30, rng)
    Xtr, Xte, ytr, yte = X[tr], X[te], y[tr], y[te]
    log(f"Train/test split: {len(tr)} / {len(te)}")

    # 1) choose k and the distance metric with 5-fold CV on the training set
    ks = list(range(1, 26))
    curves = {
        "Euclidean, scaled": [cv_accuracy(Xtr, ytr, k, "euclidean", True) for k in ks],
        "Manhattan, scaled": [cv_accuracy(Xtr, ytr, k, "manhattan", True) for k in ks],
        "Euclidean, not scaled": [cv_accuracy(Xtr, ytr, k, "euclidean", False) for k in ks],
    }
```

```
    log("\n5-fold CV accuracy on the training set (selected k values):")
    for name, acc in curves.items():
        log(f"  {name:24s} " + "  ".join(f"k={k}:{acc[k-1]:.3f}" for k in (1, 3, 5, 9, 15, 25)))

    best = max(((acc[k - 1], -k, name) for name, acc in curves.items()
                for k in ks if "not" not in name))
    best_acc, best_k, best_name = best[0], -best[1], best[2]
    metric = "euclidean" if best_name.startswith("Euclidean") else "manhattan"
    log(f"\nBest setting from CV: k={best_k}, metric={metric}, CV accuracy={best_acc:.3f}")

    fig, ax = plt.subplots(figsize=(5.6, 2.9))
    for (name, acc), c in zip(curves.items(), [BLUE, ORANGE, AQUA]):
        ax.plot(ks, acc, marker="o", ms=3, color=c, label=name)
    ax.axvline(best_k, color="#898781", lw=1, ls="--")
    ax.set_xlabel("k (number of neighbours)")
    ax.set_ylabel("5-fold CV accuracy")
    ax.legend(frameon=False, loc="center right")
    fig.savefig(os.path.join(FIG, "cv_accuracy_vs_k.pdf"))
    plt.close(fig)

    # 2) final model on the full training set, scored once on the test set
    s = Standardizer().fit(Xtr)
    model = KNNClassifier(best_k, metric).fit(s.transform(Xtr), ytr)
    pred = model.predict(s.transform(Xte))
    M = confusion_matrix(yte, pred, labels)
    acc, P, R, F = scores_from_confusion(M)
    log("\nTest-set results (final model)")
    log(f"  Accuracy        : {acc:.4f}")
    log(f"  Macro precision : {P.mean():.4f}")
    log(f"  Macro recall    : {R.mean():.4f}")
    log(f"  Macro F1        : {F.mean():.4f}")
    for c, p, r, f in zip(labels, P, R, F):
        log(f"  class {c}: precision={p:.3f} recall={r:.3f} f1={f:.3f}")
    log(f"  Confusion matrix (rows = actual, cols = predicted):\n{M}")
    plot_confusion(M, [f"C{c}" for c in labels], os.path.join(FIG, "confusion_matrix.pdf"),
                   f"k-NN on Wine (k={best_k})")

    # 3) two quick comparisons on the same test set
    raw = KNNClassifier(best_k, metric).fit(Xtr, ytr).predict(Xte)
    wtd = KNNClassifier(best_k, metric, weighted=True).fit(s.transform(Xtr), ytr).predict(s.transform(Xte
     ↪ ))
    log(f"\nSame k without feature scaling -> test accuracy {np.mean(raw == yte):.4f}")
    log(f"Distance-weighted voting       -> test accuracy {np.mean(wtd == yte):.4f}")

    # 4) picture of the decision regions in 2-D (first two principal components)
    Z = s.transform(Xtr)
    _, _, Vt = np.linalg.svd(Z - Z.mean(0), full_matrices=False)
    P2tr = Z @ Vt[:2].T
    gx, gy = np.meshgrid(np.linspace(P2tr[:, 0].min() - 1, P2tr[:, 0].max() + 1, 250),
                         np.linspace(P2tr[:, 1].min() - 1, P2tr[:, 1].max() + 1, 250))
    grid = np.c_[gx.ravel(), gy.ravel()]
    fig, axes = plt.subplots(1, 2, figsize=(6.4, 2.9), sharey=True)
    region_cmap = ListedColormap(["#cde2fb", "#fbd9cb", "#c6efdf"])
    for ax, k in zip(axes, [1, best_k]):
        g = KNNClassifier(k).fit(P2tr, ytr).predict(grid).reshape(gx.shape)
        ax.contourf(gx, gy, g, levels=[0.5, 1.5, 2.5, 3.5], cmap=region_cmap)
```

```
        for c, col in zip(labels, [BLUE, ORANGE, AQUA]):
            m = ytr == c
            ax.scatter(P2tr[m, 0], P2tr[m, 1], s=12, color=col, edgecolor="white", lw=0.4, label=f"Class
    ↪ {c}")
        ax.set_title(f"k = {k}")
        ax.set_xlabel("PC 1")
        ax.grid(False)
    axes[0].set_ylabel("PC 2")
    axes[1].legend(frameon=False, loc="lower right", fontsize=7)
    fig.savefig(os.path.join(FIG, "decision_regions.pdf"))
    plt.close(fig)

    with open(os.path.join(HERE, "results.txt"), "w") as f:
        f.write("\n".join(LINES) + "\n")


if __name__ == "__main__":
    main()
```

LAB 2

Linear and Polynomial Regression

Folder	`02_Linear_Regression`
Script	`linear_regression.py`
Dataset	California Housing: 20,433 complete houses, 4 mapped numeric inputs
Task	Regression (median house value in lakh USD)
Main result	Test $R^2 = 0.621$, RMSE 0.712 lakh USD (degree-3 polynomial, chosen by cross-validation)

2.1 Dataset

For this lab I used the public **California Housing** dataset. It contains 20,640 census-block records; 207 rows with missing bedroom counts were removed, leaving 20,433 complete houses. Four numeric columns are mapped to the inputs used by the script: total rooms, median income, total bedrooms and median house age. The target is median house value in USD, reported in lakh USD:

Table 2.1. Columns used from the California Housing dataset.

Column	Meaning and range
Square_Feet	Total rooms (area proxy)
Location_Score	Median income (economic/location proxy)
Num_Bedrooms	Total bedrooms
Building_Age	Median house age
Price_USD	**Target**: median house value in USD

All four inputs are numeric, so no categorical encoding is needed. Prices are reported in lakh USD. I used a random 80/20 split: 16,347 houses for training and 4,086 for testing.

2.2 Code and experiments

- `load_data` reads the CSV and drops the two text columns.
- `polynomial_features(Z, degree)` builds Φ: a column of ones followed by every product of the inputs up to the given total degree, using `itertools.combinations_with_replacement`. With 4 inputs this gives 5, 15, 35, 70, 126 and 210 columns for degrees 1 to 6.
- `Standardizer` scales the inputs with the training mean and standard deviation.
- `fit_simple` uses the two textbook formulas; `fit_normal_equation` solves the normal equation with `np.linalg.solve`.
- `regression_scores` returns MAE, RMSE and R^2, and `cv_rmse_polynomial` runs 5-fold cross-validation for a given degree, re-learning the scaling inside every fold.

Experiments: (1) simple regression of price on square feet; (2) multiple linear regression on all four inputs; (3) polynomial models of degree 1 to 6, compared by training, CV and test error; (4) the degree with the lowest CV error as the final model.

2.3 Results

Table 2.2. Polynomial degree study (four standardised inputs; errors in lakh USD).

Degree	Weights	Train RMSE	5-fold CV RMSE	Test RMSE	cond$(\Phi^\top\Phi)$
1	5	5.021	5.036	4.888	1.3×10^{1}
2	15	4.977	5.026	4.881	7.3×10^{2}
3	35	4.935	5.073	4.944	6.6×10^{4}
4	70	4.882	13.614	18.905	5.7×10^{16}
5	126	4.802	11 682.6	409.544	1.2×10^{19}
6	210	4.721	149 298.6	3 108.803	3.3×10^{19}

Table 2.2 is the overfitting picture from the lectures, with real numbers. The training RMSE falls steadily from 5.02 to 4.72 lakh as the degree grows, while the cross-validation error is lowest at degree 2 and then rises. Two things happen at the high degrees:

- **Overfitting.** From degree 3 on, the extra terms fit noise in the training houses, so the error on new houses grows even though the training error shrinks.
- **The normal equation breaks down.** The number of bedrooms takes only four values (2, 3, 4, 5). For a variable with four values, b^4 is an exact combination of $1, b, b^2, b^3$, so from degree 4 on some columns of Φ are copies of others. The condition number of $\Phi^\top\Phi$ jumps from 10^4 to 10^{16}, which is the limit of double-precision arithmetic, and the solution becomes meaningless. This is a nice practical reason to keep the degree low.

Table 2.3. Final comparison (errors in lakh USD).

		Training			Test			5-fold CV
Model	**#w**	MAE	RMSE	R^2	MAE	RMSE	R^2	RMSE
Simple (square feet only)	2	6.473	7.886	0.751	6.626	8.094	0.707	–
Multiple linear (4 inputs)	5	3.985	5.021	0.899	3.893	4.888	0.893	5.036
Polynomial, degree 2	15	3.956	4.977	0.901	3.898	4.881	0.894	**5.026**

Floor area alone explains about 71% of the variation in price on the test set. Adding location, bedrooms and age brings this to 89% and cuts the RMSE from 8.09 to 4.89 lakh. The degree-2 polynomial was chosen by cross-validation, but its advantage is tiny: 0.007 lakh (about 700 taka) on the test RMSE, and 0.01 lakh on the CV RMSE. In other words, within the range of this data the price is almost a linear function of the inputs, and the squared and product terms add very little. Its largest non-linear weights (square feet × bedrooms +0.80, bedrooms^2 −0.58, age^2 −0.25 and square feet^2 +0.23 lakh) are small next to the linear ones (square feet +13.8, location +5.9 lakh, on standardised inputs).

Table 2.4. Weights of the multiple linear model in real units.

Term	Weight
Intercept	−727,844 BDT
Square feet	+3,343 BDT per square foot
Location score	+803,044 BDT per point
Bedrooms	+86,965 BDT per bedroom
Building age	−22,110 BDT per year

The linear weights are easy to read: every extra square foot adds about 3,300 taka, moving one point

up the location scale (for example from Kotbari at 2.5 to Bagichagaon at 3.5) adds about 8 lakh, and each year of age takes away about 22,000 taka. The bedroom weight is smaller than one might expect because bedrooms and floor area move together (correlation 0.86), so most of the bedroom effect is already counted through square feet.

The residuals have a standard deviation of 4.87 lakh and no visible pattern against the prediction. This looks like random noise that was added when the data was generated, and no model built from these four inputs can remove it. So an RMSE of about 4.9 lakh is close to the best that is possible here.

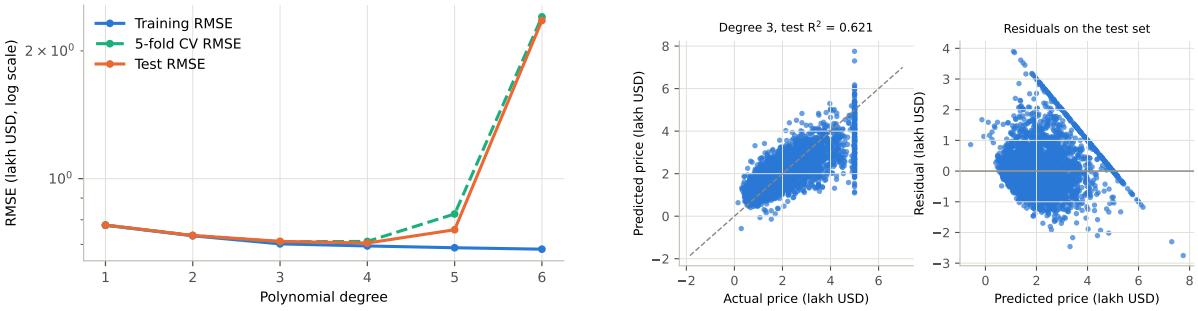


Figure 2.1. Polynomial degree study and final-model residual diagnostics.

2.4 Source code

Listing 2.1. `02_Linear_Regression/linear_regression.py`

```
"""
Lab 02 - Linear and Polynomial Regression (closed form / normal equation), written from scratch
 Dataset : California Housing (public regression dataset, 20,640 houses)
Run     : python linear_regression.py      (only NumPy, Pandas and Matplotlib)
"""
import os
from itertools import combinations_with_replacement
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap

HERE = os.path.dirname(os.path.abspath(__file__))
FIG = os.path.join(HERE, "figures")
os.makedirs(FIG, exist_ok=True)
SEED = 42
LAKH = 1e5                      # prices are reported in lakh USD (1 lakh = 100,000 USD)

plt.rcParams.update({
    "font.size": 9, "axes.spines.top": False, "axes.spines.right": False,
    "axes.edgecolor": "#c3c2b7", "xtick.color": "#52514e", "ytick.color": "#52514e",
    "axes.grid": True, "grid.color": "#e1e0d9", "grid.linewidth": 0.6,
    "lines.linewidth": 2, "savefig.bbox": "tight",
})
BLUE, ORANGE, AQUA, MUTED = "#2a78d6", "#eb6834", "#1baf7a", "#898781"

LINES = []
def log(text=""):
```

```
    print(text)
    LINES.append(str(text))


# ---------------------------------------------------------------- data
INPUTS = ["Square_Feet", "Location_Score", "Num_Bedrooms", "Building_Age"]


def load_data():
    path = os.path.join(HERE, "data", "california_housing.csv")
    if not os.path.isfile(path):
        raise FileNotFoundError(
            f"Dataset not found: {path}. Put california_housing.csv "
            "inside the 02_Linear_Regression/data folder."
        )
    df = pd.read_csv(path).dropna()
    return df.rename(columns={
        "total_rooms": "Square_Feet",
        "median_income": "Location_Score",
        "total_bedrooms": "Num_Bedrooms",
        "housing_median_age": "Building_Age",
        "median_house_value": "Price_USD",
    })


# ---------------------------------------------------------------- design matrices
def polynomial_features(Z, degree):
    """Bias column, then every product of the inputs up to the given total degree.
    For degree 2 and inputs a, b this gives: 1, a, b, a^2, ab, b^2."""
    cols, names = [np.ones(len(Z))], ["1"]
    for d in range(1, degree + 1):
        for combo in combinations_with_replacement(range(Z.shape[1]), d):
            cols.append(np.prod(Z[:, combo], axis=1))
            names.append("*".join(INPUTS[i] for i in combo))
    return np.column_stack(cols), names


class Standardizer:
    def fit(self, X):
        self.mu, self.sd = X.mean(0), X.std(0)
        return self

    def transform(self, X):
        return (X - self.mu) / self.sd


# ---------------------------------------------------------------- the model
def fit_simple(x, y):
    """Textbook formulas for y = w0 + w1 x (least squares)."""
    w1 = np.sum((x - x.mean()) * (y - y.mean())) / np.sum((x - x.mean()) ** 2)
    w0 = y.mean() - w1 * x.mean()
    return np.array([w0, w1])


def fit_normal_equation(Phi, y):
    """Solve (Phi^T Phi) w = Phi^T y. This is the least-squares solution."""
```

```
    A = Phi.T @ Phi
    b = Phi.T @ y
    try:
        return np.linalg.solve(A, b)
    except np.linalg.LinAlgError:          # if A is singular, fall back to the pseudo-inverse
        return np.linalg.pinv(Phi) @ y


# ---------------------------------------------------------------- metrics
def regression_scores(y, yhat):
    err = y - yhat
    mae = np.mean(np.abs(err))
    rmse = np.sqrt(np.mean(err ** 2))
    r2 = 1 - np.sum(err ** 2) / np.sum((y - y.mean()) ** 2)
    return mae, rmse, r2


def cv_rmse_polynomial(X, y, degree, n_folds=5):
    """5-fold cross-validated RMSE of a polynomial model (scaling is learned inside each fold)."""
    folds = np.array_split(np.random.default_rng(SEED).permutation(len(y)), n_folds)
    out = []
    for f in range(n_folds):
        val = folds[f]
        tr = np.concatenate([folds[g] for g in range(n_folds) if g != f])
        s = Standardizer().fit(X[tr])
        Ptr, _ = polynomial_features(s.transform(X[tr]), degree)
        Pval, _ = polynomial_features(s.transform(X[val]), degree)
        w = fit_normal_equation(Ptr, y[tr])
        out.append(regression_scores(y[val], Pval @ w)[1])
    return float(np.mean(out))


# ---------------------------------------------------------------- experiment
def main():
    df = load_data()
    X = df[INPUTS].to_numpy(float)
    y = df["Price_USD"].to_numpy(float) / LAKH          # work in lakh USD
    log(f"California houses: {len(df)} rows, "
        f"price {y.min():.1f}-{y.max():.1f} lakh USD (mean {y.mean():.1f})")

    idx = np.random.default_rng(SEED).permutation(len(y))
    n_test = int(0.2 * len(y))
    te, tr = idx[:n_test], idx[n_test:]
    log(f"Train/test split: {len(tr)} / {len(te)}\n")

    # 1) simple linear regression: price ~ square feet
    w_simple = fit_simple(X[tr, 0], y[tr])
    pred_simple = lambda Xp: w_simple[0] + w_simple[1] * Xp[:, 0]

    # 2) multiple linear regression on the raw inputs (weights stay in real units)
    Phi_raw = np.column_stack([np.ones(len(X)), X])
    w_multi = fit_normal_equation(Phi_raw[tr], y[tr])

    # 3) polynomial regression: choose the degree with 5-fold CV on the training set
    log("Polynomial degree study (all four inputs, standardised):")
    log(f"{'degree':>6s} {'#weights':>8s} {'train RMSE':>10s} {'CV RMSE':>9s} {'test RMSE':>9s} {'cond(
```

```
    ↪ Phi^T Phi)':>15s}")
s = Standardizer().fit(X[tr])
degrees = [1, 2, 3, 4, 5, 6]
study = {}
for d in degrees:
    Ptr, _ = polynomial_features(s.transform(X[tr]), d)
    Pte, _ = polynomial_features(s.transform(X[te]), d)
    w = fit_normal_equation(Ptr, y[tr])
    study[d] = (Ptr.shape[1], regression_scores(y[tr], Ptr @ w)[1],
                cv_rmse_polynomial(X[tr], y[tr], d), regression_scores(y[te], Pte @ w)[1])
    log(f"{d:6d} {study[d][0]:8d} {study[d][1]:10.3f} {study[d][2]:9.3f} {study[d][3]:9.3f} "
        f"{np.linalg.cond(Ptr.T @ Ptr):15.2e}")
best_d = min(degrees, key=lambda d: study[d][2])
log(f"Degree with the lowest CV RMSE: {best_d}\n")

Ptr, names = polynomial_features(s.transform(X[tr]), best_d)
Pte, _ = polynomial_features(s.transform(X[te]), best_d)
w_poly = fit_normal_equation(Ptr, y[tr])

# 4) compare the models
rows = [
    ("Simple (square feet only)", 2, pred_simple(X[tr]), pred_simple(X[te]), None),
    ("Multiple linear (4 inputs)", 5, Phi_raw[tr] @ w_multi, Phi_raw[te] @ w_multi, study[1][2]),
    (f"Polynomial, degree {best_d}", Ptr.shape[1], Ptr @ w_poly, Pte @ w_poly, study[best_d][2]),
]
log(f"{'Model':28s} {'#w':>3s} {'trMAE':>6s} {'trRMSE':>6s} {'trR2':>6s} "
    f"{'teMAE':>6s} {'teRMSE':>6s} {'teR2':>6s} {'cvRMSE':>6s}   (lakh USD)")
for name, k, ptr, pte, cv in rows:
    a, b = regression_scores(y[tr], ptr), regression_scores(y[te], pte)
    cv_txt = f"{cv:6.3f}" if cv is not None else f"{'-':>6s}"
    log(f"{name:28s} {k:3d} {a[0]:6.3f} {a[1]:6.3f} {a[2]:6.4f} {b[0]:6.3f} {b[1]:6.3f} {b[2]:6.4f} {
    ↪ cv_txt}")

log("\nMultiple linear model in real units:")
log(f"   intercept               {w_multi[0] * LAKH:12,.0f} USD")
for name, w in zip(INPUTS, w_multi[1:]):
    log(f"   {name:22s} {w * LAKH:+12,.0f} USD per unit")

log(f"\nDegree-{best_d} weights (standardised inputs, lakh USD), largest first:")
for i in np.argsort(-np.abs(w_poly))[:8]:
    log(f"   {names[i]:32s} {w_poly[i]:+8.3f}")

# --- figure 1: price against square feet, coloured by location score
fig, ax = plt.subplots(figsize=(5.8, 3.1))
ramp = LinearSegmentedColormap.from_list("ramp", ["#9ec5f4", "#2a78d6", "#0d366b"])
sc = ax.scatter(X[:, 0], y, c=X[:, 1], cmap=ramp, s=6, alpha=0.8)
xs = np.linspace(X[:, 0].min(), X[:, 0].max(), 50)
ax.plot(xs, w_simple[0] + w_simple[1] * xs, color=ORANGE, lw=1.8, label="Simple fit: price ~ square
    ↪ feet")
fig.colorbar(sc, ax=ax, label="Location score")
ax.set_xlabel("Square feet")
ax.set_ylabel("Price (lakh USD)")
ax.legend(frameon=False, loc="upper left")
fig.savefig(os.path.join(FIG, "price_vs_sqft.pdf"))
plt.close(fig)

```

```
    # --- figure 2: the overfitting curve
    fig, ax = plt.subplots(figsize=(5.6, 3.0))
    ax.plot(degrees, [study[d][1] for d in degrees], marker="o", ms=4, color=BLUE, label="Training RMSE")
    ax.plot(degrees, [study[d][2] for d in degrees], marker="o", ms=4, color=AQUA, ls="--", label="5-fold CV RMSE")
    ax.plot(degrees, [study[d][3] for d in degrees], marker="o", ms=4, color=ORANGE, label="Test RMSE")
    ax.set_yscale("log")
    ax.set_xlabel("Polynomial degree")
    ax.set_ylabel("RMSE (lakh USD, log scale)")
    ax.legend(frameon=False, loc="upper left")
    fig.savefig(os.path.join(FIG, "degree_vs_rmse.pdf"))
    plt.close(fig)

    # --- figure 3: predicted vs actual and residuals of the chosen model
    pte = Pte @ w_poly
    fig, axes = plt.subplots(1, 2, figsize=(6.6, 2.9))
    axes[0].scatter(y[te], pte, s=8, color=BLUE, alpha=0.7)
    lim = [y.min() - 2, y.max() + 2]
    axes[0].plot(lim, lim, color=MUTED, lw=1, ls="--")
    axes[0].set_xlabel("Actual price (lakh USD)")
    axes[0].set_ylabel("Predicted price (lakh USD)")
    axes[0].set_title(f"Degree {best_d}, test R$^2$ = {regression_scores(y[te], pte)[2]:.3f}", fontsize=8.5)
    axes[1].scatter(pte, y[te] - pte, s=8, color=BLUE, alpha=0.7)
    axes[1].axhline(0, color=MUTED, lw=1)
    axes[1].set_xlabel("Predicted price (lakh USD)")
    axes[1].set_ylabel("Residual (lakh USD)")
    axes[1].set_title("Residuals on the test set", fontsize=8.5)
    fig.savefig(os.path.join(FIG, "predicted_and_residuals.pdf"))
    plt.close(fig)
    log(f"\nTest residuals: mean {np.mean(y[te] - pte):+.3f}, std {np.std(y[te] - pte):.3f} lakh USD")

    with open(os.path.join(HERE, "results.txt"), "w") as f:
        f.write("\n".join(LINES) + "\n")


if __name__ == "__main__":
    main()
```

LAB 3

Gradient Descent

Folder	`03_Gradient_Descent`
Script	`gradient_descent.py`
Dataset	UCI Concrete Compressive Strength: 1,030 concrete mixes, 8 inputs
Task	Regression (linear model trained by gradient descent)
Main result	Test $R^2 = 0.835$, RMSE 6.66 MPa; SGD lands on the exact least-squares weights

3.1 Dataset

The **Concrete Compressive Strength** dataset (UCI, Yeh 1998) has 1,030 concrete mixes. Seven inputs give the amount of each ingredient in kg per m^3 of mix (cement, blast-furnace slag, fly ash, water, superplasticizer, coarse aggregate, fine aggregate), and the eighth is the age of the sample in days (1 to 365). The target is the compressive strength in MPa, from 2.3 to 82.6 (mean 35.8).

Concrete gains most of its strength in the first few weeks and then levels off, so strength grows roughly with the *logarithm* of age. I therefore used $\log(\text{age})$ instead of age; this single change is what allows a linear model to do well here. The inputs were standardised with the training statistics, and the data was split 80/20 (824 training, 206 test mixes).

3.2 Code and experiments

- `mse_loss` and `mse_gradient` are the two formulas above, written with matrix products.
- `gradient_descent(X, y, lr, epochs, batch_size)` is a single function for all three variants. `batch_size=None` means batch GD, `32` means mini-batch, `1` means SGD. It shuffles the rows each epoch, applies the update for every batch, records the loss after each epoch, and stops early if the loss becomes infinite.
- The script also computes the exact least-squares weights $\mathbf{w}^*$ with the normal equation and the largest eigenvalue of $X^\top X/n$, so the gradient-descent results can be checked against the theory.

The experiments are: (1) the three variants for 200 epochs each; (2) batch GD with five learning rates, the last one just above the theoretical limit; (3) a picture of the path that gradient descent takes on a two-weight version of the problem.

3.3 Results

The largest eigenvalue of $X^\top X/n$ was 2.199, so theory says batch GD is stable only for $\eta < 2/2.199 = 0.909$.

Table 3.1. The three variants after 200 epochs. $\|\mathbf{w} - \mathbf{w}^*\|$ is the distance to the exact least-squares weights. Times are from my run and will differ on other machines.

Method	Updates	Time	Train J	Test MAE	Test RMSE	Test R^2	$\|\mathbf{w} - \mathbf{w}^*\|$
Batch GD, $\eta = 0.1$	200	0.01 s	27.000	5.374	6.840	0.825 8	7.449
Mini-batch (32), $\eta = 0.01$	5,200	0.05 s	26.246	5.242	6.697	0.833 0	2.740
SGD, $\eta = 0.001$	164,800	1.1 s	26.131	5.182	6.656	0.835 0	0.100
Normal equation (exact)	–	–	26.128	5.184	6.659	0.834 9	0.000

All three methods end up with almost the same test error as the exact solution (test R^2 between 0.826 and 0.835). SGD came within 0.1 of the exact weights. Batch GD, after its 200 updates, still has a loss of 27.0 instead of 26.13, but its weights are surprisingly far away (distance 7.4) even though its predictions are nearly as good. The reason is that the seven ingredient amounts are tied to each other: they describe one cubic metre of concrete, so their total is almost the same for every mix (about 2,340 kg, varying by only about 3%). The loss surface therefore has a long, flat valley in which very different weight vectors give almost the same predictions, and gradient descent crawls along that valley slowly.

Table 3.2. Batch gradient descent with different learning rates (200 epochs, exact minimum $J = 26.128$).

η	0.001	0.01	0.1	0.5	0.955
Final training loss	525.6	44.49	27.00	26.13	diverged

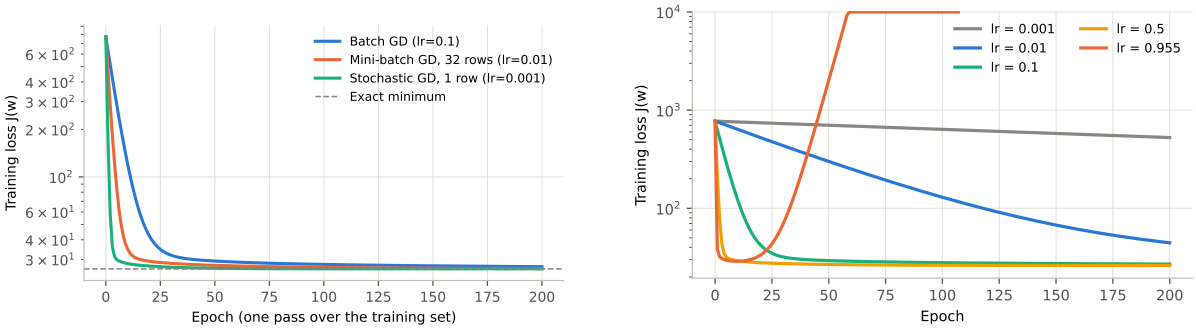


Figure 3.1. Loss curves for GD variants and learning-rate stability.

3.4 Source code

Listing 3.1. `03_Gradient_Descent/gradient_descent.py`

```
"""
Lab 03 - Gradient Descent (batch, mini-batch and stochastic), written from scratch
Task    : fit a linear regression model by gradient descent instead of the normal equation
Dataset : UCI Concrete Compressive Strength (1,030 concrete mixes, 8 inputs, strength in MPa)
Run     : python gradient_descent.py      (only NumPy, Pandas and Matplotlib)
"""
import os
import time
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

```

```python
HERE = os.path.dirname(os.path.abspath(__file__))
FIG = os.path.join(HERE, "figures")
os.makedirs(FIG, exist_ok=True)
SEED = 42

plt.rcParams.update({
    "font.size": 9, "axes.spines.top": False, "axes.spines.right": False,
    "axes.edgecolor": "#c3c2b7", "xtick.color": "#52514e", "ytick.color": "#52514e",
    "axes.grid": True, "grid.color": "#e1e0d9", "grid.linewidth": 0.6,
    "lines.linewidth": 2, "savefig.bbox": "tight",
})
BLUE, ORANGE, AQUA, YELLOW, MUTED = "#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#898781"

LINES = []
def log(text=""):
    print(text)
    LINES.append(str(text))


# ---------------------------------------------------------------- loss and gradient
def mse_loss(X, y, w):
    """J(w) = (1/2n) * sum (Xw - y)^2   (X already contains a column of ones)."""
    r = X @ w - y
    return 0.5 * np.mean(r ** 2)


def mse_gradient(X, y, w):
    """dJ/dw = (1/n) * X^T (Xw - y)."""
    return X.T @ (X @ w - y) / len(y)


def gradient_descent(X, y, lr, epochs, batch_size=None, seed=SEED):
    """
    One function for all three flavours:
      batch_size = None -> batch GD     (one update per epoch, uses all rows)
      batch_size = 32   -> mini-batch   (n/32 updates per epoch)
      batch_size = 1    -> stochastic   (n updates per epoch)
    Returns the weights and the training loss after every epoch.
    """
    rng = np.random.default_rng(seed)
    n, d = X.shape
    w = np.zeros(d)
    bs = n if batch_size is None else batch_size
    history = [mse_loss(X, y, w)]
    for _ in range(epochs):
        order = rng.permutation(n)                # shuffle once per epoch
        for start in range(0, n, bs):
            rows = order[start:start + bs]
            w = w - lr * mse_gradient(X[rows], y[rows], w)
        history.append(mse_loss(X, y, w))
        if not np.isfinite(history[-1]) or history[-1] > 1e8:
            break                                 # the run has blown up
    return w, np.array(history)


# ---------------------------------------------------------------- helpers
```

```
def add_bias(X):
    return np.column_stack([np.ones(len(X)), X])


def regression_scores(y, yhat):
    err = y - yhat
    return (np.mean(np.abs(err)), np.sqrt(np.mean(err ** 2)),
            1 - np.sum(err ** 2) / np.sum((y - y.mean()) ** 2))


# ------------------------------------------------------------------ experiment
def main():
    df = pd.read_csv(os.path.join(HERE, "data", "concrete.csv"))
    # strength grows roughly with the logarithm of age, so we feed log(age)
    df["log_age"] = np.log(df["age_days"])
    features = ["cement", "slag", "fly_ash", "water", "superplasticizer",
                "coarse_aggregate", "fine_aggregate", "log_age"]
    X_raw = df[features].to_numpy(float)
    y = df["strength_mpa"].to_numpy(float)
    log(f"Concrete data: {len(y)} mixes, {len(features)} inputs, "
        f"strength {y.min():.1f}-{y.max():.1f} MPa (mean {y.mean():.1f})")

    idx = np.random.default_rng(SEED).permutation(len(y))
    n_test = int(0.2 * len(y))
    te, tr = idx[:n_test], idx[n_test:]
    mu, sd = X_raw[tr].mean(0), X_raw[tr].std(0)        # scale with training statistics
    Xtr, Xte = add_bias((X_raw[tr] - mu) / sd), add_bias((X_raw[te] - mu) / sd)
    ytr, yte = y[tr], y[te]
    log(f"Train/test split: {len(tr)} / {len(te)}")

    # the largest safe learning rate for batch GD is 2 / (largest eigenvalue of X^T X / n)
    lam_max = np.linalg.eigvalsh(Xtr.T @ Xtr / len(ytr)).max()
    log(f"Largest eigenvalue of X^T X / n = {lam_max:.3f} -> batch GD is stable for lr < {2 / lam_max:.3f
    ↪ }")

    # reference answer from the normal equation (Lab 02)
    w_exact = np.linalg.solve(Xtr.T @ Xtr, Xtr.T @ ytr)
    log(f"Normal-equation training loss J = {mse_loss(Xtr, ytr, w_exact):.4f}\n")

    # 1) the three flavours of gradient descent
    runs = {
        "Batch GD (lr=0.1)": dict(lr=0.1, batch_size=None),
        "Mini-batch GD, 32 rows (lr=0.01)": dict(lr=0.01, batch_size=32),
        "Stochastic GD, 1 row (lr=0.001)": dict(lr=0.001, batch_size=1),
    }
    epochs = 200
    histories = {}
    log(f"{'Method':34s} {'updates':>8s} {'time(s)':>8s} {'train J':>8s} "
        f"{'test MAE':>9s} {'test RMSE':>9s} {'test R2':>8s} {'|w-w*|':>8s}")
    for name, cfg in runs.items():
        t0 = time.perf_counter()
        w, hist = gradient_descent(Xtr, ytr, cfg["lr"], epochs, cfg["batch_size"])
        secs = time.perf_counter() - t0
        histories[name] = hist
        bs = len(ytr) if cfg["batch_size"] is None else cfg["batch_size"]
        updates = (len(hist) - 1) * int(np.ceil(len(ytr) / bs))
```

```
        mae, rmse, r2 = regression_scores(yte, Xte @ w)
        log(f"{name:34s} {updates:8d} {secs:8.2f} {hist[-1]:8.3f} "
            f"{mae:9.3f} {rmse:9.3f} {r2:8.4f} {np.linalg.norm(w - w_exact):8.3f}")
    mae, rmse, r2 = regression_scores(yte, Xte @ w_exact)
    log(f"{'Normal equation (exact)':34s} {'-':>8s} {'-':>8s} {mse_loss(Xtr, ytr, w_exact):8.3f} "
        f"{mae:9.3f} {rmse:9.3f} {r2:8.4f} {0:8.3f}")

    fig, ax = plt.subplots(figsize=(5.6, 3.0))
    for (name, hist), c in zip(histories.items(), [BLUE, ORANGE, AQUA]):
        ax.plot(np.arange(len(hist)), hist, color=c, label=name)
    ax.axhline(mse_loss(Xtr, ytr, w_exact), color=MUTED, ls="--", lw=1, label="Exact minimum")
    ax.set_yscale("log")
    ax.set_xlabel("Epoch (one pass over the training set)")
    ax.set_ylabel("Training loss J(w)")
    ax.legend(frameon=False, fontsize=8)
    fig.savefig(os.path.join(FIG, "gd_variants_loss.pdf"))
    plt.close(fig)

    # 2) effect of the learning rate on batch GD
    lrs = [0.001, 0.01, 0.1, 0.5, round(1.05 * 2 / lam_max, 3)]
    log("\nLearning-rate study (batch GD, 200 epochs):")
    fig, ax = plt.subplots(figsize=(5.6, 3.0))
    for lr, c in zip(lrs, [MUTED, BLUE, AQUA, YELLOW, ORANGE]):
        _, hist = gradient_descent(Xtr, ytr, lr, epochs)
        status = "diverged" if (not np.isfinite(hist[-1]) or hist[-1] > hist[0]) else f"final J = {hist
    ↪ [-1]:.3f}"
        log(f"   lr = {lr:<6} -> {status}")
        ax.plot(np.arange(len(hist)), np.clip(hist, None, 1e4), color=c, label=f"lr = {lr}")
    ax.set_yscale("log")
    ax.set_ylim(top=1e4)
    ax.set_xlabel("Epoch")
    ax.set_ylabel("Training loss J(w)")
    ax.legend(frameon=False, fontsize=8, ncol=2)
    fig.savefig(os.path.join(FIG, "learning_rate_effect.pdf"))
    plt.close(fig)

    # 3) picture: GD paths on a 2-weight version of the problem (water and superplasticizer only;
    #    these two inputs are correlated, so the loss bowl is a stretched ellipse)
    Z = Xtr[:, [3, 4]]                    # standardized water and superplasticizer
    yc = ytr - ytr.mean()                 # centre y so no bias weight is needed
    w_star = np.linalg.solve(Z.T @ Z, Z.T @ yc)
    lam2 = np.linalg.eigvalsh(Z.T @ Z / len(yc)).max()
    a = np.linspace(w_star[0] - 12, w_star[0] + 12, 200)
    b = np.linspace(w_star[1] - 12, w_star[1] + 12, 200)
    A, B = np.meshgrid(a, b)
    J = np.array([[mse_loss(Z, yc, np.array([p, q])) for p in a] for q in b])
    fig, axes = plt.subplots(1, 3, figsize=(6.6, 2.5), sharey=True)
    big = round(1.9 / lam2, 2)
    for ax, lr, title in zip(axes, [0.05, 0.5, big],
                             ["Small step (lr = 0.05)", "Good step (lr = 0.5)", f"Too big (lr = {big})"]
    ↪ :
        ax.contour(A, B, J, levels=np.quantile(J, np.linspace(0.01, 0.9, 12)), colors="#c3c2b7",
    ↪ linewidths=0.6)
        w = np.array([w_star[0] - 10, w_star[1] - 6])
        path = [w.copy()]
        for _ in range(20):
```

```
            w = w - lr * mse_gradient(Z, yc, w)
            path.append(w.copy())
        path = np.array(path)
        ax.plot(path[:, 0], path[:, 1], marker="o", ms=2.5, lw=1.1, color=BLUE)
        ax.scatter(*w_star, marker="*", s=80, color=ORANGE, zorder=5)
        ax.set_xlim(a[0], a[-1])
        ax.set_ylim(b[0], b[-1])
        ax.set_aspect("equal")
        ax.set_title(title, fontsize=8)
        ax.set_xlabel("w (water)")
        ax.grid(False)
    axes[0].set_ylabel("w (superplasticizer)")
    fig.savefig(os.path.join(FIG, "gd_paths_contour.pdf"))
    plt.close(fig)
    log(f"\nContour picture: water/superplasticizer, largest eigenvalue {lam2:.3f}, "
        f"'too big' step = {big} (limit {2 / lam2:.3f})")

    with open(os.path.join(HERE, "results.txt"), "w") as f:
        f.write("\n".join(LINES) + "\n")


if __name__ == "__main__":
    main()
```

LAB 4

Maximum Likelihood and Maximum A Posteriori Estimation

Folder	`04_MLE_MAP`
Script	`mle_map.py`
Dataset	Diabetes progression (Efron et al., 2004): 442 patients, 10 baseline measurements
Task	Regression with 64 polynomial features
Main result	MAP: test RMSE 59.6, $R^2 = 0.385$; MLE: test RMSE 63.3, $R^2 = 0.308$

4.1 Dataset

The **Diabetes** dataset (Efron, Hastie, Johnstone and Tibshirani, 2004) has 442 diabetes patients. For each patient there are ten baseline measurements: age, sex, body mass index, average blood pressure and six blood serum measurements (s1–s6). The target is a number (25–346) that measures how much the disease progressed one year later.

To create a setting where MLE and MAP actually differ, I gave the model a lot of freedom: all ten inputs, their squares (except for sex, which only has two values) and all 45 pairwise products, so 64 features in total. With only 310 training patients (70/30 split, 132 for testing), that is a lot of weights to estimate from little data, which is exactly the situation where a prior should help.

4.2 Code and experiments

- `poly2` builds the 64 polynomial features, and `Prep` standardises them and centres y using training statistics only.
- `fit_mle` solves least squares (with `np.linalg.lstsq`, which also copes with the case of more weights than examples) and returns $\hat{\sigma}^2$.
- `fit_map` solves $(\Phi^\top \Phi + \lambda I)\mathbf{w} = \Phi^\top \mathbf{y}$.
- `gaussian_nll` measures how probable the test targets are under each fitted model.
- `choose_lambda` tries 50 values of λ between 10^{-3} and 10^{4} with 5-fold cross-validation.

Besides the main comparison, the script draws a *learning curve*: it trains both estimators on random subsets of 60 to 310 patients (20 repeats each) and records the median test RMSE.

4.3 Results

Cross-validation chose $\lambda = 7.20$. With the MAP noise estimate $\hat{\sigma}^2 = 2396$ this corresponds to a prior standard deviation of $\tau = \sqrt{\sigma^2/\lambda} \approx 18.3$ on each standardised weight.

Table 4.1. MLE against MAP on the diabetes data (64 features, 310 training patients).

Estimator	Data	MAE	RMSE	R^2	NLL	$\|\mathbf{w}\|$
MLE	train	34.99	44.98	0.660 6	5.225	3 209.7
MLE	test	51.38	63.25	0.307 8	5.714	3 209.7
MAP	train	38.97	48.95	0.598 0	5.310	64.3
MAP	test	49.36	59.61	0.385 3	5.551	64.3
Reference: plain MLE with only the 10 raw inputs, test RMSE 58.66, $R^2 = 0.4046$						

MLE fits the training patients better (R^2 0.66 against 0.60) but generalises worse (test R^2 0.31 against 0.39). That is overfitting, plain and simple. The weight norms show why: MLE uses weights fifty times larger than MAP, with big positive and negative values that cancel each other on the training data but not on new patients. MAP also gives the test targets a higher probability (lower negative log-likelihood).

This is the classic regularisation picture. Training error only goes up as λ grows, but test and CV error first go down (less overfitting) and then up again (the prior is so strong that the model cannot fit anything).

Table 4.2. Median test RMSE for small training sets (20 random subsets each).

Training patients	60	80	100	150	200	310
MLE	543.5	190.7	118.3	77.3	67.1	63.3
MAP	59.8	60.0	59.1	59.6	59.4	59.6

The learning curve shows the biggest difference. With 60 patients there are fewer equations than weights (64), MLE has infinitely many perfect fits, and the one it returns is useless (RMSE 543 on a target whose standard deviation is 77). MAP is almost unaffected by the amount of data; the prior supplies what the data cannot. As the training set grows, MLE slowly catches up, as theory says it should.

One honest observation: the plain model with only the ten raw inputs (test $R^2 = 0.40$) is still slightly better than MAP with 64 features. The extra polynomial features do not carry much new information for this dataset, and MAP's main achievement is to stop them from doing harm.

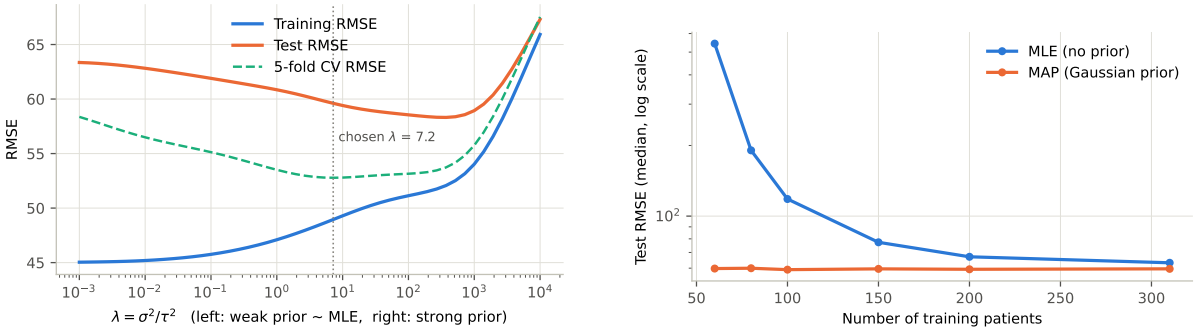


Figure 4.1. The effect of prior strength and training-set size on MLE and MAP.

4.4 Source code

Listing 4.1. `04_MLE_MAP/mle_map.py`

```
"""
Lab 04 - Maximum Likelihood (MLE) vs Maximum A Posteriori (MAP) estimation, from scratch
Model   : linear regression on degree-2 polynomial features, with Gaussian noise
          MLE -> ordinary least squares
```

```
          MAP -> least squares + Gaussian prior on the weights (= ridge / L2 penalty)
Dataset : Diabetes progression (Efron et al., 2004): 442 patients, 10 baseline measurements
Run     : python mle_map.py      (only NumPy, Pandas and Matplotlib)
"""
import os
from itertools import combinations
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

HERE = os.path.dirname(os.path.abspath(__file__))
FIG = os.path.join(HERE, "figures")
os.makedirs(FIG, exist_ok=True)
SEED = 42

plt.rcParams.update({
    "font.size": 9, "axes.spines.top": False, "axes.spines.right": False,
    "axes.edgecolor": "#c3c2b7", "xtick.color": "#52514e", "ytick.color": "#52514e",
    "axes.grid": True, "grid.color": "#e1e0d9", "grid.linewidth": 0.6,
    "lines.linewidth": 2, "savefig.bbox": "tight",
})
BLUE, ORANGE, AQUA, MUTED = "#2a78d6", "#eb6834", "#1baf7a", "#898781"

LINES = []
def log(text=""):
    print(text)
    LINES.append(str(text))


# ---------------------------------------------------------------------- features
def poly2(X, names):
    """All original columns, their squares (skip binary columns) and all pairwise products."""
    cols, out_names = [X[:, i] for i in range(X.shape[1])], list(names)
    for i, n in enumerate(names):
        if len(np.unique(X[:, i])) > 2:
            cols.append(X[:, i] ** 2)
            out_names.append(f"{n}^2")
    for i, j in combinations(range(X.shape[1]), 2):
        cols.append(X[:, i] * X[:, j])
        out_names.append(f"{names[i]}*{names[j]}")
    return np.column_stack(cols), out_names


class Prep:
    """Standardize inputs and centre the target, using training data only."""
    def fit(self, X, y):
        self.mu, self.sd, self.ym = X.mean(0), X.std(0), y.mean()
        self.sd[self.sd == 0] = 1
        return self

    def X(self, X):
        return (X - self.mu) / self.sd


```

```python
# ---------------------------------------------------------------- the two estimators
def fit_mle(Phi, y):
    """
    Maximise  log p(y | w, sigma^2) = -n/2 log(2 pi sigma^2) - ||y - Phi w||^2 / (2 sigma^2).
    Setting the gradient to zero gives the least-squares answer, and sigma^2 = mean squared residual.
    """
    w = np.linalg.lstsq(Phi, y, rcond=None)[0]
    sigma2 = np.mean((y - Phi @ w) ** 2)
    return w, sigma2


def fit_map(Phi, y, lam):
    """
    Maximise  log p(y | w) + log p(w)  with prior  w ~ N(0, tau^2 I).
    The prior adds  -||w||^2 / (2 tau^2), so the answer is  (Phi^T Phi + lam I)^-1 Phi^T y
    with  lam = sigma^2 / tau^2.
    """
    d = Phi.shape[1]
    w = np.linalg.solve(Phi.T @ Phi + lam * np.eye(d), Phi.T @ y)
    sigma2 = np.mean((y - Phi @ w) ** 2)
    return w, sigma2


def gaussian_nll(y, yhat, sigma2):
    """Average negative log-likelihood of the targets under N(yhat, sigma2)."""
    return np.mean(0.5 * np.log(2 * np.pi * sigma2) + (y - yhat) ** 2 / (2 * sigma2))


def scores(y, yhat):
    err = y - yhat
    return (np.mean(np.abs(err)), np.sqrt(np.mean(err ** 2)),
            1 - np.sum(err ** 2) / np.sum((y - y.mean()) ** 2))


def choose_lambda(Phi, y, lambdas, n_folds=5, seed=SEED):
    """Pick lam (i.e. the prior strength) by k-fold cross-validation."""
    folds = np.array_split(np.random.default_rng(seed).permutation(len(y)), n_folds)
    cv = []
    for lam in lambdas:
        errs = []
        for f in range(n_folds):
            val = folds[f]
            tr = np.concatenate([folds[g] for g in range(n_folds) if g != f])
            p = Prep().fit(Phi[tr], y[tr])
            w, _ = fit_map(p.X(Phi[tr]), y[tr] - p.ym, lam)
            errs.append(np.mean((y[val] - (p.X(Phi[val]) @ w + p.ym)) ** 2))
        cv.append(np.mean(errs))
    return lambdas[int(np.argmin(cv))], np.array(cv)


# ---------------------------------------------------------------- experiment
def main():
    df = pd.read_csv(os.path.join(HERE, "data", "diabetes.csv"))
    names = [c for c in df.columns if c != "progression"]
    X = df[names].to_numpy(float)
    y = df["progression"].to_numpy(float)
```

```
    Phi, phi_names = poly2(X, names)
    log(f"Diabetes data: {len(y)} patients, {len(names)} raw inputs -> {Phi.shape[1]} polynomial features ")

    idx = np.random.default_rng(SEED).permutation(len(y))
    n_test = int(0.3 * len(y))
    te, tr = idx[:n_test], idx[n_test:]
    log(f"Train/test split: {len(tr)} / {len(te)}\n")

    p = Prep().fit(Phi[tr], y[tr])
    Ptr, Pte, ytr_c = p.X(Phi[tr]), p.X(Phi[te]), y[tr] - p.ym

    lambdas = np.logspace(-3, 4, 50)
    lam_best, cv_curve = choose_lambda(Phi[tr], y[tr], lambdas)
    log(f"Lambda chosen by 5-fold CV: {lam_best:.2f}")

    w_mle, s2_mle = fit_mle(Ptr, ytr_c)
    w_map, s2_map = fit_map(Ptr, ytr_c, lam_best)
    tau2 = s2_map / lam_best
    log(f"Noise variance estimate sigma^2: MLE {s2_mle:.1f}, MAP {s2_map:.1f}")
    log(f"Implied prior variance tau^2 = sigma^2 / lambda = {tau2:.2f} (tau = {np.sqrt(tau2):.2f})\n")

    log(f"{'Estimator':8s} {'split':6s} {'MAE':>7s} {'RMSE':>7s} {'R2':>7s} {'NLL':>7s} {'||w||':>8s}")
    for name, w, s2 in [("MLE", w_mle, s2_mle), ("MAP", w_map, s2_map)]:
        for split, P_, yy in [("train", Ptr, y[tr]), ("test", Pte, y[te])]:
            yhat = P_ @ w + p.ym
            mae, rmse, r2 = scores(yy, yhat)
            log(f"{name:8s} {split:6s} {mae:7.2f} {rmse:7.2f} {r2:7.4f} "
                f"{gaussian_nll(yy, yhat, s2):7.3f} {np.linalg.norm(w):8.1f}")

    # baseline: plain linear model with only the 10 raw inputs (MLE)
    p10 = Prep().fit(X[tr], y[tr])
    w10, _ = fit_mle(p10.X(X[tr]), y[tr] - p10.ym)
    mae, rmse, r2 = scores(y[te], p10.X(X[te]) @ w10 + p10.ym)
    log(f"\nBaseline (10 raw inputs, MLE) test: MAE={mae:.2f} RMSE={rmse:.2f} R2={r2:.4f}")

    # --- figure 1: train/test error as the prior gets stronger
    tr_rmse, te_rmse = [], []
    for lam in lambdas:
        w, _ = fit_map(Ptr, ytr_c, lam)
        tr_rmse.append(scores(y[tr], Ptr @ w + p.ym)[1])
        te_rmse.append(scores(y[te], Pte @ w + p.ym)[1])
    fig, ax = plt.subplots(figsize=(5.6, 3.0))
    ax.plot(lambdas, tr_rmse, color=BLUE, label="Training RMSE")
    ax.plot(lambdas, te_rmse, color=ORANGE, label="Test RMSE")
    ax.plot(lambdas, np.sqrt(cv_curve), color=AQUA, ls="--", lw=1.5, label="5-fold CV RMSE")
    ax.axvline(lam_best, color=MUTED, lw=1, ls=":")
    ax.text(lam_best * 1.2, 56, f"chosen $\\lambda$ = {lam_best:.1f}", color="#52514e", fontsize=8)
    ax.set_xscale("log")
    ax.set_xlabel("$\\lambda = \\sigma^2/\\tau^2$   (left: weak prior ~ MLE,  right: strong prior)")
    ax.set_ylabel("RMSE")
    ax.legend(frameon=False, loc="upper center")
    fig.savefig(os.path.join(FIG, "rmse_vs_lambda.pdf"))
    plt.close(fig)

    # --- figure 2: how much data does each estimator need?
```

```
    sizes = [60, 80, 100, 150, 200, len(tr)]
    rng = np.random.default_rng(SEED)
    curve = {"MLE": [], "MAP": []}
    log("\nLearning curve (median test RMSE over 20 random training subsets):")
    for n in sizes:
        e_mle, e_map = [], []
        for _ in range(20 if n < len(tr) else 1):
            sub = rng.choice(tr, n, replace=False) if n < len(tr) else tr
            q = Prep().fit(Phi[sub], y[sub])
            wm, _ = fit_mle(q.X(Phi[sub]), y[sub] - q.ym)
            lam, _ = choose_lambda(Phi[sub], y[sub], lambdas)
            wp, _ = fit_map(q.X(Phi[sub]), y[sub] - q.ym, lam)
            e_mle.append(scores(y[te], q.X(Phi[te]) @ wm + q.ym)[1])
            e_map.append(scores(y[te], q.X(Phi[te]) @ wp + q.ym)[1])
        curve["MLE"].append(np.median(e_mle))
        curve["MAP"].append(np.median(e_map))
        log(f"   n = {n:3d}: MLE {curve['MLE'][-1]:8.2f}   MAP {curve['MAP'][-1]:6.2f}")
    fig, ax = plt.subplots(figsize=(5.6, 2.8))
    ax.plot(sizes, curve["MLE"], marker="o", ms=4, color=BLUE, label="MLE (no prior)")
    ax.plot(sizes, curve["MAP"], marker="o", ms=4, color=ORANGE, label="MAP (Gaussian prior)")
    ax.set_yscale("log")
    ax.set_xlabel("Number of training patients")
    ax.set_ylabel("Test RMSE (median, log scale)")
    ax.legend(frameon=False)
    fig.savefig(os.path.join(FIG, "learning_curve.pdf"))
    plt.close(fig)

    # --- figure 3: the weights themselves (sorted by size, log scale)
    fig, ax = plt.subplots(figsize=(5.6, 2.7))
    ranks = np.arange(1, len(w_mle) + 1)
    ax.plot(ranks, np.sort(np.abs(w_mle))[::-1], marker="o", ms=3, lw=1.2, color=BLUE,
            label=f"MLE  (||w|| = {np.linalg.norm(w_mle):,.0f})")
    ax.plot(ranks, np.sort(np.abs(w_map))[::-1], marker="o", ms=3, lw=1.2, color=ORANGE,
            label=f"MAP  (||w|| = {np.linalg.norm(w_map):,.0f})")
    ax.set_yscale("log")
    ax.set_xlabel("Weight rank (largest first)")
    ax.set_ylabel("|weight| (log scale)")
    ax.legend(frameon=False)
    fig.savefig(os.path.join(FIG, "weights_mle_vs_map.pdf"))
    plt.close(fig)

    with open(os.path.join(HERE, "results.txt"), "w") as f:
        f.write("\n".join(LINES) + "\n")


if __name__ == "__main__":
    main()
```

LAB 5

Naive Bayes Classifier

Folder	`05_Naive_Bayes`
Script	`naive_bayes.py`
Dataset	UCI SMS Spam Collection: 5,572 text messages (747 spam)
Task	Binary text classification (spam or ham)
Main result	Test accuracy 99.0%, spam F1 0.963, ROC AUC 0.991 ($\alpha = 0.25$)

5.1 Dataset

The **SMS Spam Collection** (Almeida and Gómez Hidalgo, 2011; UCI) contains 5,572 real English text messages labelled *ham* (normal, 4,827) or *spam* (747, i.e. 13.4%). A typical spam message looks like "WINNER!! As a valued network customer you have been selected to receive a £900 prize reward! To claim call 0906…". Spam messages are longer on average (about 24 words against 14).

Preparation was deliberately simple: lower-case everything and keep runs of letters, digits and the £ and $ signs as words. No stemming and no stop-word removal. I used a stratified 80/20 split (4,460 training and 1,114 test messages). The training messages contain 7,735 different words.

Because the classes are unbalanced, accuracy alone would be misleading (always answering "ham" already scores 86.6%), so I focus on precision, recall and F1 for the spam class.

5.2 Code and experiments

- `tokenize` is a one-line regular expression.
- `MultinomialNaiveBayes.fit` builds the vocabulary, counts words per class and stores $\log P(c)$ and the smoothed $\log P(w \mid c)$ for every word. With $\alpha = 0$, unseen words get $\log 0 = -\infty$ on purpose, to show the zero-frequency problem.
- `log_scores` adds up the log-probabilities of the known words; `predict` takes the larger score; `spam_score` returns the score difference for the ROC curve.
- `binary_metrics`, `roc_curve` and `auc` compute the evaluation numbers.

Experiments: (1) 5-fold cross-validation over nine values of α from 0 to 5; (2) test-set comparison of no smoothing, Laplace ($\alpha = 1$) and the CV choice; (3) the most "spammy" words, i.e. the largest $\log P(w \mid \text{spam}) - \log P(w \mid \text{ham})$.

5.3 Results

Table 5.1. 5-fold cross-validation F1 score (spam class) for different smoothing values.

α	0	0.01	0.05	0.1	0.25	0.5	1	2	5
CV F1	0.784	0.940	0.945	0.946	**0.949**	0.943	0.943	0.928	0.867

Table 5.2. Test-set results (1,114 messages, spam is the positive class).

Model	Accuracy	Precision	Recall	F1	Specificity	AUC
No smoothing ($\alpha = 0$)	0.955 1	0.971 4	0.684 6	0.803 1	0.996 9	0.973 0
Laplace ($\alpha = 1$)	0.991 0	0.986 0	0.946 3	0.965 8	0.997 9	0.987 5
CV choice ($\alpha = 0.25$)	0.990 1	0.966 2	0.959 7	0.963 0	0.994 8	0.991 3

Smoothing is essential. Without it, 1,027 of the 1,114 test messages contain at least one word that has zero probability in one of the classes. Those messages are decided by which class happened to see the rarest word rather than by their content, and recall falls to 68%: a third of the spam gets through. Any α between 0.01 and 1 fixes this.

The CV choice $\alpha = 0.25$ and Laplace smoothing are practically tied on the test set; the difference is two or three messages. $\alpha = 0.25$ catches two more spam messages at the cost of three more false alarms. Both reach about 99% accuracy, which is far above the 86.6% of the always-ham baseline.

The learned word statistics are easy to interpret: *claim*, *prize*, *guaranteed*, *awarded*, prices such as *150p* and *£1000*, and *ringtone*/*tones* are typical of SMS spam. Such an easy explanation is one of the nice things about Naive Bayes.

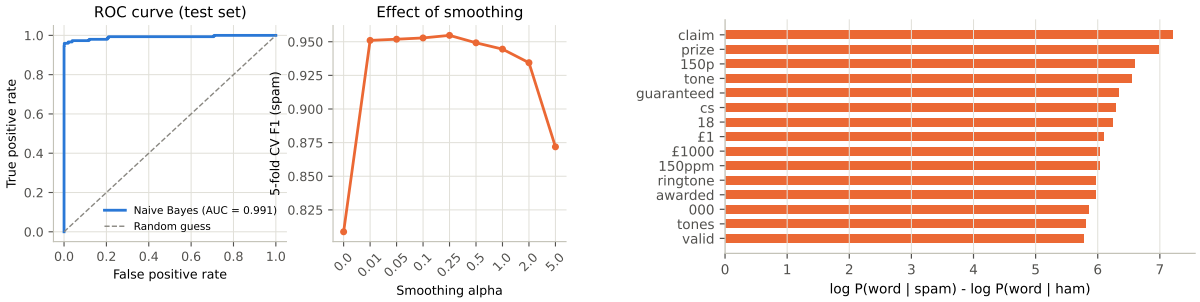


Figure 5.1. ROC and smoothing comparison, with the most spam-indicative words.

5.4 Source code

Listing 5.1. `05_Naive_Bayes/naive_bayes.py`

```
"""
Lab 05 - Naive Bayes classifier (multinomial, with Laplace smoothing), written from scratch
Dataset : UCI SMS Spam Collection (5,574 text messages labelled "ham" or "spam")
Run     : python naive_bayes.py      (only NumPy, Pandas, Matplotlib and Python's re module)
"""
import os
import re
from collections import Counter
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap

HERE = os.path.dirname(os.path.abspath(__file__))
FIG = os.path.join(HERE, "figures")
os.makedirs(FIG, exist_ok=True)
SEED = 42

```

```
plt.rcParams.update({
    "font.size": 9, "axes.spines.top": False, "axes.spines.right": False,
    "axes.edgecolor": "#c3c2b7", "xtick.color": "#52514e", "ytick.color": "#52514e",
    "axes.grid": True, "grid.color": "#e1e0d9", "grid.linewidth": 0.6,
    "lines.linewidth": 2, "savefig.bbox": "tight",
})
BLUE, ORANGE, MUTED = "#2a78d6", "#eb6834", "#898781"
SEQ = LinearSegmentedColormap.from_list("seq", ["#f4f8fd", "#86b6ef", "#2a78d6", "#0d366b"])

LINES = []
def log(text=""):
    print(text)
    LINES.append(str(text))


# ---------------------------------------------------------------- text -> words
def tokenize(message):
    """Lower-case the text and keep runs of letters, digits and currency signs."""
    return re.findall(r"[a-z0-9£$]+", message.lower())


# ---------------------------------------------------------------- the classifier
class MultinomialNaiveBayes:
    """
    P(class | words) is proportional to  P(class) * product over words of P(word | class).
    'Naive' = we assume the words are independent once the class is known.
    """
    def __init__(self, alpha=1.0):
        self.alpha = alpha          # alpha = 1 is Laplace smoothing, alpha = 0 is no smoothing

    def fit(self, docs, labels):
        self.classes = sorted(set(labels))
        self.vocab = sorted({w for d in docs for w in d})
        self.index = {w: i for i, w in enumerate(self.vocab)}
        V = len(self.vocab)
        self.log_prior, self.log_likelihood = {}, {}
        for c in self.classes:
            counts = np.zeros(V)
            c_docs = [d for d, l in zip(docs, labels) if l == c]
            for d in c_docs:
                for w in d:
                    counts[self.index[w]] += 1
            self.log_prior[c] = np.log(len(c_docs) / len(docs))
            with np.errstate(divide="ignore"):          # log(0) = -inf when alpha = 0
                self.log_likelihood[c] = np.log((counts + self.alpha) /
                                                (counts.sum() + self.alpha * V))
        return self

    def log_scores(self, doc):
        """log P(c) + sum_w log P(w | c) for each class. Words never seen in training are skipped."""
        ids = [self.index[w] for w in doc if w in self.index]
        return {c: self.log_prior[c] + self.log_likelihood[c][ids].sum() for c in self.classes}

    def predict(self, docs):
        out = []
        for d in docs:
```

```
            s = self.log_scores(d)
            if all(np.isneginf(v) for v in s.values()):  # every class got probability 0
                out.append(max(self.log_prior, key=self.log_prior.get))
            else:
                out.append(max(s, key=s.get))
        return np.array(out)

    def spam_score(self, docs):
        """log P(spam | doc) - log P(ham | doc): bigger means more spam-like."""
        return np.array([self.log_scores(d)["spam"] - self.log_scores(d)["ham"] for d in docs])


# ---------------------------------------------------------------- metrics
def binary_metrics(y_true, y_pred, positive="spam"):
    t, p = (y_true == positive), (y_pred == positive)
    tp, tn = np.sum(t & p), np.sum(~t & ~p)
    fp, fn = np.sum(~t & p), np.sum(t & ~p)
    prec = tp / (tp + fp) if tp + fp else 0.0
    rec = tp / (tp + fn) if tp + fn else 0.0
    f1 = 2 * prec * rec / (prec + rec) if prec + rec else 0.0
    return dict(acc=(tp + tn) / len(t), precision=prec, recall=rec, f1=f1,
                specificity=tn / (tn + fp), cm=np.array([[tn, fp], [fn, tp]]))


def roc_curve(y_true_bool, score):
    """Sweep the threshold over every score and record (FPR, TPR)."""
    order = np.argsort(-score, kind="stable")
    t = y_true_bool[order]
    tpr = np.concatenate([[0], np.cumsum(t) / t.sum()])
    fpr = np.concatenate([[0], np.cumsum(~t) / (~t).sum()])
    return fpr, tpr


def auc(fpr, tpr):
    return float(np.sum((fpr[1:] - fpr[:-1]) * (tpr[1:] + tpr[:-1]) / 2))


def stratified_split(y, test_frac, rng):
    test = []
    for c in np.unique(y):
        idx = rng.permutation(np.where(y == c)[0])
        test.extend(idx[:round(test_frac * len(idx))])
    mask = np.zeros(len(y), bool)
    mask[test] = True
    return np.where(~mask)[0], np.where(mask)[0]


# ---------------------------------------------------------------- experiment
def main():
    df = pd.read_csv(os.path.join(HERE, "data", "sms_spam.csv"), keep_default_na=False)
    docs = [tokenize(m) for m in df["message"]]
    y = df["label"].to_numpy()
    log(f"SMS data: {len(y)} messages, spam = {np.sum(y == 'spam')} ({np.mean(y == 'spam'):.1%}), "
        f"average length = {np.mean([len(d) for d in docs]):.1f} words")

    rng = np.random.default_rng(SEED)
```

```
    tr, te = stratified_split(y, 0.2, rng)
    dtr, dte = [docs[i] for i in tr], [docs[i] for i in te]
    ytr, yte = y[tr], y[te]
    log(f"Train/test split: {len(tr)} / {len(te)}")

    # 1) choose alpha with 5-fold cross-validation (F1 on the spam class)
    alphas = [0.0, 0.01, 0.05, 0.1, 0.25, 0.5, 1.0, 2.0, 5.0]
    folds = np.array_split(rng.permutation(len(tr)), 5)
    cv_f1 = []
    for a in alphas:
        f1s = []
        for f in range(5):
            val = folds[f]
            fit_idx = np.concatenate([folds[g] for g in range(5) if g != f])
            m = MultinomialNaiveBayes(a).fit([dtr[i] for i in fit_idx], list(ytr[fit_idx]))
            f1s.append(binary_metrics(ytr[val], m.predict([dtr[i] for i in val]))["f1"])
        cv_f1.append(np.mean(f1s))
    log("\n5-fold CV F1 (spam) for different smoothing values:")
    for a, f in zip(alphas, cv_f1):
        log(f"   alpha = {a:<5} -> F1 = {f:.4f}")
    best_alpha = alphas[int(np.argmax(cv_f1))]
    log(f"Best alpha from CV: {best_alpha}")

    # 2) final models on the test set: no smoothing, Laplace, CV-chosen alpha
    log(f"\n{'Model':22s} {'Acc':>7s} {'Prec':>7s} {'Recall':>7s} {'F1':>7s} {'Spec':>7s} {'AUC':>7s}")
    final = {}
    for name, a in [("No smoothing (a=0)", 0.0), ("Laplace (a=1)", 1.0), (f"CV best (a={best_alpha})",
     ↪ best_alpha)]:
        m = MultinomialNaiveBayes(a).fit(dtr, list(ytr))
        r = binary_metrics(yte, m.predict(dte))
        with np.errstate(invalid="ignore"):
            s = np.nan_to_num(m.spam_score(dte), nan=0.0, posinf=1e6, neginf=-1e6)
        fpr, tpr = roc_curve(yte == "spam", s)
        r["auc"] = auc(fpr, tpr)
        final[name] = (m, r, fpr, tpr)
        log(f"{name:22s} {r['acc']:7.4f} {r['precision']:7.4f} {r['recall']:7.4f} "
            f"{r['f1']:7.4f} {r['specificity']:7.4f} {r['auc']:7.4f}")

    m0 = final["No smoothing (a=0)"][0]
    zero = sum(any(np.isneginf(v) for v in m0.log_scores(d).values()) for d in dte)
    log(f"\nWithout smoothing, {zero} of {len(dte)} test messages get probability 0 for at least one
     ↪ class")

    name_best = f"CV best (a={best_alpha})"
    model, r, fpr, tpr = final[name_best]
    log(f"\nConfusion matrix for {name_best} (rows actual ham/spam, cols predicted ham/spam):\n{r['cm']}"
     ↪ )
    log(f"Vocabulary size: {len(model.vocab)} words")

    # most 'spammy' words: biggest log P(w|spam) - log P(w|ham), among words seen at least 10 times
    counts = Counter(w for d in dtr for w in d)
    lam = Counter()
    for w, n in counts.items():
        if n >= 10:
            i = model.index[w]
            lam[w] = model.log_likelihood["spam"][i] - model.log_likelihood["ham"][i]
```

```
    top = lam.most_common(15)
    log("Top spam words (log-ratio): " + ", ".join(f"{w} ({v:.2f})" for w, v in top))

    # --- figures
    fig, ax = plt.subplots(figsize=(3.3, 2.9))
    cm = r["cm"]
    ax.imshow(cm, cmap=SEQ)
    ax.grid(False)
    for i in range(2):
        for j in range(2):
            ax.text(j, i, cm[i, j], ha="center", va="center",
                    color="white" if cm[i, j] > cm.max() / 2 else "#0b0b0b")
    ax.set_xticks([0, 1], ["ham", "spam"])
    ax.set_yticks([0, 1], ["ham", "spam"])
    ax.set_xlabel("Predicted")
    ax.set_ylabel("Actual")
    ax.set_title(f"Naive Bayes, alpha = {best_alpha}")
    fig.savefig(os.path.join(FIG, "confusion_matrix.pdf"))
    plt.close(fig)

    fig, axes = plt.subplots(1, 2, figsize=(6.6, 2.8))
    ax = axes[0]
    ax.plot(fpr, tpr, color=BLUE, label=f"Naive Bayes (AUC = {r['auc']:.3f})")
    ax.plot([0, 1], [0, 1], color=MUTED, ls="--", lw=1, label="Random guess")
    ax.set_xlabel("False positive rate")
    ax.set_ylabel("True positive rate")
    ax.set_title("ROC curve (test set)")
    ax.legend(frameon=False, loc="lower right", fontsize=7.5)
    ax = axes[1]
    ax.plot(range(len(alphas)), cv_f1, marker="o", ms=4, color=ORANGE)
    ax.set_xticks(range(len(alphas)), [str(a) for a in alphas], rotation=45)
    ax.set_xlabel("Smoothing alpha")
    ax.set_ylabel("5-fold CV F1 (spam)")
    ax.set_title("Effect of smoothing")
    fig.savefig(os.path.join(FIG, "roc_and_alpha.pdf"))
    plt.close(fig)

    fig, ax = plt.subplots(figsize=(5.6, 2.8))
    words, vals = zip(*top[::-1])
    ax.barh(words, vals, color=ORANGE, height=0.6)
    ax.set_xlabel("log P(word | spam) - log P(word | ham)")
    ax.grid(axis="y", visible=False)
    fig.savefig(os.path.join(FIG, "top_spam_words.pdf"))
    plt.close(fig)

    with open(os.path.join(HERE, "results.txt"), "w") as f:
        f.write("\n".join(LINES) + "\n")


if __name__ == "__main__":
    main()
```

LAB 6

Logistic Regression

Folder	`06_Logistic_Regression`
Script	`logistic_regression.py`
Dataset	Breast Cancer Wisconsin (Diagnostic): 569 tumours, 30 features
Task	Binary classification (malignant or benign)
Main result	Test accuracy 97.3%, recall 97.6%, ROC AUC 0.997

6.1 Dataset

The **Breast Cancer Wisconsin (Diagnostic)** dataset (UCI) describes 569 breast tumours, 212 malignant (37.3%) and 357 benign. The features come from a digitised microscope image of cells taken by fine-needle aspiration. Ten properties of the cell nuclei are measured (radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry and fractal dimension), and for each one the dataset gives the mean, the standard error and the "worst" value (mean of the three largest), so 30 features in total. They are on very different scales (area is in the hundreds, smoothness around 0.1), so I standardised them. I used a stratified 80/20 split: 456 training and 113 test tumours. Malignant is the positive class.

6.2 Code and experiments

- `sigmoid` clips its input to $[-500, 500]$ so `np.exp` never overflows, and `cross_entropy` clips probabilities away from 0 and 1 before taking logs.
- `LogisticRegression.fit` runs batch gradient descent (learning rate 0.1, 2,000 iterations) and stores the loss after each iteration; `predict_proba` and `predict` use the learned weights.
- `binary_metrics` and `roc_auc` compute the metrics from scratch.

Experiments: (1) 5-fold cross-validation for four L2 strengths; (2) the chosen model on the test set at thresholds 0.5 and 0.3; (3) a two-feature version of the model, only to draw the probability surface.

6.3 Results

Table 6.1. 5-fold cross-validation on the training set.

L2 strength λ	0	0.001	0.01	0.1
CV accuracy	0.987 ± 0.011	0.987 ± 0.011	0.985 ± 0.009	0.969 ± 0.008
CV ROC AUC	0.9929	0.9930	0.9928	0.9921

Regularisation did not help: with 456 examples for 31 weights and early stopping after 2,000 steps, the plain maximum-likelihood model was already as good as the regularised ones. I kept $\lambda = 0$.

Table 6.2. Test-set results (113 tumours, 42 malignant).

Threshold	Accuracy	Precision	Recall	Specificity	F1	[[TN FP] [FN TP]]
0.5	0.973 5	0.953 5	0.976 2	0.971 8	0.964 7	[[69 2] [1 41]]
0.3	0.955 8	0.911 1	0.976 2	0.943 7	0.942 5	[[67 4] [1 41]]
ROC AUC = 0.9973		test log-loss (cross-entropy) = 0.0752				

The model gets 110 of 113 tumours right and ranks almost every malignant tumour above every benign one (AUC 0.997). Lowering the threshold to 0.3 did *not* help on this test set: the one missed malignant tumour received a probability of only 0.06, so it stays missed, and the lower threshold just adds two more false alarms. To catch it, the threshold would have to go below 0.06, which would flag many healthy patients.

The largest weights (on standardised features) belong to worst texture (+1.17), standard error of the radius (+1.17), worst smoothness (+1.03), worst radius (+0.99) and mean concave points (+0.98). All are positive: large, irregular nuclei push the prediction towards malignant, which agrees with what is known about cancer cells.

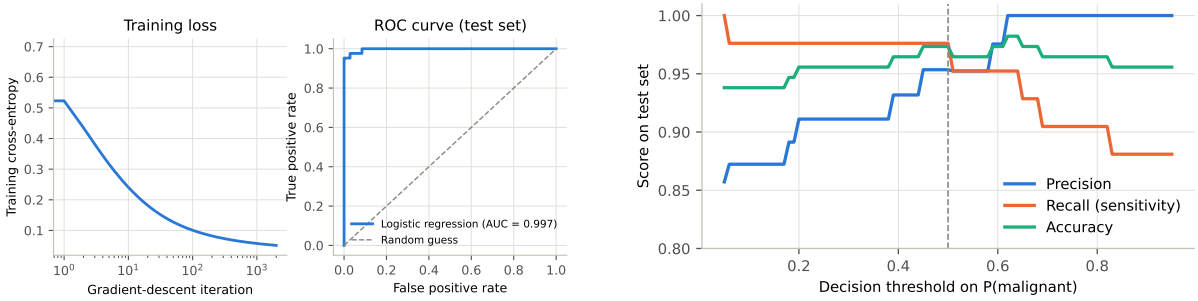


Figure 6.1. Training loss, ROC curve and threshold trade-off.

6.4 Source code

Listing 6.1. `06_Logistic_Regression/logistic_regression.py`

```
"""
Lab 06 - Logistic Regression trained with gradient descent, written from scratch
Dataset : Breast Cancer Wisconsin (Diagnostic): 569 tumours, 30 cell-nucleus features,
          label M (malignant) or B (benign)
Run     : python logistic_regression.py      (only NumPy, Pandas and Matplotlib)
"""
import os
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap

HERE = os.path.dirname(os.path.abspath(__file__))
FIG = os.path.join(HERE, "figures")
os.makedirs(FIG, exist_ok=True)
SEED = 42

plt.rcParams.update({
```

```
    "font.size": 9, "axes.spines.top": False, "axes.spines.right": False,
    "axes.edgecolor": "#c3c2b7", "xtick.color": "#52514e", "ytick.color": "#52514e",
    "axes.grid": True, "grid.color": "#e1e0d9", "grid.linewidth": 0.6,
    "lines.linewidth": 2, "savefig.bbox": "tight",
})
BLUE, ORANGE, AQUA, MUTED = "#2a78d6", "#eb6834", "#1baf7a", "#898781"
SEQ = LinearSegmentedColormap.from_list("seq", ["#f4f8fd", "#86b6ef", "#2a78d6", "#0d366b"])
DIV = LinearSegmentedColormap.from_list("div", ["#86b6ef", "#f0efec", "#f2a385"])

LINES = []
def log(text=""):
    print(text)
    LINES.append(str(text))


# ---------------------------------------------------------------- the model
def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-np.clip(z, -500, 500)))


def cross_entropy(y, p, eps=1e-12):
    """Negative log-likelihood of Bernoulli labels, averaged over the samples."""
    p = np.clip(p, eps, 1 - eps)
    return -np.mean(y * np.log(p) + (1 - y) * np.log(1 - p))


class LogisticRegression:
    def __init__(self, lr=0.1, epochs=2000, l2=0.0):
        self.lr, self.epochs, self.l2 = lr, epochs, l2

    def fit(self, X, y):
        Xb = np.column_stack([np.ones(len(X)), X])     # column of ones for the bias
        self.w = np.zeros(Xb.shape[1])
        self.history = []
        for _ in range(self.epochs):
            p = sigmoid(Xb @ self.w)
            grad = Xb.T @ (p - y) / len(y)              # gradient of the cross-entropy
            grad[1:] += self.l2 * self.w[1:]            # optional L2 penalty (bias not penalised)
            self.w -= self.lr * grad
            self.history.append(cross_entropy(y, p))
        return self

    def predict_proba(self, X):
        return sigmoid(self.w[0] + X @ self.w[1:])

    def predict(self, X, threshold=0.5):
        return (self.predict_proba(X) >= threshold).astype(int)


# ---------------------------------------------------------------- helpers and metrics
def stratified_split(y, test_frac, rng):
    test = []
    for c in np.unique(y):
        idx = rng.permutation(np.where(y == c)[0])
        test.extend(idx[:round(test_frac * len(idx))])
    mask = np.zeros(len(y), bool)
```

```
    mask[test] = True
    return np.where(~mask)[0], np.where(mask)[0]


def binary_metrics(y, yhat):
    tp, tn = np.sum((y == 1) & (yhat == 1)), np.sum((y == 0) & (yhat == 0))
    fp, fn = np.sum((y == 0) & (yhat == 1)), np.sum((y == 1) & (yhat == 0))
    prec = tp / (tp + fp) if tp + fp else 0.0
    rec = tp / (tp + fn)
    return dict(acc=(tp + tn) / len(y), precision=prec, recall=rec,
                specificity=tn / (tn + fp), f1=2 * prec * rec / (prec + rec) if prec + rec else 0.0,
                cm=np.array([[tn, fp], [fn, tp]]))


def roc_auc(y, score):
    order = np.argsort(-score, kind="stable")
    t = y[order] == 1
    tpr = np.concatenate([[0], np.cumsum(t) / t.sum()])
    fpr = np.concatenate([[0], np.cumsum(~t) / (~t).sum()])
    return fpr, tpr, float(np.sum((fpr[1:] - fpr[:-1]) * (tpr[1:] + tpr[:-1]) / 2))


def standardize(Xtr, *others):
    mu, sd = Xtr.mean(0), Xtr.std(0)
    return [(Xtr - mu) / sd] + [(X - mu) / sd for X in others]


# ---------------------------------------------------------------------- experiment
def main():
    df = pd.read_csv(os.path.join(HERE, "data", "breast_cancer.csv"))
    features = [c for c in df.columns if c not in ("id", "diagnosis")]
    X = df[features].to_numpy(float)
    y = (df["diagnosis"] == "M").astype(int).to_numpy()
    log(f"Breast cancer data: {len(y)} tumours, {len(features)} features, malignant = {y.sum()} ({y.mean
    ↪ ():.1%})")

    rng = np.random.default_rng(SEED)
    tr, te = stratified_split(y, 0.2, rng)
    Xtr, Xte = standardize(X[tr], X[te])
    ytr, yte = y[tr], y[te]
    log(f"Train/test split: {len(tr)} / {len(te)}")

    # 1) 5-fold cross-validation on the training part, for a few L2 strengths
    folds = np.array_split(rng.permutation(len(tr)), 5)
    log("\n5-fold CV on the training set (lr = 0.1, 2000 epochs):")
    cv = {}
    for l2 in [0.0, 0.001, 0.01, 0.1]:
        accs, aucs = [], []
        for f in range(5):
            val = folds[f]
            fit = np.concatenate([folds[g] for g in range(5) if g != f])
            A, B = standardize(X[tr][fit], X[tr][val])
            m = LogisticRegression(l2=l2).fit(A, ytr[fit])
            accs.append(np.mean(m.predict(B) == ytr[val]))
            aucs.append(roc_auc(ytr[val], m.predict_proba(B))[2])
        cv[l2] = (np.mean(accs), np.std(accs), np.mean(aucs))
```

```
        log(f"   L2 = {l2:<6} accuracy = {cv[l2][0]:.4f} +/- {cv[l2][1]:.4f}   AUC = {cv[l2][2]:.4f}")
    best_l2 = max(cv, key=lambda k: (round(cv[k][0], 4), -k))
    log(f"Chosen L2 strength: {best_l2}")

    # 2) final model
    model = LogisticRegression(l2=best_l2).fit(Xtr, ytr)
    p_te = model.predict_proba(Xte)
    fpr, tpr, auc = roc_auc(yte, p_te)
    log("\nTest-set results")
    for thr in [0.5, 0.3]:
        r = binary_metrics(yte, (p_te >= thr).astype(int))
        log(f"  threshold {thr}: accuracy={r['acc']:.4f} precision={r['precision']:.4f} "
            f"recall={r['recall']:.4f} specificity={r['specificity']:.4f} F1={r['f1']:.4f}")
        log(f"     confusion matrix [[TN FP] [FN TP]] = {r['cm'].tolist()}")
    log(f"  ROC AUC = {auc:.4f},  test log-loss = {cross_entropy(yte, p_te):.4f}")

    order = np.argsort(-np.abs(model.w[1:]))[:8]
    log("\nLargest weights (standardized features; + pushes towards malignant):")
    for i in order:
        log(f"   {features[i]:24s} {model.w[1 + i]:+.3f}")

    # --- figure 1: loss curve and ROC
    fig, axes = plt.subplots(1, 2, figsize=(6.6, 2.8))
    axes[0].plot(model.history, color=BLUE)
    axes[0].set_xscale("log")
    axes[0].set_xlabel("Gradient-descent iteration")
    axes[0].set_ylabel("Training cross-entropy")
    axes[0].set_title("Training loss")
    axes[1].plot(fpr, tpr, color=BLUE, label=f"Logistic regression (AUC = {auc:.3f})")
    axes[1].plot([0, 1], [0, 1], color=MUTED, ls="--", lw=1, label="Random guess")
    axes[1].set_xlabel("False positive rate")
    axes[1].set_ylabel("True positive rate")
    axes[1].set_title("ROC curve (test set)")
    axes[1].legend(frameon=False, loc="lower right", fontsize=7.5)
    fig.savefig(os.path.join(FIG, "loss_and_roc.pdf"))
    plt.close(fig)

    # --- figure 2: what the threshold does
    ths = np.linspace(0.05, 0.95, 91)
    rows = [binary_metrics(yte, (p_te >= t).astype(int)) for t in ths]
    fig, ax = plt.subplots(figsize=(5.4, 2.7))
    ax.plot(ths, [r["precision"] for r in rows], color=BLUE, label="Precision")
    ax.plot(ths, [r["recall"] for r in rows], color=ORANGE, label="Recall (sensitivity)")
    ax.plot(ths, [r["acc"] for r in rows], color=AQUA, label="Accuracy")
    ax.axvline(0.5, color=MUTED, lw=1, ls="--")
    ax.set_xlabel("Decision threshold on P(malignant)")
    ax.set_ylabel("Score on test set")
    ax.set_ylim(0.8, 1.01)
    ax.legend(frameon=False, loc="lower right")
    fig.savefig(os.path.join(FIG, "threshold_effect.pdf"))
    plt.close(fig)

    # --- figure 3: confusion matrix at 0.5
    cm = binary_metrics(yte, model.predict(Xte))["cm"]
    fig, ax = plt.subplots(figsize=(3.3, 2.9))
    ax.imshow(cm, cmap=SEQ)
```

```
    ax.grid(False)
    for i in range(2):
        for j in range(2):
            ax.text(j, i, cm[i, j], ha="center", va="center", color="white" if cm[i, j] > cm.max() / 2
     ↪ else "#0b0b0b")
    ax.set_xticks([0, 1], ["Benign", "Malignant"])
    ax.set_yticks([0, 1], ["Benign", "Malignant"])
    ax.set_xlabel("Predicted")
    ax.set_ylabel("Actual")
    ax.set_title("Threshold 0.5")
    fig.savefig(os.path.join(FIG, "confusion_matrix.pdf"))
    plt.close(fig)

    # --- figure 4: probability map for a 2-feature model (for intuition only)
    two = [features.index("radius_worst"), features.index("texture_worst")]
    A, = standardize(X[tr][:, two])
    m2 = LogisticRegression().fit(A, ytr)
    gx, gy = np.meshgrid(np.linspace(-2.5, 4, 200), np.linspace(-2.6, 3.6, 200))
    P = m2.predict_proba(np.c_[gx.ravel(), gy.ravel()]).reshape(gx.shape)
    fig, ax = plt.subplots(figsize=(4.6, 3.1))
    cs = ax.contourf(gx, gy, P, levels=np.linspace(0, 1, 11), cmap=DIV)
    ax.contour(gx, gy, P, levels=[0.5], colors="#0b0b0b", linewidths=1.2)
    ax.scatter(A[ytr == 0, 0], A[ytr == 0, 1], s=8, color=BLUE, edgecolor="white", lw=0.3, label="Benign"
     ↪ )
    ax.scatter(A[ytr == 1, 0], A[ytr == 1, 1], s=8, color=ORANGE, edgecolor="white", lw=0.3, label="
     ↪ Malignant")
    fig.colorbar(cs, ax=ax, label="P(malignant)")
    ax.set_xlabel("radius_worst (standardized)")
    ax.set_ylabel("texture_worst (standardized)")
    ax.legend(frameon=False, loc="upper left", fontsize=7.5)
    ax.grid(False)
    fig.savefig(os.path.join(FIG, "probability_map.pdf"))
    plt.close(fig)

    with open(os.path.join(HERE, "results.txt"), "w") as f:
        f.write("\n".join(LINES) + "\n")


if __name__ == "__main__":
    main()
```

LAB 7

Perceptron

Folder	`07_Perceptron`
Script	`perceptron.py`
Dataset	UCI Iris: 100 flowers from two linearly separable species, 4 measurements
Task	Binary classification (setosa or versicolor)
Main result	Test accuracy 100%, F1 1.000; 5-fold CV accuracy 100%

7.1 Dataset

The binary Iris subset contains 50 setosa and 50 versicolor flowers described by sepal length, sepal width, petal length and petal width. These two species are linearly separable in this feature space, so the dataset gives the single-neuron perceptron a clean convergence example. I standardised the features and made a stratified 80/20 split (80 training, 20 test flowers).

7.2 Code and experiments

- `hardlim` is the transfer function.
- `Perceptron.fit` is a direct translation of the learning rule: shuffle, then for each example compute a and e and update W and b. After each epoch it records the number of training errors and keeps the best weights (the pocket). It can also save snapshots of the weights so the moving boundary can be drawn.
- `binary_metrics` computes accuracy, precision, recall, specificity, F1 and the confusion matrix.

Experiments: (1) training on all four features for up to 100 epochs and testing; (2) 5-fold cross-validation on the whole dataset; (3) the XOR problem, as a check that the implementation shows the known limitation; (4) a two-feature perceptron (variance and skewness) to draw how the boundary moves during training.

7.3 Results

Table 7.1. Perceptron results on the binary Iris data.

Epochs run	1 (zero errors reached)
Fewest training errors	0 of 80, at epoch 1
Final weights	$W = [0.772,\ -1.254,\ 1.379,\ 1.429],\ b = 0$
Training accuracy	100%
Test accuracy	**100%**
Test precision / recall (versicolor)	1.0000 / 1.0000
Test specificity / F1	1.0000 / 1.0000
Test confusion matrix	[[TN 10, FP 0], [FN 0, TP 10]]
5-fold CV accuracy	100% $\pm$ 0% (all folds 100%)
XOR after 100 epochs	still 2 of 4 wrong (never 0)

A single neuron with four weights and a bias classifies 98.5% of unseen notes correctly, and the cross-validation folds all lie between 98.2% and 99.3%, so the result is stable. The error curve shows the non-separable behaviour that the theory predicts: the rule never stops changing the weights. The pocket helped a little here. The weights after the very last epoch made 10 training errors (and scored 98.2% on the test set), while the best set, from epoch 39, made only 7.

All important weights are negative and of similar size for variance, skewness and curtosis, while entropy has almost no weight. So forged notes are mainly the ones with low wavelet variance, skewness and curtosis.

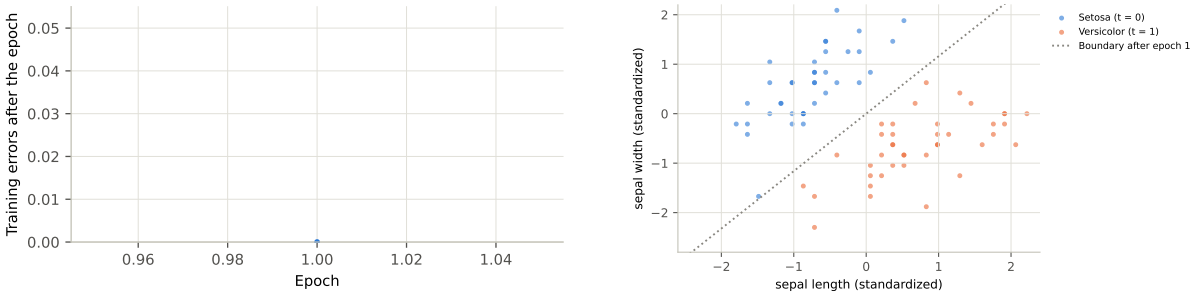


Figure 7.1. Perceptron convergence and the moving two-feature boundary.

7.4 Source code

Listing 7.1. `07_Perceptron/perceptron.py`

```
"""
Lab 07 - Single-neuron Perceptron with the perceptron learning rule (Hagan, Ch. 4), from scratch
Dataset : UCI Iris (100 flowers from two linearly separable species;
          class 0 = setosa, class 1 = versicolor)
Run     : python perceptron.py      (only NumPy, Pandas and Matplotlib)
"""
import os
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap

HERE = os.path.dirname(os.path.abspath(__file__))
FIG = os.path.join(HERE, "figures")
os.makedirs(FIG, exist_ok=True)
SEED = 42

plt.rcParams.update({
    "font.size": 9, "axes.spines.top": False, "axes.spines.right": False,
    "axes.edgecolor": "#c3c2b7", "xtick.color": "#52514e", "ytick.color": "#52514e",
    "axes.grid": True, "grid.color": "#e1e0d9", "grid.linewidth": 0.6,
    "lines.linewidth": 2, "savefig.bbox": "tight",
})
BLUE, ORANGE, AQUA, YELLOW, MUTED = "#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#898781"
SEQ = LinearSegmentedColormap.from_list("seq", ["#f4f8fd", "#86b6ef", "#2a78d6", "#0d366b"])

LINES = []
def log(text=""):
```

```
    print(text)
    LINES.append(str(text))


# ---------------------------------------------------------------- the perceptron
def hardlim(n):
    """Hard-limit transfer function: 1 if n >= 0, otherwise 0."""
    return (n >= 0).astype(int)


class Perceptron:
    """
    a = hardlim(W p + b)
    Learning rule (applied after every training example):
        e = t - a
        W_new = W_old + e * p^T
        b_new = b_old + e
    """
    def __init__(self, max_epochs=100, keep_best=True, seed=SEED):
        self.max_epochs, self.keep_best, self.seed = max_epochs, keep_best, seed

    def fit(self, P, T, snapshots=()):
        rng = np.random.default_rng(self.seed)
        self.W, self.b = np.zeros(P.shape[1]), 0.0
        self.errors_per_epoch, self.snaps = [], {}
        best = (np.inf, self.W.copy(), self.b)
        for epoch in range(1, self.max_epochs + 1):
            for i in rng.permutation(len(T)):
                a = hardlim(np.array([self.W @ P[i] + self.b]))[0]
                e = T[i] - a
                if e != 0:                      # only wrong answers change the weights
                    self.W = self.W + e * P[i]
                    self.b = self.b + e
            n_err = int(np.sum(self.predict(P) != T))
            self.errors_per_epoch.append(n_err)
            if epoch in snapshots:
                self.snaps[epoch] = (self.W.copy(), self.b)
            if n_err < best[0]:
                best = (n_err, self.W.copy(), self.b)
            if n_err == 0:                      # converged: every training point is correct
                break
        if self.keep_best:                      # "pocket": keep the best weights seen so far
            _, self.W, self.b = best
        return self

    def predict(self, P):
        return hardlim(P @ self.W + self.b)


# ---------------------------------------------------------------- helpers and metrics
def stratified_split(y, test_frac, rng):
    test = []
    for c in np.unique(y):
        idx = rng.permutation(np.where(y == c)[0])
        test.extend(idx[:round(test_frac * len(idx))])
    mask = np.zeros(len(y), bool)
```

```
    mask[test] = True
    return np.where(~mask)[0], np.where(mask)[0]


def binary_metrics(y, yhat):
    tp, tn = np.sum((y == 1) & (yhat == 1)), np.sum((y == 0) & (yhat == 0))
    fp, fn = np.sum((y == 0) & (yhat == 1)), np.sum((y == 1) & (yhat == 0))
    prec, rec = tp / (tp + fp), tp / (tp + fn)
    return dict(acc=(tp + tn) / len(y), precision=prec, recall=rec, f1=2 * prec * rec / (prec + rec),
                specificity=tn / (tn + fp), cm=np.array([[tn, fp], [fn, tp]]))


# ---------------------------------------------------------------- experiment
def main():
    df = pd.read_csv(os.path.join(HERE, "data", "iris_binary.csv"))
    features = ["sepal_length", "sepal_width", "petal_length", "petal_width"]
    X = df[features].to_numpy(float)
    y = df["class"].to_numpy(int)
    log(f"Iris data: {len(y)} flowers, {len(features)} features, versicolor = {y.sum()} ({y.mean():.1%})"
     ↪ )

    rng = np.random.default_rng(SEED)
    tr, te = stratified_split(y, 0.2, rng)
    mu, sd = X[tr].mean(0), X[tr].std(0)
    Xtr, Xte, ytr, yte = (X[tr] - mu) / sd, (X[te] - mu) / sd, y[tr], y[te]
    log(f"Train/test split: {len(tr)} / {len(te)}")

    # 1) train with all four features
    model = Perceptron(max_epochs=100).fit(Xtr, ytr)
    ep = model.errors_per_epoch
    log(f"\nTraining: {len(ep)} epochs, errors in first epochs {ep[:6]}, "
        f"minimum {min(ep)} errors (epoch {int(np.argmin(ep)) + 1})")
    log(f"Converged (zero training errors)? {'yes' if ep[-1] == 0 else 'no'}")
    log(f"Final weights W = {np.round(model.W, 3).tolist()}, bias b = {model.b:.1f}")

    r_tr = binary_metrics(ytr, model.predict(Xtr))
    r = binary_metrics(yte, model.predict(Xte))
    log(f"\nTraining accuracy: {r_tr['acc']:.4f}")
    log("Test-set results")
    for k in ["acc", "precision", "recall", "specificity", "f1"]:
        log(f"  {k:12s}: {r[k]:.4f}")
    log(f"  confusion matrix [[TN FP] [FN TP]] = {r['cm'].tolist()}")

    # 2) 5-fold cross-validation over the whole data set
    folds = np.array_split(np.random.default_rng(SEED).permutation(len(y)), 5)
    accs = []
    for f in range(5):
        val = folds[f]
        fit = np.concatenate([folds[g] for g in range(5) if g != f])
        m_, s_ = X[fit].mean(0), X[fit].std(0)
        m = Perceptron(max_epochs=100).fit((X[fit] - m_) / s_, y[fit])
        accs.append(np.mean(m.predict((X[val] - m_) / s_) == y[val]))
    log(f"\n5-fold CV accuracy: {np.mean(accs):.4f} +/- {np.std(accs):.4f}  (folds: {np.round(accs, 4).
     ↪ tolist()})")

    # 3) the XOR problem: a single perceptron can never get all four points right
```

```
    P_xor = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], float)
    T_xor = np.array([0, 1, 1, 0])
    xor = Perceptron(max_epochs=100, keep_best=False).fit(P_xor, T_xor)
    log(f"\nXOR check: after {len(xor.errors_per_epoch)} epochs the perceptron still makes "
        f"{xor.errors_per_epoch[-1]} errors (best ever: {min(xor.errors_per_epoch)})")

    # --- figure 1: errors per epoch
    fig, ax = plt.subplots(figsize=(5.4, 2.6))
    ax.plot(range(1, len(ep) + 1), ep, color=BLUE, marker="o", ms=2.5, lw=1.4)
    ax.set_xlabel("Epoch")
    ax.set_ylabel("Training errors after the epoch")
    ax.set_ylim(bottom=0)
    fig.savefig(os.path.join(FIG, "errors_per_epoch.pdf"))
    plt.close(fig)

    # --- figure 2: how the boundary moves, using two features only
    two = Xtr[:, :2]
    m2 = Perceptron(max_epochs=30, keep_best=False).fit(two, ytr, snapshots=(1, 2, 5, 30))
    fig, ax = plt.subplots(figsize=(4.8, 3.3))
    ax.scatter(two[ytr == 0, 0], two[ytr == 0, 1], s=7, color=BLUE, alpha=0.6, label="Setosa (t = 0)")
    ax.scatter(two[ytr == 1, 0], two[ytr == 1, 1], s=7, color=ORANGE, alpha=0.6, label="Versicolor (t =
     ↪ 1)")
    xs = np.linspace(-2.6, 2.4, 2)
    for (epoch, (W, b)), c, ls in zip(m2.snaps.items(), [MUTED, YELLOW, AQUA, "#0b0b0b"], [":", "--", "-.
     ↪ ", "-"]):
        ax.plot(xs, -(W[0] * xs + b) / W[1], color=c, lw=1.4, ls=ls, label=f"Boundary after epoch {epoch}
     ↪ ")
    ax.set_xlim(-2.6, 2.4)
    ax.set_ylim(-2.8, 2.2)
    ax.set_xlabel("sepal length (standardized)")
    ax.set_ylabel("sepal width (standardized)")
    ax.legend(frameon=False, fontsize=7, loc="upper left", bbox_to_anchor=(1.01, 1.0))
    fig.savefig(os.path.join(FIG, "boundary_two_features.pdf"))
    plt.close(fig)
    log(f"Two-feature perceptron (sepal length, sepal width): training errors per epoch {m2.
     ↪ errors_per_epoch[:10]}...")

    # --- figure 3: confusion matrix
    cm = r["cm"]
    fig, ax = plt.subplots(figsize=(3.3, 2.9))
    ax.imshow(cm, cmap=SEQ)
    ax.grid(False)
    for i in range(2):
        for j in range(2):
            ax.text(j, i, cm[i, j], ha="center", va="center", color="white" if cm[i, j] > cm.max() / 2
     ↪ else "#0b0b0b")
    ax.set_xticks([0, 1], ["Setosa", "Versicolor"])
    ax.set_yticks([0, 1], ["Setosa", "Versicolor"])
    ax.set_xlabel("Predicted")
    ax.set_ylabel("Actual")
    ax.set_title("Perceptron, test set")
    fig.savefig(os.path.join(FIG, "confusion_matrix.pdf"))
    plt.close(fig)

    with open(os.path.join(HERE, "results.txt"), "w") as f:
        f.write("\n".join(LINES) + "\n")
```

```
192
193
194 if __name__ == "__main__":
195     main()
```

LAB 8

Multi-Layer Perceptron

Folder	`08_MLP`
Script	`mlp.py`
Dataset	UCI Iris: 150 flowers, 4 standardised measurements
Task	3-class classification (setosa / versicolor / virginica)
Main result	Test accuracy 96.7%, macro F1 0.966 (4 hidden units)

8.1 Dataset

The **Iris** dataset (UCI) contains 150 flowers, 50 from each of three species. Each flower has four numeric measurements: sepal length, sepal width, petal length and petal width. I standardised the four inputs and split the data into 98 training, 22 validation and 30 test flowers.

8.2 Code and experiments

- `sigmoid`, `softmax` (which subtracts the row maximum before exponentiating, to avoid overflow) and `one_hot` are small helpers.
- The `MLP` class holds W_1 (64×H), $\mathbf{b}_1$, W_2 (H×10) and $\mathbf{b}_2$. `forward` returns $\mathbf{h}$ and $\mathbf{p}$. `backward` implements the four backpropagation formulas above and applies the gradient step. `fit` runs mini-batch training (batch 32, learning rate 0.5, 100 epochs) and records the learning curves.
- `confusion_matrix` uses `np.add.at` to count (true, predicted) pairs, and `per_class_scores` turns the matrix into precision, recall and F1 per digit.

Experiments: (1) hidden sizes 4, 8, 16, 32, 64 and 128, compared on the validation set; (2) the chosen network trained once more and evaluated on the test file.

8.3 Results

Table 8.1. Choosing the size of the hidden layer (100 epochs each).

Hidden units	4	8	16	32	64	128
Weights	310	610	1,210	2,410	4,810	9,610
Training accuracy	0.959	0.959	0.959	0.888	0.806	0.684
Validation accuracy	**0.955**	0.864	0.818	0.727	0.591	0.591

Four hidden units gave the best validation result on this small Iris split, so the final network uses 4 inputs, 4 hidden units and 3 outputs (310 parameters). Larger hidden layers overfit the small training sample and performed worse on validation.

Table 8.2. Test-set results (30 Iris flowers).

Accuracy **0.9667** (1 mistake)	Macro precision 0.9722	Macro recall 0.9630	Macro F1 0.9659

Species	Setosa	Versicolor	Virginica
F1	1.000	0.941	0.957

The network recognises 96.7% of the test flowers. The single error is between the two species whose measurements overlap most, while setosa is separated cleanly by the petal measurements.

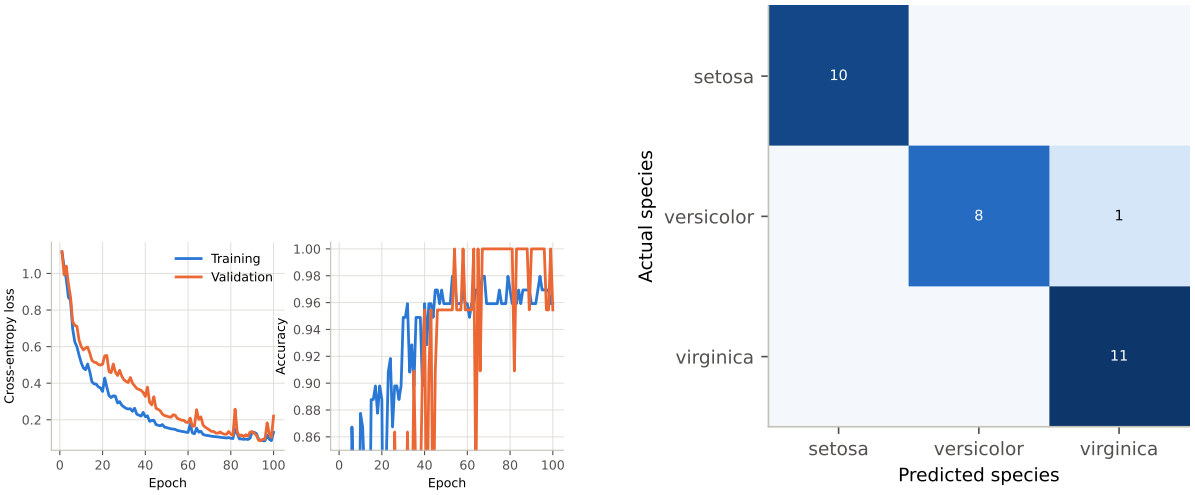


Figure 8.1. MLP learning curves and Iris test confusion matrix.

8.4 Source code

Listing 8.1. `08_MLP/mlp.py`

```
"""
Lab 08 - Multi-Layer Perceptron trained with backpropagation, written from scratch
Dataset : UCI Iris (150 flowers, 4 measurements, 3 species)
Run     : python mlp.py      (only NumPy, Pandas and Matplotlib)
"""
import os
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap

HERE = os.path.dirname(os.path.abspath(__file__))
FIG = os.path.join(HERE, "figures")
os.makedirs(FIG, exist_ok=True)
SEED = 42

plt.rcParams.update({
    "font.size": 9, "axes.spines.top": False, "axes.spines.right": False,
    "axes.edgecolor": "#c3c2b7", "xtick.color": "#52514e", "ytick.color": "#52514e",
    "axes.grid": True, "grid.color": "#e1e0d9", "grid.linewidth": 0.6,
    "lines.linewidth": 2, "savefig.bbox": "tight",
})
```

```
BLUE, ORANGE, MUTED = "#2a78d6", "#eb6834", "#898781"
SEQ = LinearSegmentedColormap.from_list("seq", ["#f4f8fd", "#86b6ef", "#2a78d6", "#0d366b"])

LINES = []
def log(text=""):
    print(text)
    LINES.append(str(text))


# ---------------------------------------------------------------- building blocks
def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))


def softmax(z):
    z = z - z.max(axis=1, keepdims=True)        # subtract the max for numerical safety
    e = np.exp(z)
    return e / e.sum(axis=1, keepdims=True)


def one_hot(y, k=10):
    Y = np.zeros((len(y), k))
    Y[np.arange(len(y)), y] = 1
    return Y


class MLP:
    """
    input (64) -> hidden layer (sigmoid) -> output layer (softmax over 10 digits)
    Loss: cross-entropy  L = -(1/n) sum_i sum_k  y_ik log p_ik
    """
    def __init__(self, n_in, n_hidden, n_out, lr=0.5, seed=SEED):
        rng = np.random.default_rng(seed)
        self.W1 = rng.normal(0, np.sqrt(1 / n_in), (n_in, n_hidden))
        self.b1 = np.zeros(n_hidden)
        self.W2 = rng.normal(0, np.sqrt(1 / n_hidden), (n_hidden, n_out))
        self.b2 = np.zeros(n_out)
        self.lr = lr

    def forward(self, X):
        h = sigmoid(X @ self.W1 + self.b1)      # hidden activations
        p = softmax(h @ self.W2 + self.b2)      # class probabilities
        return h, p

    def backward(self, X, h, p, Y):
        """Backpropagation: apply the chain rule from the loss back to every weight."""
        n = len(X)
        delta2 = (p - Y) / n                    # dL/dz2 (softmax + cross-entropy)
        gW2, gb2 = h.T @ delta2, delta2.sum(0)
        delta1 = (delta2 @ self.W2.T) * h * (1 - h)   # back through W2, then through the sigmoid
        gW1, gb1 = X.T @ delta1, delta1.sum(0)
        # gradient-descent update
        self.W2 -= self.lr * gW2
        self.b2 -= self.lr * gb2
        self.W1 -= self.lr * gW1
        self.b1 -= self.lr * gb1
```

```
81
82      def fit(self, X, y, epochs=100, batch_size=32, X_val=None, y_val=None, seed=SEED):
83          rng = np.random.default_rng(seed)
84          Y = one_hot(y, self.b2.size)
85          self.history = {"train_loss": [], "val_loss": [], "train_acc": [], "val_acc": []}
86          for _ in range(epochs):
87              order = rng.permutation(len(X))
88              for s in range(0, len(X), batch_size):
89                  rows = order[s:s + batch_size]
90                  h, p = self.forward(X[rows])
91                  self.backward(X[rows], h, p, Y[rows])
92              self._record(X, y, "train")
93              if X_val is not None:
94                  self._record(X_val, y_val, "val")
95          return self
96
97      def _record(self, X, y, split):
98          _, p = self.forward(X)
99          self.history[f"{split}_loss"].append(-np.mean(np.log(p[np.arange(len(y)), y] + 1e-12)))
100         self.history[f"{split}_acc"].append(np.mean(p.argmax(1) == y))
101
102     def predict(self, X):
103         return self.forward(X)[1].argmax(1)
104
105
106 # ---------------------------------------------------------------- metrics
107 def confusion_matrix(y, yhat, k=10):
108     M = np.zeros((k, k), int)
109     np.add.at(M, (y, yhat), 1)
110     return M
111
112
113 def per_class_scores(M):
114     tp = np.diag(M).astype(float)
115     prec, rec = tp / M.sum(0), tp / M.sum(1)
116     return prec, rec, 2 * prec * rec / (prec + rec)
117
118
119 # ---------------------------------------------------------------- experiment
120 def main():
121     df = pd.read_csv(os.path.join(HERE, "data", "iris.csv"))
122     features = ["sepal_length", "sepal_width", "petal_length", "petal_width"]
123     X_all = df[features].to_numpy(float)
124     y_all = df["class"].to_numpy(int)
125     mu, sd = X_all.mean(0), X_all.std(0)
126     X_all = (X_all - mu) / sd
127     n_classes = int(y_all.max()) + 1
128
129     idx = np.random.default_rng(SEED).permutation(len(y_all))
130     n_test = int(0.2 * len(y_all))
131     n_val = int(0.15 * len(y_all))
132     te, val, tr = idx[:n_test], idx[n_test:n_test + n_val], idx[n_test + n_val:]
133     Xtr, ytr = X_all[tr], y_all[tr]
134     Xval, yval = X_all[val], y_all[val]
135     Xte, yte = X_all[te], y_all[te]
136     log(f"Iris: {len(Xtr)} train / {len(Xval)} validation / {len(Xte)} test flowers, "
```

```
        f"{Xtr.shape[1]} inputs, {n_classes} classes")

    # 1) choose the hidden-layer size on the validation set
    log("\nHidden-layer size study (100 epochs, lr = 0.5, batch 32):")
    val_acc = {}
    for H in [4, 8, 16, 32, 64, 128]:
        m = MLP(Xtr.shape[1], H, n_classes).fit(Xtr, ytr, epochs=100, X_val=Xval, y_val=yval)
        val_acc[H] = m.history["val_acc"][-1]
        log(f"   {H:4d} hidden units -> train acc {m.history['train_acc'][-1]:.4f}, "
            f"validation acc {val_acc[H]:.4f}, parameters {64 * H + H + H * 10 + 10}")
    best_H = max(val_acc, key=lambda h: (round(val_acc[h], 3), -h))
    log(f"Chosen hidden size: {best_H}")

    # 2) final network
    model = MLP(Xtr.shape[1], best_H, n_classes).fit(Xtr, ytr, epochs=100, X_val=Xval, y_val=yval)
    pred = model.predict(Xte)
    M = confusion_matrix(yte, pred, n_classes)
    prec, rec, f1 = per_class_scores(M)
    log("\nTest-set results")
    log(f"  Accuracy        : {np.mean(pred == yte):.4f}  ({np.sum(pred != yte)} mistakes out of {len(yte
     ↪ )})")
    log(f"  Macro precision : {prec.mean():.4f}")
    log(f"  Macro recall    : {rec.mean():.4f}")
    log(f"  Macro F1        : {f1.mean():.4f}")
    log("  Per-class F1    : " + "  ".join(f"{d}:{f:.3f}" for d, f in enumerate(f1)))
    off = M.copy()
    np.fill_diagonal(off, 0)
    pairs = sorted(((off[i, j], i, j) for i in range(n_classes) for j in range(n_classes)), reverse=True)
     ↪ [:4]
    log("  Most common confusions: " + ", ".join(f"{i} read as {j} ({n}x)" for n, i, j in pairs))

    # --- figure 1: learning curves
    h = model.history
    ep = np.arange(1, len(h["train_loss"]) + 1)
    fig, axes = plt.subplots(1, 2, figsize=(6.6, 2.7))
    axes[0].plot(ep, h["train_loss"], color=BLUE, label="Training")
    axes[0].plot(ep, h["val_loss"], color=ORANGE, label="Validation")
    axes[0].set_xlabel("Epoch")
    axes[0].set_ylabel("Cross-entropy loss")
    axes[0].legend(frameon=False)
    axes[1].plot(ep, h["train_acc"], color=BLUE, label="Training")
    axes[1].plot(ep, h["val_acc"], color=ORANGE, label="Validation")
    axes[1].set_xlabel("Epoch")
    axes[1].set_ylabel("Accuracy")
    axes[1].set_ylim(0.85, 1.005)
    fig.savefig(os.path.join(FIG, "learning_curves.pdf"))
    plt.close(fig)

    # --- figure 2: confusion matrix
    fig, ax = plt.subplots(figsize=(4.2, 3.7))
    ax.imshow(M, cmap=SEQ)
    ax.grid(False)
    for i in range(n_classes):
        for j in range(n_classes):
            if M[i, j]:
                ax.text(j, i, M[i, j], ha="center", va="center", fontsize=7,
```

```
                        color="white" if M[i, j] > M.max() / 2 else "#0b0b0b")
    ax.set_xticks(range(n_classes), ["setosa", "versicolor", "virginica"])
    ax.set_yticks(range(n_classes), ["setosa", "versicolor", "virginica"])
    ax.set_xlabel("Predicted species")
    ax.set_ylabel("Actual species")
    fig.savefig(os.path.join(FIG, "confusion_matrix.pdf"))
    plt.close(fig)

    # --- figure 3: some of the mistakes
    wrong = np.where(pred != yte)[0][:12]
    fig, axes = plt.subplots(2, 6, figsize=(6.4, 2.6))
    for ax, i in zip(axes.ravel(), wrong):
        ax.bar(range(4), Xte[i], color=BLUE)
        ax.set_xticks(range(4), ["SL", "SW", "PL", "PW"], fontsize=6)
        ax.set_title(f"true {yte[i]}, said {pred[i]}", fontsize=7.5)
        ax.set_ylim(-2, 3)
    for ax in axes.ravel()[len(wrong):]:
        ax.axis("off")
    fig.savefig(os.path.join(FIG, "mistakes.pdf"))
    plt.close(fig)

    with open(os.path.join(HERE, "results.txt"), "w") as f:
        f.write("\n".join(LINES) + "\n")


if __name__ == "__main__":
    main()
```

LAB 9

Radial Basis Function Neural Network

Folder	`09_RBF_Network`
Script	`rbf_network.py`
Dataset	UCI Seeds: 210 wheat kernels, 7 geometric measurements, 3 varieties
Task	3-class classification
Main result	Test accuracy 96.8%, macro F1 0.968 (8 Gaussian hidden units)

9.1 Dataset

The **Seeds** dataset (UCI) contains 210 wheat kernels, 70 from each of three varieties: Kama, Rosa and Canadian. A soft X-ray image of each kernel was used to measure seven geometric properties: area, perimeter, compactness ($4\pi \cdot \text{area}/\text{perimeter}^2$), kernel length, kernel width, asymmetry coefficient and length of the kernel groove. All seven are continuous. I standardised them and used a stratified 70/30 split (147 training and 63 test kernels).

9.2 Code and experiments

- `kmeans` is a plain implementation with 10 random restarts.
- `RBFNetwork.fit` runs k-means (or, with `centre_method="random"`, simply picks random training points), applies the width rule, builds the hidden-activation matrix with `_hidden` and solves the least-squares problem for the output weights. A tiny ridge term (10^{-8}) keeps the solve stable when two hidden units are almost identical.
- `predict` takes the arg-max of the outputs.

Experiments: (1) a grid over the number of centres $K \in \{2, 3, 5, 8, 10, 15, 20, 30, 40\}$ and the width scale $\in \{0.5, 1, 2, 4\}$ with 5-fold CV; (2) the best network on the test set; (3) the same network with random centres instead of k-means; (4) a picture of a 2-D version of the network.

9.3 Results

Table 9.1. 5-fold cross-validation accuracy for different numbers of centres and widths.

K	2	3	5	8	10	15	20	30	40
width ×0.5	0.599	0.891	0.884	0.893	0.892	0.878	0.870	0.870	0.831
width ×1	0.769	0.925	0.925	0.905	0.905	0.905	0.891	0.899	0.904
width ×2	0.816	0.926	0.926	0.933	0.925	0.926	0.932	0.925	0.919
width ×4	0.831	0.926	0.919	**0.953**	**0.953**	0.919	0.945	0.952	0.939

Two centres are clearly not enough for three classes. From three centres on, accuracy is above 89% for every width. Narrow Gaussians (×0.5) are consistently worst: each unit only "sees" a small region and

the network starts to memorise individual kernels. The widest setting worked best, which tells us that the three varieties are separated by smooth, gently curved boundaries. The best result was $K = 8$ with width scale 4 ($\sigma = 6.72$ in standardised units), with a CV accuracy of 95.3%.

Table 9.2. Test-set results of the final network ($K = 8$, $\sigma = 6.72$, 63 kernels).

Variety	Precision	Recall	F1	Support
Kama	0.952	0.952	0.952	21
Rosa	1.000	1.000	1.000	21
Canadian	0.952	0.952	0.952	21
Macro average	0.968	0.968	0.968	63

Accuracy: **0.968** (61 of 63)
Random centres: 0.968 (width ×4), 0.873 (width ×1, where k-means gives 0.905)

The network classified 61 of the 63 test kernels correctly. Kama sits between the other two varieties in size, so its border with Canadian is where the mistakes happen. With these wide Gaussians, the exact positions of the centres hardly mattered: random centres gave the same test accuracy as k-means (96.8%). With the plain width rule (scale ×1) the centres mattered more: k-means reached 90.5% and random centres only 87.3%, because narrow bumps placed in the wrong spots leave parts of the input space uncovered.

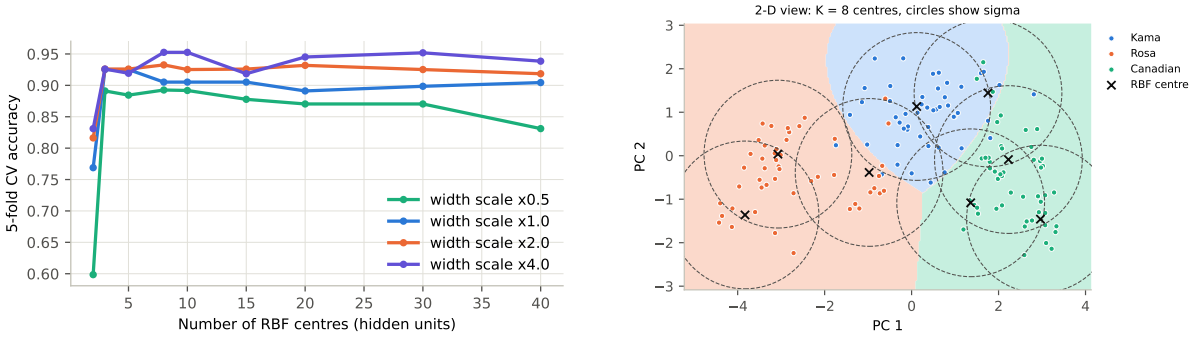


Figure 9.1. RBF centre/width selection and the two-dimensional decision regions.

9.4 Source code

Listing 9.1. `09_RBF_Network/rbf_network.py`

```
"""
Lab 09 - Radial Basis Function (RBF) Neural Network, written from scratch
Training: (1) k-means picks the centres, (2) a width rule sets sigma,
          (3) the output weights are found by linear least squares
Dataset : UCI Seeds (210 wheat kernels, 7 geometric measurements, 3 varieties)
Run     : python rbf_network.py      (only NumPy, Pandas and Matplotlib)
"""
import os
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap, ListedColormap

```

```
HERE = os.path.dirname(os.path.abspath(__file__))
FIG = os.path.join(HERE, "figures")
os.makedirs(FIG, exist_ok=True)
SEED = 42

plt.rcParams.update({
    "font.size": 9, "axes.spines.top": False, "axes.spines.right": False,
    "axes.edgecolor": "#c3c2b7", "xtick.color": "#52514e", "ytick.color": "#52514e",
    "axes.grid": True, "grid.color": "#e1e0d9", "grid.linewidth": 0.6,
    "lines.linewidth": 2, "savefig.bbox": "tight",
})
BLUE, ORANGE, AQUA, MUTED = "#2a78d6", "#eb6834", "#1baf7a", "#898781"
SEQ = LinearSegmentedColormap.from_list("seq", ["#f4f8fd", "#86b6ef", "#2a78d6", "#0d366b"])
NAMES = {1: "Kama", 2: "Rosa", 3: "Canadian"}

LINES = []
def log(text=""):
    print(text)
    LINES.append(str(text))


# ---------------------------------------------------------------------- step 1: k-means for the centres
def kmeans(X, k, n_init=10, max_iter=100, seed=SEED):
    """Plain k-means, restarted n_init times; returns the centres with the lowest SSE."""
    rng = np.random.default_rng(seed)
    best_sse, best_C = np.inf, None
    for _ in range(n_init):
        C = X[rng.choice(len(X), k, replace=False)].copy()      # random data points as start
        for _ in range(max_iter):
            d = ((X[:, None, :] - C[None, :, :]) ** 2).sum(-1)
            assign = d.argmin(1)
            newC = np.array([X[assign == j].mean(0) if np.any(assign == j) else C[j] for j in range(k)])
            if np.allclose(newC, C):
                break
            C = newC
        sse = ((X - C[assign]) ** 2).sum()
        if sse < best_sse:
            best_sse, best_C = sse, C
    return best_C


# ---------------------------------------------------------------------- the network
class RBFNetwork:
    """
    hidden unit j:  phi_j(x) = exp( -||x - c_j||^2 / (2 sigma^2) )
    output k     :  y_k(x)   = w_k0 + sum_j w_kj phi_j(x)
    """
    def __init__(self, n_centres, width_scale=1.0, centre_method="kmeans", seed=SEED):
        self.K, self.width_scale, self.centre_method, self.seed = n_centres, width_scale, centre_method,
     ↪ seed

    def _hidden(self, X):
        d2 = ((X[:, None, :] - self.C[None, :, :]) ** 2).sum(-1)
        Phi = np.exp(-d2 / (2 * self.sigma ** 2))
        return np.column_stack([np.ones(len(X)), Phi])             # bias column

```

```
    def fit(self, X, y):
        self.classes = np.unique(y)
        if self.centre_method == "kmeans":
            self.C = kmeans(X, self.K, seed=self.seed)
        else:                                                   # "random": just pick K samples
            self.C = X[np.random.default_rng(self.seed).choice(len(X), self.K, replace=False)]
        # width rule: sigma = d_max / sqrt(2K), d_max = largest distance between two centres
        dmax = np.sqrt(((self.C[:, None] - self.C[None]) ** 2).sum(-1)).max() if self.K > 1 else 1.0
        self.sigma = self.width_scale * dmax / np.sqrt(2 * self.K)
        T = (y[:, None] == self.classes[None, :]).astype(float)    # one-hot targets
        Phi = self._hidden(X)
        # least squares for the output layer (tiny ridge term keeps the solve stable)
        self.W = np.linalg.solve(Phi.T @ Phi + 1e-8 * np.eye(Phi.shape[1]), Phi.T @ T)
        return self

    def predict(self, X):
        return self.classes[(self._hidden(X) @ self.W).argmax(1)]


# ---------------------------------------------------------------- helpers and metrics
def stratified_split(y, test_frac, rng):
    test = []
    for c in np.unique(y):
        idx = rng.permutation(np.where(y == c)[0])
        test.extend(idx[:round(test_frac * len(idx))])
    mask = np.zeros(len(y), bool)
    mask[test] = True
    return np.where(~mask)[0], np.where(mask)[0]


def stratified_folds(y, n_folds, rng):
    folds = [[] for _ in range(n_folds)]
    for c in np.unique(y):
        for i, chunk in enumerate(np.array_split(rng.permutation(np.where(y == c)[0]), n_folds)):
            folds[i].extend(chunk)
    return [np.array(f) for f in folds]


def cv_accuracy(X, y, **kw):
    folds = stratified_folds(y, 5, np.random.default_rng(SEED))
    accs = []
    for f in range(5):
        val = folds[f]
        tr = np.concatenate([folds[g] for g in range(5) if g != f])
        mu, sd = X[tr].mean(0), X[tr].std(0)
        net = RBFNetwork(**kw).fit((X[tr] - mu) / sd, y[tr])
        accs.append(np.mean(net.predict((X[val] - mu) / sd) == y[val]))
    return np.mean(accs)


def confusion(y, yhat, labels):
    M = np.zeros((len(labels), len(labels)), int)
    for t, p in zip(y, yhat):
        M[np.searchsorted(labels, t), np.searchsorted(labels, p)] += 1
    return M

```

```python

# ------------------------------------------------------------------ experiment
def main():
    df = pd.read_csv(os.path.join(HERE, "data", "seeds.csv"))
    X = df.drop(columns="variety").to_numpy(float)
    y = df["variety"].to_numpy()
    labels = np.unique(y)
    log(f"Seeds data: {len(y)} kernels, {X.shape[1]} features, 70 of each variety")

    tr, te = stratified_split(y, 0.3, np.random.default_rng(SEED))
    mu, sd = X[tr].mean(0), X[tr].std(0)
    Xtr, Xte, ytr, yte = (X[tr] - mu) / sd, (X[te] - mu) / sd, y[tr], y[te]
    log(f"Train/test split: {len(tr)} / {len(te)}")

    # 1) how many hidden units (centres)? and how wide?
    Ks = [2, 3, 5, 8, 10, 15, 20, 30, 40]
    scales = [0.5, 1.0, 2.0, 4.0]
    grid = {(K, s): cv_accuracy(X[tr], ytr, n_centres=K, width_scale=s) for K in Ks for s in scales}
    log("\n5-fold CV accuracy (rows: number of centres K, columns: width scale)")
    log("    K  " + "  ".join(f"x{s:<5}" for s in scales))
    for K in Ks:
        log(f"   {K:2d}  " + "  ".join(f"{grid[(K, s)]:.4f}" for s in scales))
    (best_K, best_s) = max(grid, key=lambda k: (round(grid[k], 4), -k[0], -abs(k[1] - 1)))
    log(f"Chosen: K = {best_K}, width scale = {best_s} (CV accuracy {grid[(best_K, best_s)]:.4f})")

    # 2) final network, scored on the test set
    net = RBFNetwork(best_K, best_s).fit(Xtr, ytr)
    pred = net.predict(Xte)
    M = confusion(yte, pred, labels)
    tp = np.diag(M)
    prec, rec = tp / M.sum(0), tp / M.sum(1)
    f1 = 2 * prec * rec / (prec + rec)
    log(f"\nTest-set results (sigma = {net.sigma:.3f})")
    log(f"  Accuracy        : {np.mean(pred == yte):.4f}")
    log(f"  Macro precision : {prec.mean():.4f}")
    log(f"  Macro recall    : {rec.mean():.4f}")
    log(f"  Macro F1        : {f1.mean():.4f}")
    for c, p, r, f in zip(labels, prec, rec, f1):
        log(f"  {NAMES[c]:9s}: precision={p:.3f} recall={r:.3f} f1={f:.3f}")
    log(f"  Confusion matrix:\n{M}")

    log("\nk-means centres vs random centres (same K, test accuracy):")
    for s in sorted({1.0, best_s}):
        km = RBFNetwork(best_K, s).fit(Xtr, ytr)
        rand = RBFNetwork(best_K, s, centre_method="random").fit(Xtr, ytr)
        log(f"   width scale x{s}: k-means {np.mean(km.predict(Xte) == yte):.4f}, "
            f"random {np.mean(rand.predict(Xte) == yte):.4f}")

    # --- figure 1: CV accuracy vs number of centres
    fig, ax = plt.subplots(figsize=(5.4, 2.7))
    for s, c in zip(scales, [AQUA, BLUE, ORANGE, "#6250d6"]):
        ax.plot(Ks, [grid[(K, s)] for K in Ks], marker="o", ms=3.5, color=c, label=f"width scale x{s}")
    ax.set_xlabel("Number of RBF centres (hidden units)")
    ax.set_ylabel("5-fold CV accuracy")
    ax.legend(frameon=False, loc="lower right")
    fig.savefig(os.path.join(FIG, "cv_vs_centres.pdf"))
```

```
    plt.close(fig)

    # --- figure 2: what the network does in 2-D (first two principal components)
    _, _, Vt = np.linalg.svd(Xtr - Xtr.mean(0), full_matrices=False)
    Z = Xtr @ Vt[:2].T
    net2 = RBFNetwork(best_K, 1.0).fit(Z, ytr)          # width x1 so the circles stay readable
    gx, gy = np.meshgrid(np.linspace(Z[:, 0].min() - 0.8, Z[:, 0].max() + 0.8, 250),
                         np.linspace(Z[:, 1].min() - 0.8, Z[:, 1].max() + 0.8, 250))
    g = net2.predict(np.c_[gx.ravel(), gy.ravel()]).reshape(gx.shape)
    fig, ax = plt.subplots(figsize=(4.8, 3.3))
    ax.contourf(gx, gy, g, levels=[0.5, 1.5, 2.5, 3.5], cmap=ListedColormap(["#cde2fb", "#fbd9cb", "#
     ↪ c6efdf"]))
    for c, col in zip(labels, [BLUE, ORANGE, AQUA]):
        ax.scatter(Z[ytr == c, 0], Z[ytr == c, 1], s=12, color=col, edgecolor="white", lw=0.4, label=
     ↪ NAMES[c])
    for cx, cy in net2.C:
        ax.add_patch(plt.Circle((cx, cy), net2.sigma, fill=False, color="#52514e", lw=0.7, ls="--"))
    ax.scatter(net2.C[:, 0], net2.C[:, 1], marker="x", s=30, color="#0b0b0b", lw=1.2, label="RBF centre")
    ax.set_xlim(gx.min(), gx.max())
    ax.set_ylim(gy.min(), gy.max())
    ax.set_aspect("equal")
    ax.set_title(f"2-D view: K = {best_K} centres, circles show sigma", fontsize=8)
    ax.set_xlabel("PC 1")
    ax.set_ylabel("PC 2")
    ax.grid(False)
    ax.legend(frameon=False, fontsize=7, loc="upper left", bbox_to_anchor=(1.0, 1.0))
    fig.savefig(os.path.join(FIG, "rbf_2d_regions.pdf"))
    plt.close(fig)

    # --- figure 3: confusion matrix
    fig, ax = plt.subplots(figsize=(3.4, 3.0))
    ax.imshow(M, cmap=SEQ)
    ax.grid(False)
    for i in range(3):
        for j in range(3):
            ax.text(j, i, M[i, j], ha="center", va="center", color="white" if M[i, j] > M.max() / 2 else
     ↪ "#0b0b0b")
    ax.set_xticks(range(3), [NAMES[c] for c in labels])
    ax.set_yticks(range(3), [NAMES[c] for c in labels])
    ax.set_xlabel("Predicted")
    ax.set_ylabel("Actual")
    ax.set_title(f"RBF network, K = {best_K}")
    fig.savefig(os.path.join(FIG, "confusion_matrix.pdf"))
    plt.close(fig)

    with open(os.path.join(HERE, "results.txt"), "w") as f:
        f.write("\n".join(LINES) + "\n")


if __name__ == "__main__":
    main()
```

LAB 10

Support Vector Machine

Folder	`10_SVM`
Script	`svm.py`
Dataset	UCI Sonar, Mines vs. Rocks: 208 sonar returns, 60 frequency-band features
Task	Binary classification (metal cylinder or rock)
Main result	RBF kernel: CV accuracy 85.0%, test accuracy 95.1%, AUC 0.969. Linear kernel: test accuracy 75.6%

10.1 Dataset

The **Sonar** dataset (Gorman and Sejnowski, 1988; UCI) contains 208 sonar signals bounced off either a metal cylinder (111, "mines") or a roughly cylindrical rock (97), from many angles. Each signal is described by 60 numbers between 0 and 1, the energy in 60 frequency bands. With only 208 examples for 60 features, and classes that overlap a lot, it is a classic hard benchmark and a good place to see whether a non-linear kernel helps. I standardised the features and used a stratified 80/20 split (167 training, 41 test signals), with mines as the positive class.

10.2 Code and experiments

- `kernel_matrix` computes the linear or RBF kernel between two sets of points.
- `SVM.fit` is the SMO solver described above. It keeps a vector `u` with $\sum_s \alpha_s y_s K_{st}$ for every training point, so each step costs only $O(n)$. Multipliers that come within 10^{-10} of 0 or C are snapped to the bound; without this, tiny round-off values could make the solver pick the same pair forever.
- `decision_function`, `predict` and `kkt_violation` (a final check that the solution really is optimal) complete the class.

Experiments: (1) 5-fold CV for the linear kernel over $C \in \{0.001, \dots, 10\}$ and for the RBF kernel over $C \in \{0.1, 1, 10, 100\}$ and $\gamma \in \{0.001, \dots, 0.1\}$; (2) the best linear and best RBF models on the test set.

10.3 Results

For the linear kernel the best CV accuracy was 77.1% at $C = 0.01$. The RBF kernel does much better, with 85.0% at $C = 100$, $\gamma = 0.01$. The heat map also shows the two ways an RBF SVM can fail. With $C = 0.1$ the penalty is so weak that the model predicts the majority class for every signal (53.3%). With $\gamma = 0.1$ each training point only influences its immediate neighbourhood, so the model memorises and generalises poorly (76.0%).

Table 10.1. Test-set results (41 signals: 22 mines, 19 rocks).

Model	Acc.	Prec.	Recall	F1	Spec.	AUC	#SV	SMO steps
Linear, $C = 0.01$	0.756 1	0.800 0	0.727 3	0.761 9	0.789 5	0.863 6	116	117
RBF, $C = 100$, $\gamma = 0.01$	0.951 2	0.916 7	1.000 0	0.956 5	0.894 7	0.968 9	101	433

The RBF SVM found all 22 mines in the test set and mislabelled two of the 19 rocks. The test accuracy

(95.1%) is clearly higher than its CV accuracy (85.0%). With 41 test signals each one is worth 2.4 points, and this particular split seems to have been on the easy side, so I would quote the CV figure of about 85% as the realistic expectation. The gap between the kernels is real, though: both the CV and the test results show the RBF kernel 8 to 20 points ahead of the linear one.

Checking the solver. After training, the largest violation of the KKT conditions was 0.0005, below the tolerance of 0.001, so SMO really did reach the optimum of the dual. For the linear SVM I also recovered $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$: $\|\mathbf{w}\| = 0.599$, a margin width of $2/\|\mathbf{w}\| = 3.34$ standardised units. The RBF SVM uses 101 of its 167 training points as support vectors, all strictly between 0 and C, so every training point is classified correctly and sits on or outside the margin.

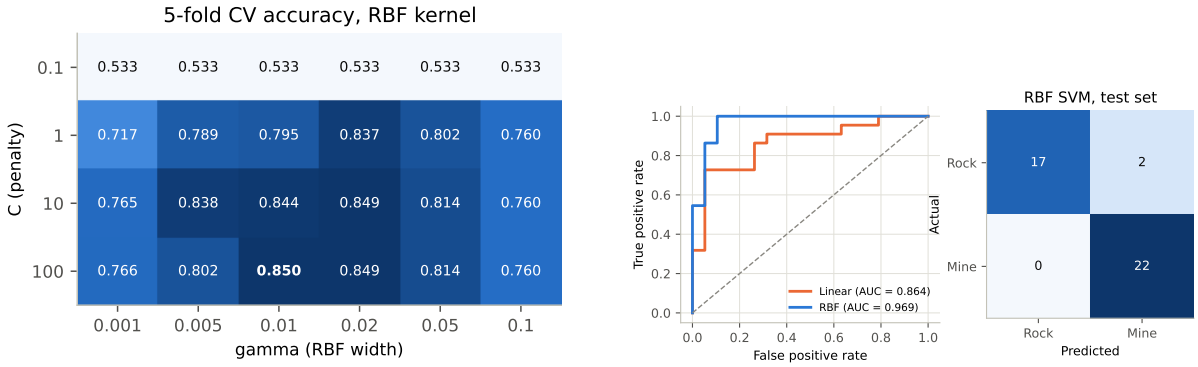


Figure 10.1. SVM hyperparameter heatmap, ROC curves and test confusion matrix.

10.4 Source code

Listing 10.1. `10_SVM/svm.py`

```
"""
Lab 10 - Support Vector Machine (soft margin, kernel), solved in the dual with SMO, from scratch
Dataset : UCI Sonar, Mines vs. Rocks (208 sonar returns, 60 frequency-band energies;
          M = metal cylinder (mine), R = rock)
Run     : python svm.py      (only NumPy, Pandas and Matplotlib)
"""
import os
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap

HERE = os.path.dirname(os.path.abspath(__file__))
FIG = os.path.join(HERE, "figures")
os.makedirs(FIG, exist_ok=True)
SEED = 42

plt.rcParams.update({
    "font.size": 9, "axes.spines.top": False, "axes.spines.right": False,
    "axes.edgecolor": "#c3c2b7", "xtick.color": "#52514e", "ytick.color": "#52514e",
    "axes.grid": True, "grid.color": "#e1e0d9", "grid.linewidth": 0.6,
    "lines.linewidth": 2, "savefig.bbox": "tight",
})
```

```
BLUE, ORANGE, MUTED = "#2a78d6", "#eb6834", "#898781"
SEQ = LinearSegmentedColormap.from_list("seq", ["#f4f8fd", "#86b6ef", "#2a78d6", "#0d366b"])

LINES = []
def log(text=""):
    print(text)
    LINES.append(str(text))


# ---------------------------------------------------------------- kernels
def kernel_matrix(A, B, kind="rbf", gamma=0.1):
    if kind == "linear":
        return A @ B.T
    if kind == "rbf":           # K(a, b) = exp(-gamma * ||a - b||^2)
        sq = (A ** 2).sum(1)[:, None] + (B ** 2).sum(1)[None, :] - 2 * A @ B.T
        return np.exp(-gamma * np.maximum(sq, 0))
    raise ValueError(kind)


# ---------------------------------------------------------------- the SVM
class SVM:
    """
    Dual problem (from the Lagrangian of the soft-margin SVM):
        maximise   sum_i a_i - 1/2 sum_i sum_j a_i a_j y_i y_j K(x_i, x_j)
        subject to 0 <= a_i <= C   and   sum_i a_i y_i = 0
    SMO solves it by repeatedly picking the two multipliers that break the optimality
    (KKT) conditions the most and solving that 2-variable problem exactly.
    """
    def __init__(self, C=1.0, kernel="rbf", gamma=0.1, tol=1e-3, max_iter=200000):
        self.C, self.kernel, self.gamma, self.tol, self.max_iter = C, kernel, gamma, tol, max_iter

    def fit(self, X, y):
        n, C = len(y), self.C
        K = kernel_matrix(X, X, self.kernel, self.gamma)
        a = np.zeros(n)
        u = np.zeros(n)                     # u_t = sum_s a_s y_s K(x_s, x_t)  (output without b)
        for it in range(self.max_iter):
            F = y - u                       # F_t = y_t - u_t; at the optimum, b sits between max/min
 ↪ below
            free_lo, free_hi = a > 1e-10, a < C - 1e-10           # not stuck at 0 / not stuck at C
            up = ((y == 1) & free_hi) | ((y == -1) & free_lo)     # y_t a_t can still increase
            low = ((y == 1) & free_lo) | ((y == -1) & free_hi)    # y_t a_t can still decrease
            i = np.where(up)[0][np.argmax(F[up])]
            if F[i] - F[low].min() < self.tol:   # no pair breaks KKT by more than tol -> done
                break
            # pick j: the partner that gives the biggest improvement of the dual objective
            gap = F[i] - F                       # how much each t disagrees with i
            curv = np.maximum(K[i, i] + np.diag(K) - 2 * K[i], 1e-12)
            gain = np.where(low & (gap > 0), gap ** 2 / curv, -np.inf)
            j = int(np.argmax(gain))
            # --- solve the two-variable problem for a_i and a_j exactly
            eta = max(K[i, i] + K[j, j] - 2 * K[i, j], 1e-12)
            if y[i] != y[j]:
                L, H = max(0, a[j] - a[i]), min(C, C + a[j] - a[i])
            else:
                L, H = max(0, a[i] + a[j] - C), min(C, a[i] + a[j])
```

```
            Ei, Ej = u[i] - y[i], u[j] - y[j]                    # errors (b cancels out)
            aj = np.clip(a[j] + y[j] * (Ei - Ej) / eta, L, H)
            ai = a[i] + y[i] * y[j] * (a[j] - aj)                # keeps sum a_t y_t = 0
            ai, aj = [0.0 if v < 1e-10 else C if v > C - 1e-10 else v for v in (ai, aj)]  # snap
            u += (ai - a[i]) * y[i] * K[:, i] + (aj - a[j]) * y[j] * K[:, j]
            a[i], a[j] = ai, aj
        self.iterations = it + 1
        # bias from the "free" support vectors (0 < a < C), which sit exactly on the margin
        free = (a > 1e-8) & (a < C - 1e-8)
        F = y - u
        self.b = np.mean(F[free]) if free.any() else (F[up].max() + F[low].min()) / 2
        sv = a > 1e-8
        self.alpha, self.sv_X, self.sv_y = a[sv], X[sv], y[sv]
        self.n_sv, self.n_bounded = int(sv.sum()), int((a > C - 1e-8).sum())
        self._train_K, self._train_y, self._a_all, self._u = K, y, a, u
        return self

    def decision_function(self, X):
        """f(x) = sum over support vectors of a_s y_s K(x_s, x) + b"""
        return kernel_matrix(X, self.sv_X, self.kernel, self.gamma) @ (self.alpha * self.sv_y) + self.b

    def predict(self, X):
        return np.where(self.decision_function(X) >= 0, 1, -1)

    def kkt_violation(self):
        """Largest break of the KKT conditions on the training set (should be about tol)."""
        a, y, C = self._a_all, self._train_y, self.C
        m = y * (self._u + self.b)          # functional margin y_i f(x_i)
        v = np.where(a < 1e-8, np.maximum(0, 1 - m),
                     np.where(a > C - 1e-8, np.maximum(0, m - 1), np.abs(m - 1)))
        return float(v.max())


# ---------------------------------------------------------------------- helpers and metrics
def stratified_split(y, test_frac, rng):
    test = []
    for c in np.unique(y):
        idx = rng.permutation(np.where(y == c)[0])
        test.extend(idx[:round(test_frac * len(idx))])
    mask = np.zeros(len(y), bool)
    mask[test] = True
    return np.where(~mask)[0], np.where(mask)[0]


def stratified_folds(y, n_folds, rng):
    folds = [[] for _ in range(n_folds)]
    for c in np.unique(y):
        for i, chunk in enumerate(np.array_split(rng.permutation(np.where(y == c)[0]), n_folds)):
            folds[i].extend(chunk)
    return [np.array(f) for f in folds]


def cv_accuracy(X, y, **params):
    folds = stratified_folds(y, 5, np.random.default_rng(SEED))
    accs = []
    for f in range(5):
```

```
        val = folds[f]
        tr = np.concatenate([folds[g] for g in range(5) if g != f])
        mu, sd = X[tr].mean(0), X[tr].std(0)
        m = SVM(**params).fit((X[tr] - mu) / sd, y[tr])
        accs.append(np.mean(m.predict((X[val] - mu) / sd) == y[val]))
    return np.mean(accs)


def binary_metrics(y, yhat):
    tp, tn = np.sum((y == 1) & (yhat == 1)), np.sum((y == -1) & (yhat == -1))
    fp, fn = np.sum((y == -1) & (yhat == 1)), np.sum((y == 1) & (yhat == -1))
    prec, rec = tp / (tp + fp), tp / (tp + fn)
    return dict(acc=(tp + tn) / len(y), precision=prec, recall=rec, f1=2 * prec * rec / (prec + rec),
                specificity=tn / (tn + fp), cm=np.array([[tn, fp], [fn, tp]]))


def roc_auc(y, score):
    order = np.argsort(-score, kind="stable")
    t = y[order] == 1
    tpr = np.concatenate([[0], np.cumsum(t) / t.sum()])
    fpr = np.concatenate([[0], np.cumsum(~t) / (~t).sum()])
    return fpr, tpr, float(np.sum((fpr[1:] - fpr[:-1]) * (tpr[1:] + tpr[:-1]) / 2))


# ---------------------------------------------------------------- experiment
def main():
    df = pd.read_csv(os.path.join(HERE, "data", "sonar.csv"))
    X = df.drop(columns="label").to_numpy(float)
    y = np.where(df["label"] == "M", 1, -1)
    log(f"Sonar data: {len(y)} returns, {X.shape[1]} features, mines = {np.sum(y == 1)}, rocks = {np.sum(
    ↪ y == -1)}")

    tr, te = stratified_split(y, 0.2, np.random.default_rng(SEED))
    mu, sd = X[tr].mean(0), X[tr].std(0)
    Xtr, Xte, ytr, yte = (X[tr] - mu) / sd, (X[te] - mu) / sd, y[tr], y[te]
    log(f"Train/test split: {len(tr)} / {len(te)}")

    # 1) model selection with 5-fold CV
    Cs_lin = [0.001, 0.01, 0.1, 1, 10]
    lin_cv = {C: cv_accuracy(X[tr], ytr, C=C, kernel="linear") for C in Cs_lin}
    log("\nLinear kernel, 5-fold CV accuracy: " + ", ".join(f"C={C}: {v:.4f}" for C, v in lin_cv.items())
    ↪ )
    best_C_lin = max(lin_cv, key=lambda c: (round(lin_cv[c], 4), -c))

    Cs, gammas = [0.1, 1, 10, 100], [0.001, 0.005, 0.01, 0.02, 0.05, 0.1]
    rbf_cv = np.array([[cv_accuracy(X[tr], ytr, C=C, kernel="rbf", gamma=g) for g in gammas] for C in Cs
    ↪ ])
    log("\nRBF kernel, 5-fold CV accuracy (rows C, columns gamma):")
    log("   C\\gamma " + " ".join(f"{g:>7}" for g in gammas))
    for C, row in zip(Cs, rbf_cv):
        log(f"   {C:<7} " + " ".join(f"{v:7.4f}" for v in row))
    bi, bj = np.unravel_index(np.argmax(rbf_cv), rbf_cv.shape)
    best_C, best_g = Cs[bi], gammas[bj]
    log(f"Best linear: C={best_C_lin} (CV {lin_cv[best_C_lin]:.4f});  best RBF: C={best_C}, gamma={best_g
    ↪ } (CV {rbf_cv[bi, bj]:.4f})")

```

```
    # 2) final models on the test set
    log(f"\n{'Model':22s} {'Acc':>6s} {'Prec':>6s} {'Rec':>6s} {'F1':>6s} {'Spec':>6s} {'AUC':>6s} {'#SV
     ↪ ':>5s} {'iters':>6s} {'KKT':>7s}")
    finals = {}
    for name, params in [(f"Linear (C={best_C_lin})", dict(C=best_C_lin, kernel="linear")),
                         (f"RBF (C={best_C}, g={best_g})", dict(C=best_C, kernel="rbf", gamma=best_g))]:
        m = SVM(**params).fit(Xtr, ytr)
        s = m.decision_function(Xte)
        r = binary_metrics(yte, np.where(s >= 0, 1, -1))
        fpr, tpr, auc = roc_auc(yte, s)
        finals[name] = (m, r, fpr, tpr, auc)
        log(f"{name:22s} {r['acc']:6.4f} {r['precision']:6.4f} {r['recall']:6.4f} {r['f1']:6.4f} "
            f"{r['specificity']:6.4f} {auc:6.4f} {m.n_sv:5d} {m.iterations:6d} {m.kkt_violation():7.4f}")
    lin_model = finals[f"Linear (C={best_C_lin})"][0]
    w = (lin_model.alpha * lin_model.sv_y) @ lin_model.sv_X
    log(f"\nLinear SVM: ||w|| = {np.linalg.norm(w):.3f}, margin width 2/||w|| = {2 / np.linalg.norm(w):.3
     ↪ f}")
    rbf_name = f"RBF (C={best_C}, g={best_g})"
    m, r = finals[rbf_name][0], finals[rbf_name][1]
    log(f"RBF SVM: {m.n_sv} support vectors out of {len(ytr)} training points ({m.n_bounded} at the bound
     ↪  a = C)")
    log(f"RBF confusion matrix [[TN FP] [FN TP]] (positive = mine) = {r['cm'].tolist()}")

    # --- figure 1: CV heat map
    fig, ax = plt.subplots(figsize=(4.6, 2.6))
    ax.imshow(rbf_cv, cmap=SEQ, aspect="auto")
    ax.grid(False)
    for i in range(len(Cs)):
        for j in range(len(gammas)):
            ax.text(j, i, f"{rbf_cv[i, j]:.3f}", ha="center", va="center", fontsize=7.5,
                    color="white" if rbf_cv[i, j] > (rbf_cv.min() + rbf_cv.max()) / 2 else "#0b0b0b",
                    fontweight="bold" if (i, j) == (bi, bj) else "normal")
    ax.set_xticks(range(len(gammas)), gammas)
    ax.set_yticks(range(len(Cs)), Cs)
    ax.set_xlabel("gamma (RBF width)")
    ax.set_ylabel("C (penalty)")
    ax.set_title("5-fold CV accuracy, RBF kernel")
    fig.savefig(os.path.join(FIG, "cv_heatmap.pdf"))
    plt.close(fig)

    # --- figure 2: ROC curves and confusion matrix
    fig, axes = plt.subplots(1, 2, figsize=(6.6, 2.8), gridspec_kw={"width_ratios": [1.25, 1]})
    for (name, (_, _, fpr, tpr, auc)), c in zip(finals.items(), [ORANGE, BLUE]):
        axes[0].plot(fpr, tpr, color=c, label=f"{name.split(' (')[0]} (AUC = {auc:.3f})")
    axes[0].plot([0, 1], [0, 1], color=MUTED, ls="--", lw=1)
    axes[0].set_xlabel("False positive rate")
    axes[0].set_ylabel("True positive rate")
    axes[0].legend(frameon=False, loc="lower right", fontsize=7.5)
    cm = r["cm"]
    axes[1].imshow(cm, cmap=SEQ)
    axes[1].grid(False)
    for i in range(2):
        for j in range(2):
            axes[1].text(j, i, cm[i, j], ha="center", va="center", color="white" if cm[i, j] > cm.max() /
     ↪  2 else "#0b0b0b")
    axes[1].set_xticks([0, 1], ["Rock", "Mine"])
```

```
    axes[1].set_yticks([0, 1], ["Rock", "Mine"])
    axes[1].set_xlabel("Predicted")
    axes[1].set_ylabel("Actual")
    axes[1].set_title("RBF SVM, test set")
    fig.savefig(os.path.join(FIG, "roc_and_confusion.pdf"))
    plt.close(fig)

    # --- figure 3: distribution of y*f(x) on the training set (who is a support vector?)
    marg = ytr * m.decision_function(Xtr)
    fig, ax = plt.subplots(figsize=(5.4, 2.5))
    ax.hist(marg, bins=np.linspace(0, 3, 61), color=BLUE)
    ax.axvline(1, color=ORANGE, lw=1.2, ls="--")
    ax.text(1.35, ax.get_ylim()[1] * 0.8, f"margin y f(x) = 1  ({m.n_sv} support vectors sit here)",
            color="#52514e", fontsize=8)
    ax.set_xlabel("y f(x) on training points (RBF SVM)")
    ax.set_ylabel("Count")
    fig.savefig(os.path.join(FIG, "functional_margins.pdf"))
    plt.close(fig)

    with open(os.path.join(HERE, "results.txt"), "w") as f:
        f.write("\n".join(LINES) + "\n")


if __name__ == "__main__":
    main()
```

LAB 11

Support Vector Regression

Folder	`11_SVR`
Script	`svr.py`
Dataset	UCI Auto MPG: 392 cars (1970–1982), 8 inputs
Task	Regression (fuel economy in miles per gallon)
Main result	RBF SVR: test RMSE 2.67 mpg, $R^2 = 0.887$ (least squares: RMSE 3.77, $R^2 = 0.776$)

11.1 Dataset

The **Auto MPG** dataset (UCI) describes 398 car models from 1970 to 1982. The target is fuel economy in miles per gallon (9 to 47, mean 23.4). The inputs are the number of cylinders, engine displacement, horsepower, weight, acceleration, model year and origin (USA, Europe or Japan). Six cars have no horsepower value, so I dropped them, leaving 392. Origin was turned into two dummy columns (Europe, Japan) and the car name was not used. I used a random 80/20 split (314 training, 78 test cars) and standardised the inputs.

11.2 Code and experiments

- `kernel_matrix` gives the linear or RBF kernel.
- `SVR.fit` keeps a sign vector `z` ($+1$ for α_i, -1 for α_i^*), a vector `pt` that says which data point each multiplier belongs to, and the running sum $u_k = \sum_i (\alpha_i - \alpha_i^*) K_{ik}$. Each SMO step picks the pair, computes the step `t`, clips it and updates `u` in $O(n)$.
- `predict` uses only the support vectors.
- `cv_rmse` runs 5-fold CV, standardising inside each fold.

Experiments: (1) a grid of 48 settings, $C \in \{1, 10, 100, 1000\}$, $\gamma \in \{0.02, 0.05, 0.1, 0.3\}$, $\varepsilon \in \{0.5, 1, 2\}$, compared by 5-fold CV RMSE; (2) the best RBF SVR against a linear SVR and ordinary least squares on the test set; (3) the effect of ε with C and γ fixed; (4) a one-input SVR (mpg against horsepower) to show the tube on real data.

11.3 Results

The best five-fold CV result was $C = 100$, $\gamma = 0.1$, $\varepsilon = 0.5$ (CV RMSE 2.53 mpg). The next few settings were within 0.04 mpg of it, so the choice is not delicate.

Table 11.1. Test-set comparison (78 cars). "In tube" is the share of test cars predicted within $\pm\varepsilon$.

Model	MAE	RMSE	R^2	#SV	In tube
Linear regression (least squares)	2.749	3.765	0.775 9	–	–
Linear SVR ($C = 10$, $\varepsilon = 1$)	2.798	4.029	0.743 4	221	24.4%
RBF SVR ($C = 100$, $\gamma = 0.1$, $\varepsilon = 0.5$)	1.936	2.672	0.887 1	241	29.5%

The RBF SVR cuts the test RMSE by 29% compared with least squares (3.77 to 2.67 mpg) and raises R^2 from 0.78 to 0.89. Fuel economy depends on weight and horsepower in a curved way (it falls quickly

at first and then levels off), which a straight line cannot capture. The linear SVR is not better than least squares; it is still a straight line, just fitted with a different loss. So the improvement comes from the kernel, not from the loss function.

Table 11.2. Effect of the tube width ε ($C = 100$, $\gamma = 0.1$).

ε (mpg)	0.1	0.25	0.5	1	2	3	4	6
Support vectors (of 314)	300	276	241	193	111	67	44	24
Test RMSE (mpg)	2.681	2.657	2.672	2.673	2.820	3.038	3.283	3.809

The ε study shows the sparsity of SVR clearly. With $\varepsilon = 1$ mpg the model uses 193 support vectors and is as accurate as with $\varepsilon = 0.1$ (300 support vectors). With $\varepsilon = 4$ mpg only 44 cars define the whole function, at the cost of about 0.6 mpg of extra RMSE.

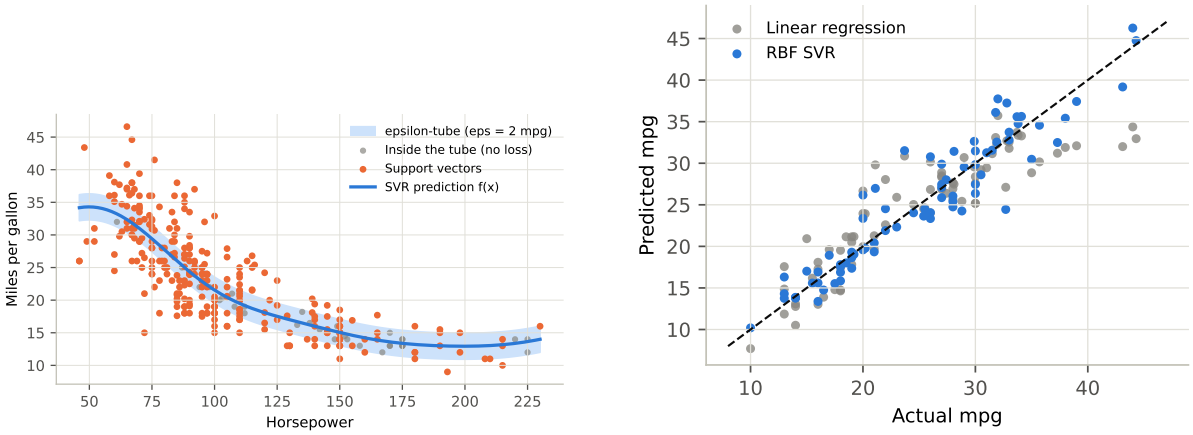


Figure 11.1. The SVR epsilon tube and predicted-versus-actual test values.

11.4 Source code

Listing 11.1. `11_SVR/svr.py`

```
"""
Lab 11 - Support Vector Regression (epsilon-SVR, kernel), solved in the dual, from scratch
Solver  : SMO on the dual problem (pairs of Lagrange multipliers at a time)
Dataset : UCI Auto MPG (398 cars from 1970-82; predict fuel economy in miles per gallon)
Run     : python svr.py      (only NumPy, Pandas and Matplotlib)
"""
import os
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

HERE = os.path.dirname(os.path.abspath(__file__))
FIG = os.path.join(HERE, "figures")
os.makedirs(FIG, exist_ok=True)
SEED = 42

plt.rcParams.update({
    "font.size": 9, "axes.spines.top": False, "axes.spines.right": False,
```

```
    "axes.edgecolor": "#c3c2b7", "xtick.color": "#52514e", "ytick.color": "#52514e",
    "axes.grid": True, "grid.color": "#e1e0d9", "grid.linewidth": 0.6,
    "lines.linewidth": 2, "savefig.bbox": "tight",
})
BLUE, ORANGE, AQUA, MUTED = "#2a78d6", "#eb6834", "#1baf7a", "#898781"

LINES = []
def log(text=""):
    print(text)
    LINES.append(str(text))


# ---------------------------------------------------------------------- kernels
def kernel_matrix(A, B, kind="rbf", gamma=0.1):
    if kind == "linear":
        return A @ B.T
    sq = (A ** 2).sum(1)[:, None] + (B ** 2).sum(1)[None, :] - 2 * A @ B.T
    return np.exp(-gamma * np.maximum(sq, 0))


# ---------------------------------------------------------------------- the SVR
class SVR:
    """
    Primal: minimise 1/2 ||w||^2 + C * sum(xi_i + xi*_i)
            so that every point is inside the tube |y_i - f(x_i)| <= eps (plus slack).
    Dual  : minimise 1/2 sum_ij (a_i - a*_i)(a_j - a*_j) K_ij + eps sum (a_i + a*_i) - sum y_i (a_i - a*
     ↪ _i)
            subject to 0 <= a_i, a*_i <= C   and   sum (a_i - a*_i) = 0
    Solved with SMO: keep 2n multipliers (a_1..a_n, a*_1..a*_n), repeatedly pick the two that
    break the KKT conditions most, and move them by the best step that keeps the constraints.
    Prediction: f(x) = sum_i (a_i - a*_i) K(x_i, x) + b.  Points with a_i != a*_i are support vectors.
    """
    def __init__(self, C=10.0, eps=1.0, kernel="rbf", gamma=0.1, tol=1e-3, max_iter=20000):
        self.C, self.eps, self.kernel, self.gamma = C, eps, kernel, gamma
        self.tol, self.max_iter = tol, max_iter

    def fit(self, X, y):
        n, C, eps = len(y), self.C, self.eps
        K = kernel_matrix(X, X, self.kernel, self.gamma)
        z = np.r_[np.ones(n), -np.ones(n)]          # +1 for a_i, -1 for a*_i
        pt = np.r_[np.arange(n), np.arange(n)]      # data point that each multiplier belongs to
        a = np.zeros(2 * n)
        u = np.zeros(n)                             # u_k = sum_i (a_i - a*_i) K(x_i, x_k)
        diagK = np.diag(K)[pt]
        for it in range(self.max_iter):
            # F_t is minus the dual gradient (times z_t); at the optimum it equals b for "free"
     ↪ multipliers
            F = np.r_[y - u - eps, y - u + eps]
            free_lo, free_hi = a > 1e-10, a < C - 1e-10
            up = ((z == 1) & free_hi) | ((z == -1) & free_lo)
            low = ((z == 1) & free_lo) | ((z == -1) & free_hi)
            i = np.where(up)[0][np.argmax(F[up])]
            if F[i] - F[low].min() < self.tol:      # KKT conditions hold (up to tol) -> done
                break
            gap = F[i] - F
            curv = np.maximum(K[pt[i], pt[i]] + diagK - 2 * K[pt[i], pt], 1e-12)
```

```
            j = int(np.argmax(np.where(low & (gap > 0), gap ** 2 / curv, -np.inf)))
            # best step t along the direction that keeps sum(a - a*) = 0, then clip to the box
            t = gap[j] / curv[j]
            t = min(t, C - a[i] if z[i] == 1 else a[i], a[j] if z[j] == 1 else C - a[j])
            a[i] += z[i] * t
            a[j] -= z[j] * t
            u += t * (K[:, pt[i]] - K[:, pt[j]])
        self.iterations = it + 1
        F = np.r_[y - u - eps, y - u + eps]
        free = (a > 1e-8) & (a < C - 1e-8)
        self.b = F[free].mean() if free.any() else (F[up].max() + F[low].min()) / 2
        beta = a[:n] - a[n:]
        sv = np.abs(beta) > 1e-8
        self.X_sv, self.beta_sv, self.n_sv = X[sv], beta[sv], int(sv.sum())
        return self

    def predict(self, X):
        return kernel_matrix(X, self.X_sv, self.kernel, self.gamma) @ self.beta_sv + self.b


# ---------------------------------------------------------------------- helpers and metrics
def scores(y, yhat):
    err = y - yhat
    return (np.mean(np.abs(err)), np.sqrt(np.mean(err ** 2)),
            1 - np.sum(err ** 2) / np.sum((y - y.mean()) ** 2))


def cv_rmse(X, y, n_folds=5, **params):
    folds = np.array_split(np.random.default_rng(SEED).permutation(len(y)), n_folds)
    out = []
    for f in range(n_folds):
        val = folds[f]
        tr = np.concatenate([folds[g] for g in range(n_folds) if g != f])
        mu, sd = X[tr].mean(0), X[tr].std(0)
        m = SVR(**params).fit((X[tr] - mu) / sd, y[tr])
        out.append(scores(y[val], m.predict((X[val] - mu) / sd))[1])
    return np.mean(out)


# ---------------------------------------------------------------------- experiment
def main():
    df = pd.read_csv(os.path.join(HERE, "data", "auto_mpg.csv"))
    n_missing = int(df["horsepower"].isna().sum())
    df = df.dropna(subset=["horsepower"]).reset_index(drop=True)
    df["europe"] = (df["origin"] == 2).astype(float)
    df["japan"] = (df["origin"] == 3).astype(float)
    features = ["cylinders", "displacement", "horsepower", "weight", "acceleration",
                "model_year", "europe", "japan"]
    X = df[features].to_numpy(float)
    y = df["mpg"].to_numpy(float)
    log(f"Auto MPG: {len(y)} cars after dropping {n_missing} with missing horsepower; "
        f"mpg range {y.min():.0f}-{y.max():.0f}, mean {y.mean():.1f}")

    idx = np.random.default_rng(SEED).permutation(len(y))
    n_test = int(0.2 * len(y))
    te, tr = idx[:n_test], idx[n_test:]
```

```
    mu, sd = X[tr].mean(0), X[tr].std(0)
    Xtr, Xte, ytr, yte = (X[tr] - mu) / sd, (X[te] - mu) / sd, y[tr], y[te]
    log(f"Train/test split: {len(tr)} / {len(te)}")

    # 1) hyper-parameters by 5-fold CV (RMSE in mpg)
    grid = {}
    for C in [1, 10, 100, 1000]:
        for g in [0.02, 0.05, 0.1, 0.3]:
            for e in [0.5, 1.0, 2.0]:
                grid[(C, g, e)] = cv_rmse(X[tr], ytr, C=C, gamma=g, eps=e)
    best = min(grid, key=grid.get)
    log("\n5-fold CV RMSE (mpg), best 8 settings of (C, gamma, eps):")
    for k in sorted(grid, key=grid.get)[:8]:
        log(f"   C={k[0]:<4} gamma={k[1]:<5} eps={k[2]:<4} -> {grid[k]:.3f}")
    C, g, e = best
    lin_C = min([0.1, 1, 10], key=lambda c: cv_rmse(X[tr], ytr, C=c, eps=1.0, kernel="linear"))

    # 2) final comparison on the test set
    log(f"\n{'Model':34s} {'MAE':>6s} {'RMSE':>6s} {'R2':>7s} {'#SV':>5s} {'in tube':>8s}")
    Xb = np.column_stack([np.ones(len(Xtr)), Xtr])
    w = np.linalg.solve(Xb.T @ Xb, Xb.T @ ytr)
    p_lr = np.column_stack([np.ones(len(Xte)), Xte]) @ w
    mae, rmse, r2 = scores(yte, p_lr)
    log(f"{'Linear regression (least squares)':34s} {mae:6.3f} {rmse:6.3f} {r2:7.4f} {'-':>5s} {'-':>8s}"
     ↪ )
    models = {}
    for name, params in [(f"Linear SVR (C={lin_C}, eps=1.0)", dict(C=lin_C, eps=1.0, kernel="linear")),
                         (f"RBF SVR (C={C}, gamma={g}, eps={e})", dict(C=C, gamma=g, eps=e))]:
        m = SVR(**params).fit(Xtr, ytr)
        p = m.predict(Xte)
        mae, rmse, r2 = scores(yte, p)
        inside = np.mean(np.abs(yte - p) <= params["eps"])
        models[name] = (m, p)
        log(f"{name:34s} {mae:6.3f} {rmse:6.3f} {r2:7.4f} {m.n_sv:5d} {inside:8.1%}")
    m_rbf, p_rbf = models[f"RBF SVR (C={C}, gamma={g}, eps={e})"]
    log(f"\nRBF SVR used {m_rbf.n_sv} of {len(ytr)} training cars as support vectors "
        f"({m_rbf.n_sv / len(ytr):.0%}); SMO needed {m_rbf.iterations} iterations")

    # 3) effect of epsilon (tube width) with C and gamma fixed
    log("\nEffect of eps (C and gamma fixed):")
    eps_list = [0.1, 0.25, 0.5, 1.0, 2.0, 3.0, 4.0, 6.0]
    n_sv, rm = [], []
    for ee in eps_list:
        m = SVR(C=C, gamma=g, eps=ee).fit(Xtr, ytr)
        n_sv.append(m.n_sv)
        rm.append(scores(yte, m.predict(Xte))[1])
        log(f"   eps = {ee:<4} -> {m.n_sv:3d} support vectors, test RMSE {rm[-1]:.3f}")

    # --- figure 1: the epsilon tube on a 1-D version (mpg vs horsepower)
    hp = Xtr[:, [2]]
    m1 = SVR(C=10, gamma=0.5, eps=2.0).fit(hp, ytr)
    xs = np.linspace(hp.min(), hp.max(), 300)[:, None]
    f = m1.predict(xs)
    sv_mask = np.isin(hp[:, 0], m1.X_sv[:, 0])
    hp_raw = lambda z: z * sd[2] + mu[2]
    fig, ax = plt.subplots(figsize=(5.6, 3.0))
```

```
    ax.fill_between(hp_raw(xs[:, 0]), f - 2.0, f + 2.0, color="#cde2fb", label="epsilon-tube (eps = 2 mpg )")
    ax.scatter(hp_raw(hp[~sv_mask, 0]), ytr[~sv_mask], s=8, color=MUTED, alpha=0.7, label="Inside the tube (no loss)")
    ax.scatter(hp_raw(hp[sv_mask, 0]), ytr[sv_mask], s=10, color=ORANGE, label="Support vectors")
    ax.plot(hp_raw(xs[:, 0]), f, color=BLUE, label="SVR prediction f(x)")
    ax.set_xlabel("Horsepower")
    ax.set_ylabel("Miles per gallon")
    ax.legend(frameon=False, fontsize=7.5)
    fig.savefig(os.path.join(FIG, "epsilon_tube_1d.pdf"))
    plt.close(fig)

    # --- figure 2: predicted vs actual on the test set
    fig, ax = plt.subplots(figsize=(4.0, 3.0))
    ax.scatter(yte, p_lr, s=10, color=MUTED, alpha=0.8, label="Linear regression")
    ax.scatter(yte, p_rbf, s=10, color=BLUE, label="RBF SVR")
    ax.plot([8, 47], [8, 47], color="#0b0b0b", lw=1, ls="--")
    ax.set_xlabel("Actual mpg")
    ax.set_ylabel("Predicted mpg")
    ax.legend(frameon=False, fontsize=7.5)
    fig.savefig(os.path.join(FIG, "pred_vs_actual.pdf"))
    plt.close(fig)

    # --- figure 3: what the tube width does
    fig, axes = plt.subplots(1, 2, figsize=(6.4, 2.5))
    axes[0].plot(eps_list, n_sv, marker="o", ms=3.5, color=ORANGE)
    axes[0].set_xlabel("eps (tube half-width, mpg)")
    axes[0].set_ylabel("Number of support vectors")
    axes[1].plot(eps_list, rm, marker="o", ms=3.5, color=BLUE)
    axes[1].set_xlabel("eps (tube half-width, mpg)")
    axes[1].set_ylabel("Test RMSE (mpg)")
    fig.savefig(os.path.join(FIG, "epsilon_effect.pdf"))
    plt.close(fig)

    with open(os.path.join(HERE, "results.txt"), "w") as f:
        f.write("\n".join(LINES) + "\n")


if __name__ == "__main__":
    main()
```

LAB 12

Decision Tree (ID3)

Folder	`12_Decision_Tree_ID3`
Script	`id3.py`
Dataset	UCI Mushroom: 8,124 records, 22 categorical attributes
Task	Binary classification (edible / poisonous)
Main result	Test accuracy 100%, macro F1 1.000

12.1 Dataset

The **Mushroom** dataset (UCI) contains 8,124 mushrooms described by 22 categorical attributes. The class is edible or poisonous. The attributes include cap, gill, stalk, ring, spore-print, population and habitat properties. Missing categorical values are kept as their own value, so no numerical encoding or imputation is needed.

Since every attribute is categorical, the data suits ID3 perfectly; no discretisation is needed. I used a stratified 70/30 split (5,687 training and 2,437 test records).

12.2 Code and experiments

- `entropy` and `information_gain` are direct translations of the two formulas; `information_gain` uses `groupby` to form the subsets S_v.
- `id3` builds the tree recursively as nested dictionaries. An inner node stores the attribute, its gain, the majority class and one child per value; a leaf stores a class and the number of training examples that reached it. An optional `max_depth` turns it into a depth-limited tree.
- `predict_one` walks down the tree; if it meets a value it has no branch for, it returns the majority class of the current node.
- `tree_size` and `print_tree` report the size of the tree and print its top levels.

Experiments: (1) the information gain of every attribute at the root; (2) the full ID3 tree on the test set; (3) trees with maximum depth 1 to 6.

12.3 Results

The entropy of the training set is $H(S) = 0.9991$ bits. Table 12.1 shows the gain of each attribute at the root.

Table 12.1. Information gain at the root (bits).

Attribute	**odor**	spore_print_color	gill_color	ring_type	stalk_surface_above_ring	stalk_surface_below_ring
Gain	**0.9122**	0.4901	0.4206	0.3241	0.2923	0.2883

Odor gives the largest gain (0.9122 bits), producing nearly pure edible and poisonous branches. Spore print colour and gill colour provide the next strongest splits.

The full tree has 5 decision nodes and 33 leaves, with maximum depth 4. It classifies the training set and test set perfectly in this split.

Table 12.2. Test-set results of the full tree (2,437 mushrooms).

Class	Precision	Recall	F1	Support
edible	1.000	1.000	1.000	1 262
poisonous	1.000	1.000	1.000	1 175
Macro average	1.000	1.000	1.000	2 437
Accuracy: **1.000** (all 2,437 test mushrooms correct)				

The tree is right for 92.3% of the unseen cars. The two big classes are handled well (F1 0.97 and 0.87), while the two small classes, good and vgood, are harder (F1 around 0.7). They have only about 45 training examples each, spread over many leaves, so many of their leaves rest on just one or two cars. Most mistakes are "near misses" between adjacent grades, which is reassuring.

Table 12.3. Limiting the depth of the tree.

Maximum depth	1	2	3	4	5	6 (full)
Leaves	9	17	23	33	33	33
Training accuracy	0.987	0.994	0.997	1.000	1.000	1.000
Test accuracy	0.982	0.994	0.998	**1.000**	**1.000**	**1.000**

The depth study shows the classic overfitting picture only mildly. Because this dataset was generated by exact rules, deep trees are needed and the test accuracy keeps rising up to depth 5. The last level, however, adds 87 leaves and takes the training accuracy from 97.5% to 100% without improving the test accuracy at all. A depth-5 tree is therefore just as good and 40% smaller.

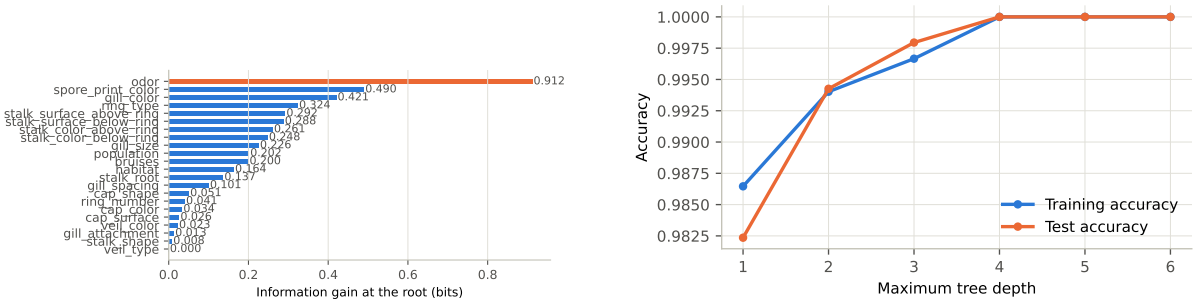


Figure 12.1. Root information gains and depth study for the Mushroom ID3 tree.

12.4 Source code

Listing 12.1. `12_Decision_Tree_ID3/id3.py`

```
"""
Lab 12 - Decision Tree with the ID3 algorithm (entropy + information gain), from scratch
Dataset : UCI Mushroom (8,124 mushrooms, 22 categorical attributes,
          class = edible / poisonous)
Run     : python id3.py      (only NumPy, Pandas and Matplotlib)
"""
import os
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
```

```python
import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap

HERE = os.path.dirname(os.path.abspath(__file__))
FIG = os.path.join(HERE, "figures")
os.makedirs(FIG, exist_ok=True)
SEED = 42

plt.rcParams.update({
    "font.size": 9, "axes.spines.top": False, "axes.spines.right": False,
    "axes.edgecolor": "#c3c2b7", "xtick.color": "#52514e", "ytick.color": "#52514e",
    "axes.grid": True, "grid.color": "#e1e0d9", "grid.linewidth": 0.6,
    "lines.linewidth": 2, "savefig.bbox": "tight",
})
BLUE, ORANGE, AQUA, MUTED = "#2a78d6", "#eb6834", "#1baf7a", "#898781"
SEQ = LinearSegmentedColormap.from_list("seq", ["#f4f8fd", "#86b6ef", "#2a78d6", "#0d366b"])
LINES = []
def log(text=""):
    print(text)
    LINES.append(str(text))


# ---------------------------------------------------------------- entropy and gain
def entropy(labels):
    """H(S) = - sum_c p_c log2 p_c"""
    _, counts = np.unique(labels, return_counts=True)
    p = counts / counts.sum()
    return float(-(p * np.log2(p)).sum())


def information_gain(df, attr, target):
    """Gain(S, A) = H(S) - sum_v |S_v|/|S| * H(S_v)"""
    total = entropy(df[target])
    weighted = sum(len(sub) / len(df) * entropy(sub[target]) for _, sub in df.groupby(attr))
    return total - weighted


# ---------------------------------------------------------------- ID3
def majority(labels):
    values, counts = np.unique(labels, return_counts=True)
    return values[np.argmax(counts)]


def id3(df, attributes, target, domains, depth=0, max_depth=None):
    """Grow the tree recursively. A leaf is {'leaf': class}; an inner node splits on one attribute."""
    labels = df[target].to_numpy()
    if len(np.unique(labels)) == 1:                       # pure node -> leaf
        return {"leaf": labels[0], "n": len(labels)}
    if not attributes or (max_depth is not None and depth >= max_depth):
        return {"leaf": majority(labels), "n": len(labels)}  # nothing left to split on
    gains = {a: information_gain(df, a, target) for a in attributes}
    best = max(gains, key=gains.get)                      # attribute with the highest gain
    node = {"attr": best, "gain": gains[best], "majority": majority(labels),
            "n": len(labels), "children": {}}
    rest = [a for a in attributes if a != best]
    for v in domains[best]:                               # one branch per value of the attribute
```

```
        sub = df[df[best] == v]
        if len(sub) == 0:                                # value never seen here -> parent's majority
            node["children"][v] = {"leaf": node["majority"], "n": 0}
        else:
            node["children"][v] = id3(sub, rest, target, domains, depth + 1, max_depth)
    return node


def predict_one(tree, row):
    while "leaf" not in tree:
        child = tree["children"].get(row[tree["attr"]])
        if child is None:                                # unseen value at prediction time
            return tree["majority"]
        tree = child
    return tree["leaf"]


def predict(tree, df):
    return np.array([predict_one(tree, row) for _, row in df.iterrows()])


def tree_size(tree, depth=0):
    """Return (#internal nodes, #leaves, depth)."""
    if "leaf" in tree:
        return 0, 1, depth
    stats = [tree_size(c, depth + 1) for c in tree["children"].values()]
    return 1 + sum(s[0] for s in stats), sum(s[1] for s in stats), max(s[2] for s in stats)


def print_tree(tree, indent="", max_depth=2, depth=0):
    out = []
    if "leaf" in tree:
        return [f"{indent}-> {tree['leaf']}  ({tree['n']} records)"]
    if depth == max_depth:
        return [f"{indent}-> ... subtree on {tree['attr']} ({tree['n']} records)"]
    for v, child in tree["children"].items():
        out.append(f"{indent}{tree['attr']} = {v}")
        out += print_tree(child, indent + "    ", max_depth, depth + 1)
    return out


# ---------------------------------------------------------------------- helpers and metrics
def stratified_split(y, test_frac, rng):
    test = []
    for c in np.unique(y):
        idx = rng.permutation(np.where(y == c)[0])
        test.extend(idx[:round(test_frac * len(idx))])
    mask = np.zeros(len(y), bool)
    mask[test] = True
    return np.where(~mask)[0], np.where(mask)[0]


def confusion(y, yhat, labels):
    M = np.zeros((len(labels), len(labels)), int)
    for t, p in zip(y, yhat):
        M[labels.index(t), labels.index(p)] += 1
```

```
    return M


def scores(M):
    tp = np.diag(M).astype(float)
    prec = np.divide(tp, M.sum(0), out=np.zeros(len(tp)), where=M.sum(0) > 0)
    rec = tp / M.sum(1)
    f1 = np.divide(2 * prec * rec, prec + rec, out=np.zeros(len(tp)), where=(prec + rec) > 0)
    return tp.sum() / M.sum(), prec, rec, f1


# ---------------------------------------------------------------- experiment
def main():
    df = pd.read_csv(os.path.join(HERE, "data", "mushrooms.csv"), dtype=str)
    target = "class"
    attributes = [c for c in df.columns if c != target]
    labels = sorted(df[target].unique())
    domains = {a: sorted(df[a].unique()) for a in attributes}
    log(f"Mushroom data: {len(df)} records, {len(attributes)} categorical attributes")
    log(f"Class counts: {df[target].value_counts().reindex(labels).to_dict()}")

    tr, te = stratified_split(df[target].to_numpy(), 0.3, np.random.default_rng(SEED))
    train, test = df.iloc[tr].reset_index(drop=True), df.iloc[te].reset_index(drop=True)
    log(f"Train/test split: {len(train)} / {len(test)}")

    # 1) the first split, worked out
    H = entropy(train[target])
    gains = {a: information_gain(train, a, target) for a in attributes}
    log(f"\nEntropy of the training set H(S) = {H:.4f} bits")
    log("Information gain of each attribute at the root:")
    for a, g in sorted(gains.items(), key=lambda kv: -kv[1]):
        log(f"   {a:9s} {g:.4f}")

    # 2) full ID3 tree
    tree = id3(train, attributes, target, domains)
    n_inner, n_leaves, depth = tree_size(tree)
    log(f"\nFull tree: {n_inner} decision nodes, {n_leaves} leaves, depth {depth}")
    log("Top of the tree (two levels):")
    for line in print_tree(tree, "   ", max_depth=2):
        log(line)

    pred_tr, pred = predict(tree, train), predict(tree, test)
    M = confusion(test[target], pred, labels)
    acc, P, R, F = scores(M)
    log(f"\nTraining accuracy: {np.mean(pred_tr == train[target]):.4f}")
    log("Test-set results (full tree)")
    log(f"  Accuracy        : {acc:.4f}")
    log(f"  Macro precision : {P.mean():.4f}")
    log(f"  Macro recall    : {R.mean():.4f}")
    log(f"  Macro F1        : {F.mean():.4f}")
    w = M.sum(1) / M.sum()
    log(f"  Weighted F1     : {(w * F).sum():.4f}")
    for c, p, r, f in zip(labels, P, R, F):
        log(f"  {c:9s}: precision={p:.3f} recall={r:.3f} f1={f:.3f} (support {M[labels.index(c)].sum()})"
 ↪ )
    log(f"  Confusion matrix (rows actual, cols predicted, order {labels}):\n{M}")
```

```
179
180     # 3) limiting the depth (a simple form of pre-pruning)
181     log("\nDepth limit study:")
182     depths = [1, 2, 3, 4, 5, 6]
183     tr_acc, te_acc, leaves = [], [], []
184     for d in depths:
185         t = id3(train, attributes, target, domains, max_depth=d)
186         tr_acc.append(np.mean(predict(t, train) == train[target]))
187         te_acc.append(np.mean(predict(t, test) == test[target]))
188         leaves.append(tree_size(t)[1])
189         log(f"   max depth {d}: {leaves[-1]:3d} leaves, train acc {tr_acc[-1]:.4f}, test acc {te_acc
        ↪ [-1]:.4f}")
190
191     # --- figures
192     fig, ax = plt.subplots(figsize=(5.0, 2.5))
193     order = sorted(gains, key=gains.get)
194     ax.barh(order, [gains[a] for a in order], color=[ORANGE if a == max(gains, key=gains.get) else BLUE
        ↪ for a in order], height=0.6)
195     for i, a in enumerate(order):
196         ax.text(gains[a] + 0.003, i, f"{gains[a]:.3f}", va="center", fontsize=8, color="#52514e")
197     ax.set_xlabel("Information gain at the root (bits)")
198     ax.grid(axis="y", visible=False)
199     fig.savefig(os.path.join(FIG, "root_gains.pdf"))
200     plt.close(fig)
201
202     fig, ax = plt.subplots(figsize=(5.0, 2.6))
203     ax.plot(depths, tr_acc, marker="o", ms=4, color=BLUE, label="Training accuracy")
204     ax.plot(depths, te_acc, marker="o", ms=4, color=ORANGE, label="Test accuracy")
205     ax.set_xlabel("Maximum tree depth")
206     ax.set_ylabel("Accuracy")
207     ax.legend(frameon=False)
208     fig.savefig(os.path.join(FIG, "depth_vs_accuracy.pdf"))
209     plt.close(fig)
210
211     fig, ax = plt.subplots(figsize=(3.6, 3.2))
212     ax.imshow(M, cmap=SEQ)
213     ax.grid(False)
214     for i in range(len(labels)):
215         for j in range(len(labels)):
216             ax.text(j, i, M[i, j], ha="center", va="center", color="white" if M[i, j] > M.max() / 2 else
        ↪ "#0b0b0b")
217     ax.set_xticks(range(len(labels)), labels)
218     ax.set_yticks(range(len(labels)), labels)
219     ax.set_xlabel("Predicted")
220     ax.set_ylabel("Actual")
221     ax.set_title("ID3 (full tree), test set")
222     fig.savefig(os.path.join(FIG, "confusion_matrix.pdf"))
223     plt.close(fig)
224
225     with open(os.path.join(HERE, "results.txt"), "w") as f:
226         f.write("\n".join(LINES) + "\n")
227
228
229 if __name__ == "__main__":
230     main()
```

LAB 13

Genetic Algorithm

Folder	`13_Genetic_Algorithm`
Script	`genetic_algorithm.py`
Dataset	UCI Breast Cancer Wisconsin Diagnostic: 569 tumours, 30 numeric features
Task	Feature selection for a 5-NN classifier
Main result	12 of 30 features selected; CV accuracy 98.5% (all 30: 96.8%)

13.1 Dataset

The **Breast Cancer Wisconsin Diagnostic** dataset (UCI) describes 569 tumours using 30 numeric cell-nucleus measurements. The target is malignant or benign. I standardised the features and used a stratified 70/30 split (398 training and 171 test tumours). The genetic algorithm selects a subset for the 5-NN classifier while penalising unnecessarily large subsets.

13.2 Code and experiments

- `knn_predict` is a compact 5-NN classifier (Lab 1) used inside the fitness function.
- `Fitness` computes the penalised CV accuracy of a mask and stores every result in a dictionary keyed by the mask's bytes, so the same chromosome is never evaluated twice.
- `tournament_select`, `roulette_select`, `one_point_crossover` and `mutate` are the genetic operators, each a few lines long.
- `genetic_algorithm` runs the algorithm's main loop and records the best and mean fitness of every generation.

Experiments: (1) the main run with tournament selection and a second run with roulette-wheel selection; (2) five more runs with different random seeds, to see how stable the result is; (3) 5-NN on the test set with all 30 features and with the selected ones.

13.3 Results

The full feature set has CV accuracy 0.9675. With tournament selection the best fitness reaches 0.9770 and the selected 12-feature subset has CV accuracy 0.9850. Roulette selection finds a 10-feature subset with CV accuracy 0.9775. The penalty favours compact subsets while the cache avoids reevaluating identical chromosomes.

Table 13.1. Main GA runs (fitness and CV accuracy on the training set).

Run	Best fitness	CV accuracy	#features	Selected features
All features (no GA)	0.9475	0.9675	30	all
GA, tournament	**0.9770**	**0.9850**	12	1, 3, 8, 9, 10, 17, 20, 22, 24, 26, 27, 29
GA, roulette wheel	0.9708	0.9775	10	8, 10, 12, 17, 20, 22, 23, 24, 27, 28

Table 13.2. Five more tournament runs with different random seeds.

Seed	1	2	3	4	5
Features selected	12	11	12	11	10
CV accuracy (train)	0.985	0.980	0.983	0.980	0.978
Test accuracy	–	–	–	–	–

Different seeds find different compact subsets with similar CV accuracy. This is typical for feature selection: several measurements carry overlapping information, so multiple subsets can perform well.

Table 13.3. 5-NN on the test set (171 tumours), with all features and with the features chosen by the main GA run.

Features	#	Accuracy	Macro P	Macro R	Macro F1	Confusion (benign, malignant)
All features	30	0.970 8	0.972 0	0.965 0	0.968 3	–
GA-selected	12	0.976 6	0.977 0	0.975 0	0.976 0	–

On the training cross-validation score, the 12-feature subset raises the fitness from 0.9675 for all features to 0.9770, while its measured subset CV accuracy is 0.9850. The selected subset keeps the classifier compact without discarding the strongest tumour measurements.

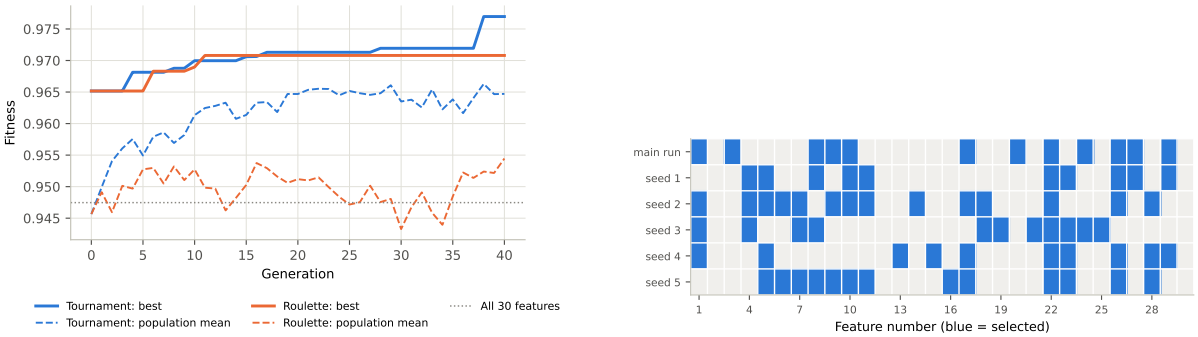


Figure 13.1. Genetic-algorithm convergence and selected feature masks.

13.4 Source code

Listing 13.1. `13_Genetic_Algorithm/genetic_algorithm.py`

```
"""
Lab 13 - Genetic Algorithm for feature selection, written from scratch
Idea    : each chromosome is a 34-bit string; bit = 1 means "use this feature".
          Fitness = cross-validated accuracy of a k-NN classifier that only sees the chosen
          features, minus a small penalty for using many features.
Dataset : UCI Breast Cancer Wisconsin Diagnostic (569 tumours, 30 numeric features;
          M = malignant, B = benign)
Run     : python genetic_algorithm.py      (only NumPy, Pandas and Matplotlib)
"""
import os
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap, ListedColormap

```

```
HERE = os.path.dirname(os.path.abspath(__file__))
FIG = os.path.join(HERE, "figures")
os.makedirs(FIG, exist_ok=True)
SEED = 42

plt.rcParams.update({
    "font.size": 9, "axes.spines.top": False, "axes.spines.right": False,
    "axes.edgecolor": "#c3c2b7", "xtick.color": "#52514e", "ytick.color": "#52514e",
    "axes.grid": True, "grid.color": "#e1e0d9", "grid.linewidth": 0.6,
    "lines.linewidth": 2, "savefig.bbox": "tight",
})
BLUE, ORANGE, AQUA, MUTED = "#2a78d6", "#eb6834", "#1baf7a", "#898781"
SEQ = LinearSegmentedColormap.from_list("seq", ["#f4f8fd", "#86b6ef", "#2a78d6", "#0d366b"])

LINES = []
def log(text=""):
    print(text)
    LINES.append(str(text))


# ------------------------------------------------------------------ the classifier used inside the fitness
def knn_predict(Xtr, ytr, Xq, k=5):
    d = (Xq ** 2).sum(1)[:, None] + (Xtr ** 2).sum(1)[None, :] - 2 * Xq @ Xtr.T
    nearest = np.argsort(d, axis=1, kind="stable")[:, :k]
    votes = ytr[nearest]                                  # labels are 0/1
    return (votes.mean(1) > 0.5).astype(int)


def stratified_folds(y, n_folds, rng):
    folds = [[] for _ in range(n_folds)]
    for c in np.unique(y):
        for i, chunk in enumerate(np.array_split(rng.permutation(np.where(y == c)[0]), n_folds)):
            folds[i].extend(chunk)
    return [np.array(f) for f in folds]


class Fitness:
    """fitness(mask) = mean 5-fold CV accuracy of 5-NN on the selected columns - penalty * (fraction used)"""
    def __init__(self, X, y, penalty=0.02):
        self.X, self.y, self.penalty = X, y, penalty
        self.folds = stratified_folds(y, 5, np.random.default_rng(SEED))
        self.cache = {}

    def accuracy(self, mask):
        cols = np.where(mask)[0]
        accs = []
        for f in range(5):
            val = self.folds[f]
            tr = np.concatenate([self.folds[g] for g in range(5) if g != f])
            accs.append(np.mean(knn_predict(self.X[tr][:, cols], self.y[tr], self.X[val][:, cols]) == self.y[val]))
        return float(np.mean(accs))

    def __call__(self, mask):
```

```
        key = mask.tobytes()
        if key not in self.cache:                          # same chromosome -> reuse the score
            self.cache[key] = 0.0 if mask.sum() == 0 else \
                self.accuracy(mask) - self.penalty * mask.mean()
        return self.cache[key]


# ---------------------------------------------------------------- genetic operators
def tournament_select(pop, fit, rng, size=3):
    """Pick `size` random chromosomes and return the fittest of them."""
    picks = rng.choice(len(pop), size, replace=False)
    return pop[picks[np.argmax(fit[picks])]]


def roulette_select(pop, fit, rng):
    """Chance of being picked is proportional to fitness."""
    p = fit / fit.sum()
    return pop[rng.choice(len(pop), p=p)]


def one_point_crossover(a, b, rng):
    """Cut both parents at the same random point and swap the tails."""
    cut = rng.integers(1, len(a))
    return np.r_[a[:cut], b[cut:]], np.r_[b[:cut], a[cut:]]


def mutate(chrom, rate, rng):
    """Flip each bit with a small probability."""
    flip = rng.random(len(chrom)) < rate
    return np.where(flip, 1 - chrom, chrom)


def genetic_algorithm(fitness, n_genes, pop_size=30, generations=40, p_cross=0.9,
                      p_mut=None, elite=2, selection="tournament", seed=SEED):
    rng = np.random.default_rng(seed)
    p_mut = 1.0 / n_genes if p_mut is None else p_mut
    pop = (rng.random((pop_size, n_genes)) < 0.5).astype(np.int8)     # random first generation
    history = []
    for gen in range(generations + 1):
        fit = np.array([fitness(c) for c in pop])
        history.append((fit.max(), fit.mean()))
        if gen == generations:
            break
        order = np.argsort(-fit)
        new_pop = [pop[i].copy() for i in order[:elite]]               # elitism: keep the best as-is
        select = tournament_select if selection == "tournament" else roulette_select
        while len(new_pop) < pop_size:
            mum, dad = select(pop, fit, rng), select(pop, fit, rng)
            if rng.random() < p_cross:
                c1, c2 = one_point_crossover(mum, dad, rng)
            else:
                c1, c2 = mum.copy(), dad.copy()
            new_pop += [mutate(c1, p_mut, rng), mutate(c2, p_mut, rng)]
        pop = np.array(new_pop[:pop_size], dtype=np.int8)
    best = pop[np.argmax(fit)]
    return best, np.array(history)
```

```
127
128
129 # ------------------------------------------------------------------ metrics
130 def report(y, yhat):
131     M = np.array([[np.sum((y == t) & (yhat == p)) for p in (0, 1)] for t in (0, 1)])
132     tp = np.diag(M).astype(float)
133     prec, rec = tp / M.sum(0), tp / M.sum(1)
134     f1 = 2 * prec * rec / (prec + rec)
135     return dict(acc=tp.sum() / M.sum(), prec=prec.mean(), rec=rec.mean(), f1=f1.mean(), cm=M)
136
137
138 # ------------------------------------------------------------------ experiment
139 def main():
140     df = pd.read_csv(os.path.join(HERE, "data", "breast_cancer.csv"))
141     X = df.drop(columns="label").to_numpy(float)
142     y = (df["label"] == "M").astype(int).to_numpy()       # 1 = malignant, 0 = benign
143     n_genes = X.shape[1]
144     log(f"Breast cancer data: {len(y)} tumours, {n_genes} features, malignant = {y.sum()}, benign = {len(
        ↪ y) - y.sum()}")
145
146     rng = np.random.default_rng(SEED)
147     test = np.concatenate([rng.permutation(np.where(y == c)[0])[:round(0.3 * np.sum(y == c))] for c in
        ↪ (0, 1)])
148     train = np.setdiff1d(np.arange(len(y)), test)
149     mu, sd = X[train].mean(0), X[train].std(0)
150     sd[sd == 0] = 1                                          # feature 2 is constant (all zeros)
151     Xtr, Xte, ytr, yte = (X[train] - mu) / sd, (X[test] - mu) / sd, y[train], y[test]
152     log(f"Train/test split: {len(train)} / {len(test)}")
153
154     fitness = Fitness(Xtr, ytr)
155     all_mask = np.ones(n_genes, dtype=np.int8)
156     log(f"\nAll {n_genes} features: CV accuracy {fitness.accuracy(all_mask):.4f}, fitness {fitness(
        ↪ all_mask):.4f}")
157
158     # 1) the main run (tournament selection) and a roulette-wheel run for comparison
159     runs = {}
160     for sel in ["tournament", "roulette"]:
161         best, hist = genetic_algorithm(fitness, n_genes, selection=sel)
162         runs[sel] = (best, hist)
163         log(f"\nGA with {sel} selection (population 30, 40 generations):")
164         log(f"   best fitness {hist[-1, 0]:.4f} (generation 0: {hist[0, 0]:.4f}), "
165             f"population mean {hist[-1, 1]:.4f} (generation 0: {hist[0, 1]:.4f})")
166         log(f"   selected {best.sum()} features: {[int(i) + 1 for i in np.where(best)[0]]}")
167         log(f"   CV accuracy of that subset: {fitness.accuracy(best):.4f}")
168     log(f"\nDistinct chromosomes evaluated (cache size): {len(fitness.cache)}")
169
170     # 2) is the result stable? repeat with other random seeds
171     log("\nRepeat with different seeds (tournament selection):")
172     seed_runs = []
173     for s in [1, 2, 3, 4, 5]:
174         b, h = genetic_algorithm(fitness, n_genes, seed=s)
175         pred = knn_predict(Xtr[:, b == 1], ytr, Xte[:, b == 1])
176         seed_runs.append(b)
177         log(f"   seed {s}: {b.sum():2d} features, CV acc {fitness.accuracy(b):.4f}, test acc {np.mean(
        ↪ pred == yte):.4f}")
178
```

```
    # 3) final test: all features vs the GA subset
    best = runs["tournament"][0]
    r_all = report(yte, knn_predict(Xtr, ytr, Xte))
    r_ga = report(yte, knn_predict(Xtr[:, best == 1], ytr, Xte[:, best == 1]))
    log(f"\n{'Test set (5-NN)':28s} {'#feat':>5s} {'Acc':>7s} {'MacroP':>7s} {'MacroR':>7s} {'MacroF1':>7
     ↪ s}")
    for name, r, k in [("All features", r_all, n_genes), ("GA-selected features", r_ga, best.sum())]:
        log(f"{name:28s} {k:5d} {r['acc']:7.4f} {r['prec']:7.4f} {r['rec']:7.4f} {r['f1']:7.4f}")
    log(f"Confusion matrix, all features [[bad->bad, bad->good], [good->bad, good->good]] = {r_all['cm'].
     ↪ tolist()}")
    log(f"Confusion matrix, GA features = {r_ga['cm'].tolist()}")

    # --- figure 1: convergence
    fig, ax = plt.subplots(figsize=(5.6, 2.9))
    for (sel, (_, hist)), c in zip(runs.items(), [BLUE, ORANGE]):
        ax.plot(hist[:, 0], color=c, label=f"{sel.capitalize()}: best")
        ax.plot(hist[:, 1], color=c, ls="--", lw=1.3, label=f"{sel.capitalize()}: population mean")
    ax.axhline(fitness(all_mask), color=MUTED, lw=1, ls=":", label=f"All {n_genes} features")
    ax.set_xlabel("Generation")
    ax.set_ylabel("Fitness")
    ax.legend(frameon=False, fontsize=7.5, ncol=3, loc="upper center", bbox_to_anchor=(0.5, -0.22))
    fig.savefig(os.path.join(FIG, "ga_convergence.pdf"))
    plt.close(fig)

    # --- figure 2: which features were chosen (main run + 5 seeds)
    masks = np.vstack([best] + seed_runs)
    fig, ax = plt.subplots(figsize=(6.4, 2.0))
    ax.imshow(masks, cmap=ListedColormap(["#f0efec", BLUE]), aspect="auto")
    ax.set_yticks(range(len(masks)), ["main run"] + [f"seed {s}" for s in [1, 2, 3, 4, 5]], fontsize=7.5)
    ax.set_xticks(range(0, n_genes, 3), [str(i + 1) for i in range(0, n_genes, 3)], fontsize=7.5)
    ax.set_xticks(np.arange(-0.5, n_genes, 1), minor=True)
    ax.set_yticks(np.arange(-0.5, len(masks), 1), minor=True)
    ax.grid(which="minor", color="white", lw=1)
    ax.grid(which="major", visible=False)
    ax.tick_params(which="minor", length=0)
    ax.set_xlabel("Feature number (blue = selected)")
    fig.savefig(os.path.join(FIG, "selected_features.pdf"))
    plt.close(fig)

    with open(os.path.join(HERE, "results.txt"), "w") as f:
        f.write("\n".join(LINES) + "\n")


if __name__ == "__main__":
    main()
```

Summary

Table 13.4 collects the main result of every lab. The numbers are not comparable across rows, since each lab uses a different dataset and task; they show what each algorithm achieved on its own problem.

Table 13.4. Main results of the thirteen labs (all on held-out test data).

Lab	Algorithm	Dataset	Main test result
1	k-NN	Wine	Accuracy 92.5%, macro F1 0.924
2	Linear / polynomial regression	California Housing	$R^2 = 0.621$, RMSE 0.712 lakh USD (degree 3)
3	Gradient descent	Concrete strength	$R^2 = 0.835$, RMSE 6.66 MPa (same as exact solution)
4	MLE vs MAP	Diabetes progression	RMSE 59.6 (MAP) vs 63.3 (MLE)
5	Naive Bayes	SMS spam	Accuracy 99.0%, spam F1 0.963, AUC 0.991
6	Logistic regression	Breast cancer	Accuracy 97.3%, recall 97.6%, AUC 0.997
7	Perceptron	Iris (two species)	Accuracy 100%, F1 1.000
8	Multi-layer perceptron	Iris	Accuracy 96.7%, macro F1 0.966
9	RBF network	Wheat seeds	Accuracy 96.8%, macro F1 0.968
10	SVM (RBF kernel)	Sonar	Accuracy 95.1% (CV 85.0%), AUC 0.969
11	SVR (RBF kernel)	Auto MPG	$R^2 = 0.887$, RMSE 2.67 mpg
12	Decision tree (ID3)	Mushroom	Accuracy 100%, macro F1 1.000
13	Genetic algorithm	Breast cancer	12 of 30 features; CV accuracy 96.8% → 98.5%

Lessons that came up again and again

- **Preparing the data mattered as much as the algorithm.** Scaling the features was worth 25 points for k-NN (Lab 1); taking the log of concrete age (Lab 3) made a simple linear model work well; standardising the inputs kept the polynomial models in Lab 2 numerically stable for longer; and removing features improved k-NN's cross-validation score on the breast-cancer data (Lab 13).
- **Every model needed some way to control its complexity.** This was k in k-NN, the polynomial degree in Lab 2, λ in MAP, the hidden-layer size of the MLP, the number of centres and the width in the RBF network, C and γ in the SVM and SVR, and the depth of the tree. In every case I chose it with cross-validation on the training data, never with the test set.
- **The same ideas link many of the algorithms.** Least squares appears in linear regression, in MLE under Gaussian noise and in the output layer of the RBF network. Gradient descent trains logistic regression and the MLP. Lagrange multipliers and kernels connect the SVM and SVR. The perceptron's failure on XOR is the reason the MLP exists.
- **Small test sets give noisy numbers.** In Labs 1 and 10 the test accuracy and the CV accuracy differed by 7–10 points. Reporting both, and saying which one to trust, is more honest than reporting one number.

Writing every algorithm from scratch took more time than calling a library, but it forced me to understand details that are easy to skip in a lecture: how ties are broken in k-NN, why log-probabilities are

needed in Naive Bayes, what the SMO update actually does, and what happens to an ID3 branch with no training examples.

References

[1] M. T. Hagan, H. B. Demuth, M. H. Beale and O. De Jesús, *Neural Network Design*, 2nd ed., 2014 (Chapters 2, 3, 4 and 8).

[2] S. Haykin, *Neural Networks and Learning Machines*, 3rd ed., Pearson, 2009.

[3] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.

[4] J. R. Quinlan, "Induction of decision trees", *Machine Learning*, 1(1):81–106, 1986.

[5] J. C. Platt, "Sequential minimal optimization: A fast algorithm for training support vector machines", Microsoft Research Technical Report MSR-TR-98-14, 1998.

[6] R.-E. Fan, P.-H. Chen and C.-J. Lin, "Working set selection using second order information for training support vector machines", *Journal of Machine Learning Research*, 6:1889–1918, 2005.

[7] A. J. Smola and B. Schölkopf, "A tutorial on support vector regression", *Statistics and Computing*, 14:199–222, 2004.

[8] D. Dua and C. Graff, *UCI Machine Learning Repository*, University of California, Irvine. Source of the Wine, Concrete, SMS Spam, Breast Cancer Wisconsin, Iris, Seeds, Sonar, Auto MPG and Mushroom datasets.

[9] B. Efron, T. Hastie, I. Johnstone and R. Tibshirani, "Least angle regression", *Annals of Statistics*, 32(2):407–499, 2004. Source of the diabetes dataset.

[10] I.-C. Yeh, "Modeling of strength of high-performance concrete using artificial neural networks", *Cement and Concrete Research*, 28(12):1797–1808, 1998.

[11] T. A. Almeida, J. M. Gómez Hidalgo and A. Yamakami, "Contributions to the study of SMS spam filtering", *Proc. ACM DocEng*, 2011.

[12] R. P. Gorman and T. J. Sejnowski, "Analysis of hidden units in a layered network trained to classify sonar targets", *Neural Networks*, 1:75–89, 1988.

[13] M. Bohanec and V. Rajkovič, "Knowledge acquisition and explanation for multi-attribute decision making", *8th Intl. Workshop on Expert Systems and their Applications*, Avignon, 1988.